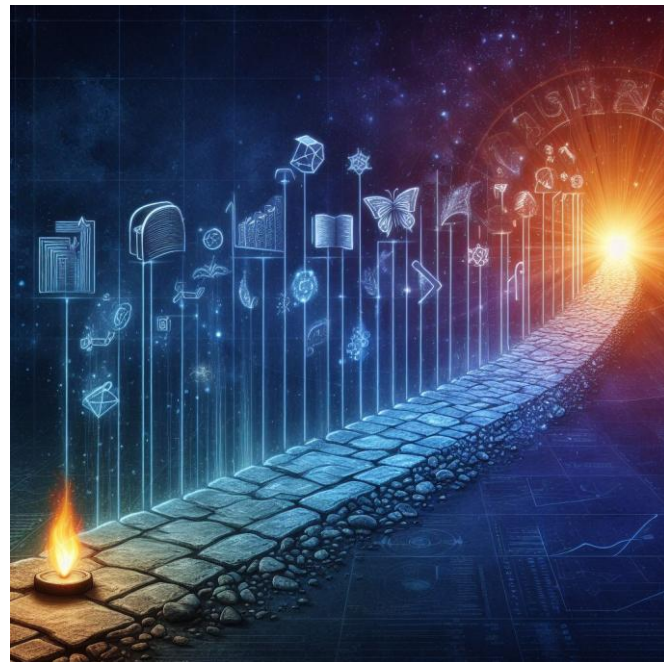


Spark V3 – formation avancée



Sommaire Apache Spark:

Part 1

1.1

Introduction à Apache Spark

- Historique de Spark
- Architecture logique
- RDD vs Dataframe vs Dataset
- Actions & Transformations
- RDD Lineage & Fault Tolerance
- Explication Closure

1.2

Use case & Tuning

- Opération RDD
- Java, Scala, Python & Apache Spark
- Opération DataFrame
- Analyse des Transactions Bancaires
- Détection de comportements suspects chez les clients bancaires
- Ventes par produit prioritaire

Sommaire Apache Spark:

Part 2

2.1 Développement Spark en production

- Spark Scala avec Maven
- Poetry & PySpark

2.2 Introduction à Spark SQL

- Batch Spark
- Apache Spark – Scala Use Case

2.3 Exploitation de Spark SQL

- Lecture de fichiers
- inferSchema
- Use Case Analyse des Données de Vols

Sommaire Apache Spark:

Part 3

3.1 Les 5S Apache Spark

3.2 Optimisation Allocation
mémoire

Sommaire Apache Spark:

Part 4

4.1 Optimisations Spark - Part 1

- Optimisation write data
- Configuration de KryoSerializer
- KryoRegistrator
- Partitionnement
- AQE & DPP

4.2 Optimisations Spark - Part 2

- cache & persist
- Shuffles
- Garbage Collector
- Broadcast et Accumulateurs
- Join

Sommaire Apache Spark:

Part 5

5.1

Overview Spark Shell & docker

- Prise en main SparkShell
- Introduction à Docker
- Introduction à Docker Compose

5.2

Overview Spark Delta Lake

- ACID
- Schema Enforcement & Evolution
- Time Travel
- Scaling Metastore
- Upserts & Deletes
- Indexing & Z-Ordering
- Batch & Streaming
- Lecture et d'Écriture
- Databricks
- Conversions de Tables
- Delta & Iceberg

Sommaire Apache Spark:

Part 5

5.3 Overview Monitoring : Yarn UI

- Yarn Overview
- Logs d'Application
- Diagnostic

5.4 Overview Paramètres et modalités d'exécution spark

- Mode de Déploiement
- Commandes et Options Générales

Sommaire Apache Spark:

Part 6

6.1 Overview Spark Streaming - Part 1

- Introduction Spark Streaming
- Structured Streaming
- Streaming DataFrame
- readStream
- Transformation
- window & Kafka
- writeStream
- foreachBatch
- Intégration

6.2 Overview Spark Streaming - Part 2

- Accumulateurs & Broadcast Variables
- Options de Streaming
- checkpointLocation
- Fault Tolerance
- Fréquence Trigger
- awaitTermination
- createOrReplaceTempView

1.1



Introduction à Apache Spark

Introduction à Apache Spark



1. Historique de Spark



2. Architecture logique

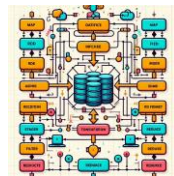


3. RDD vs Dataframe vs Dataset

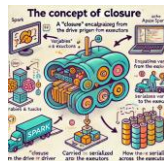
Wide Transformations (shuffles)
1 to N



4. Actions & Transformations



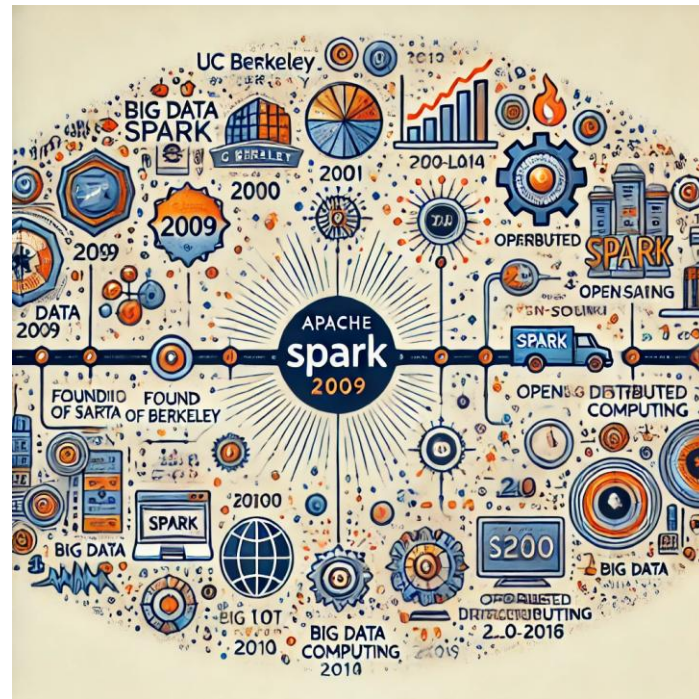
5. RDD Lineage & Fault Tolerance



6. Explication Closure

Introduction à Apache Spark

Historique de Apache Spark



Historique de Apache Spark

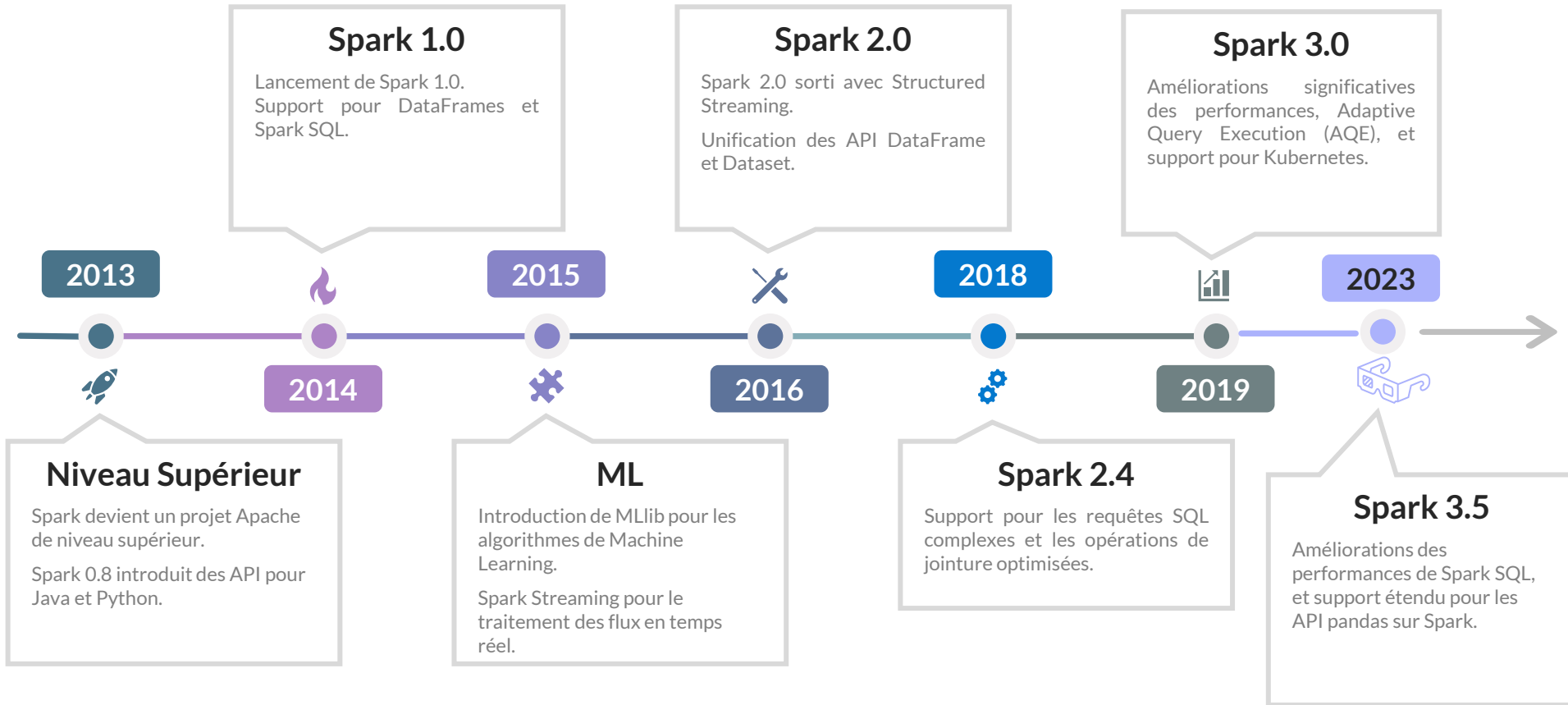
Introduction :

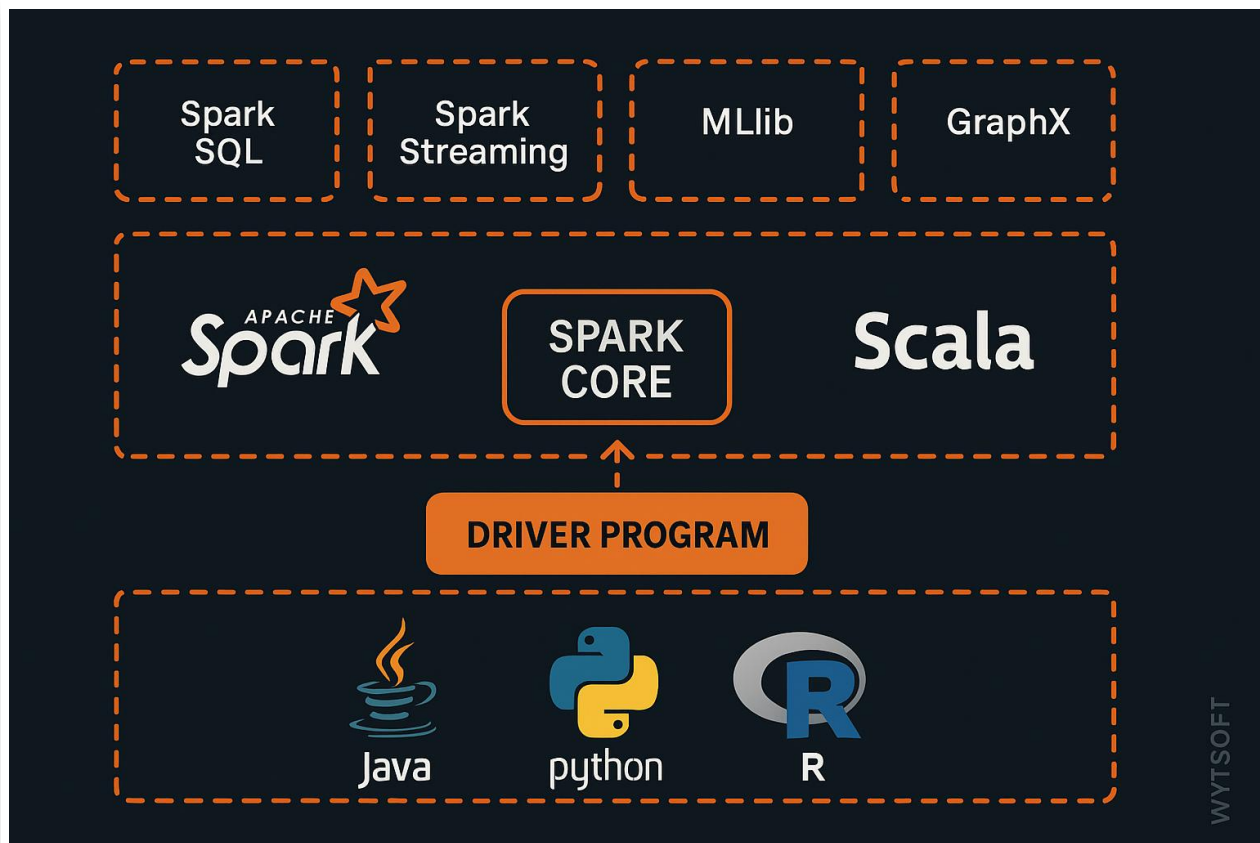
Apache Spark est un moteur pour le traitement de données à grande échelle, offrant des API en Java, Scala, Python et R. Il prend en charge Spark SQL, l'API pandas sur Spark, MLlib, GraphX et Structured Streaming.

Origines :

- **2010**: Spark est né au AMPLab de l'Université de Californie, Berkeley.
- Créé pour répondre aux limitations de Hadoop MapReduce en matière de vitesse et de facilité d'utilisation.

Historique de Apache Spark





Spark **Eco-System**

Responsabilités Clés de Spark Core



Gestion de la Mémoire



Scheduling



Récupération en Cas de Panne
(Fault Tolerance)



Distribution et
Surveillance des Jobs



Interaction avec les
Systèmes de Stockage

Composants Clés de Spark SQL



DataFrames



Datasets



Catalyst Optimizer



SparkSession

Composants Clés de Spark MLlib



Algorithmes de Machine Learning



Évaluation et Validation des Modèles



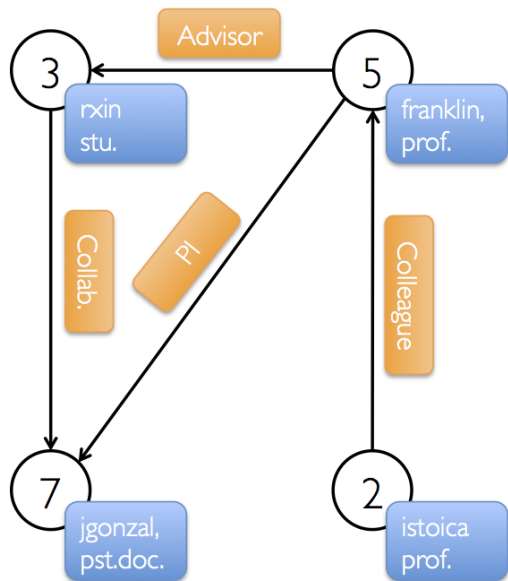
Pipelines de Machine Learning

Composants Clés de Spark Streaming



Composants Clés de Spark GraphX

Property Graph



Vertex Table

Id	Property (V)
3	(rxin, student)
7	(jgonzal, postdoc)
5	(franklin, professor)
2	(istoica, professor)

Edge Table

SrcId	DstId	Property (E)
3	7	Collaborator
5	3	Advisor
2	5	Colleague
5	7	PI

Graphes Dirigés et Non Dirigés

Property Graph

Triplets

Introduction à Apache Spark

Architecture Apache Spark

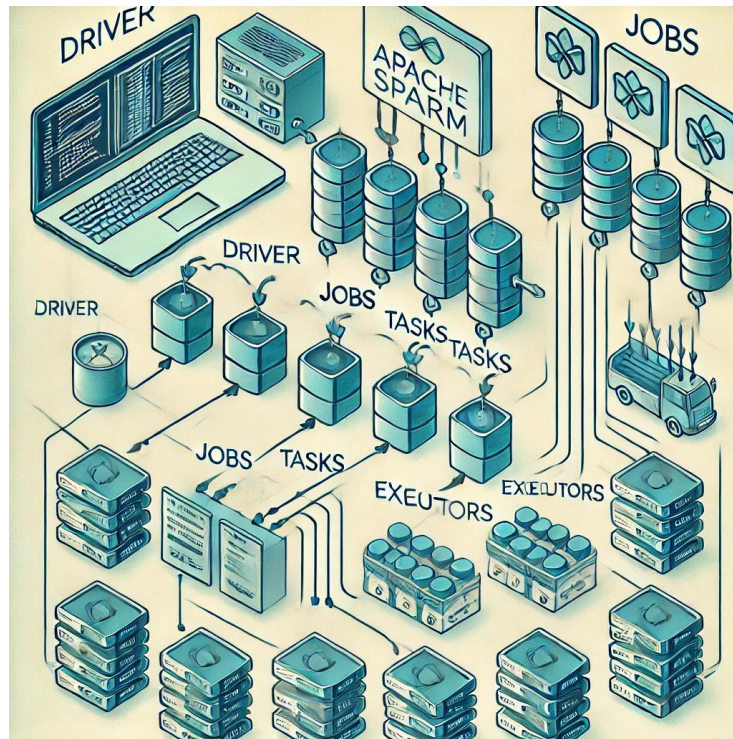
Driver

Executors

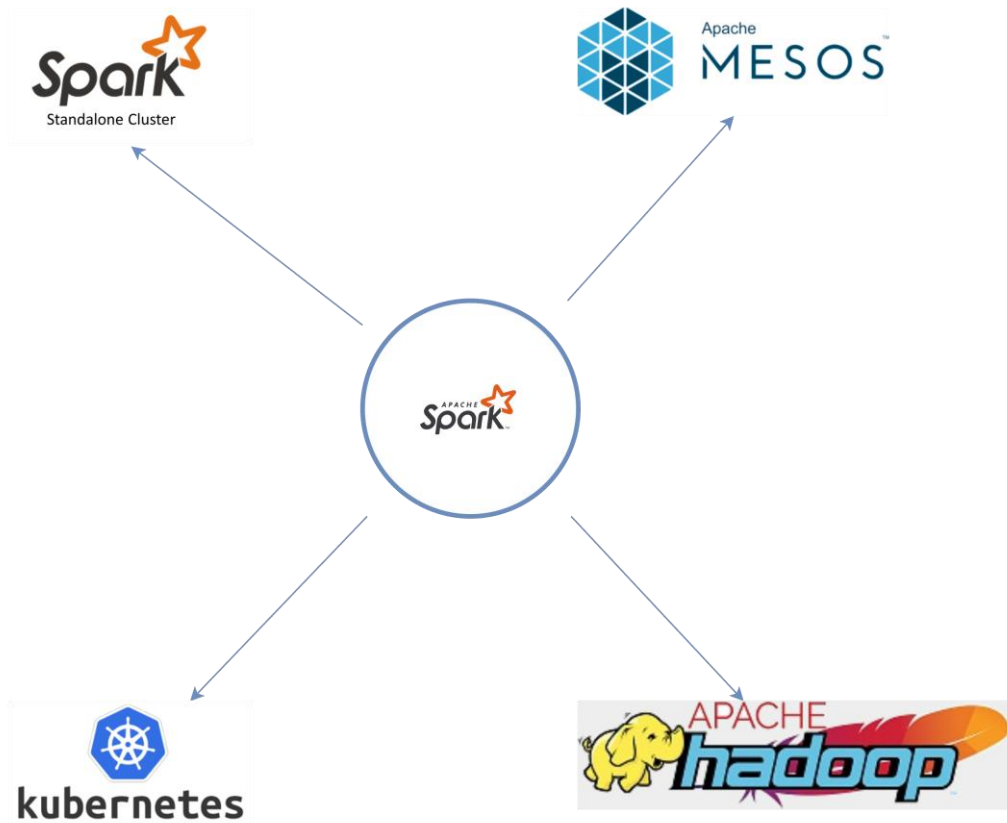
Tasks

worker node

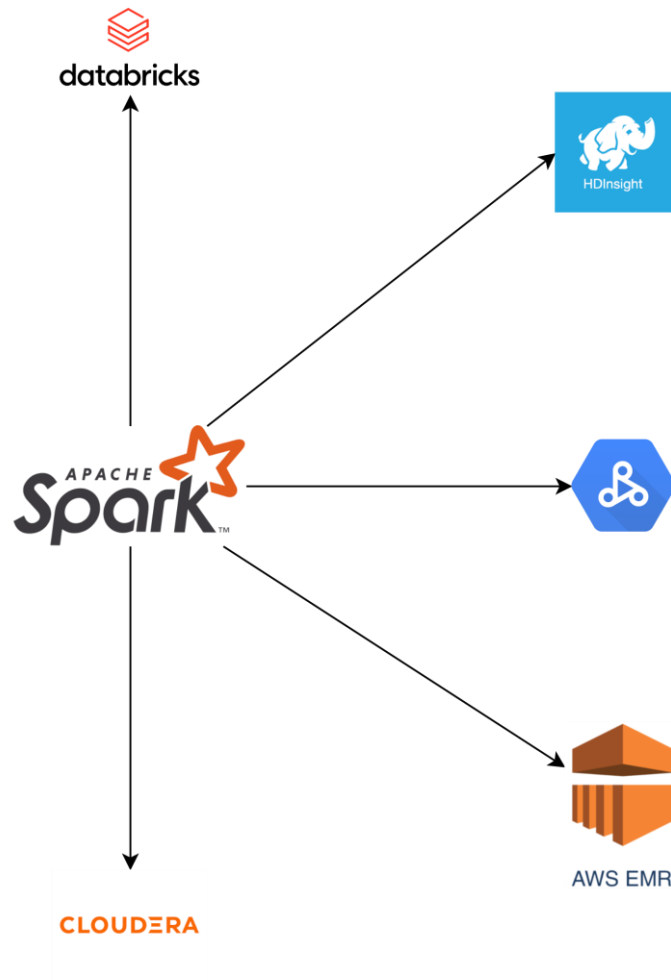
DAG (Directed Acyclic Graph)



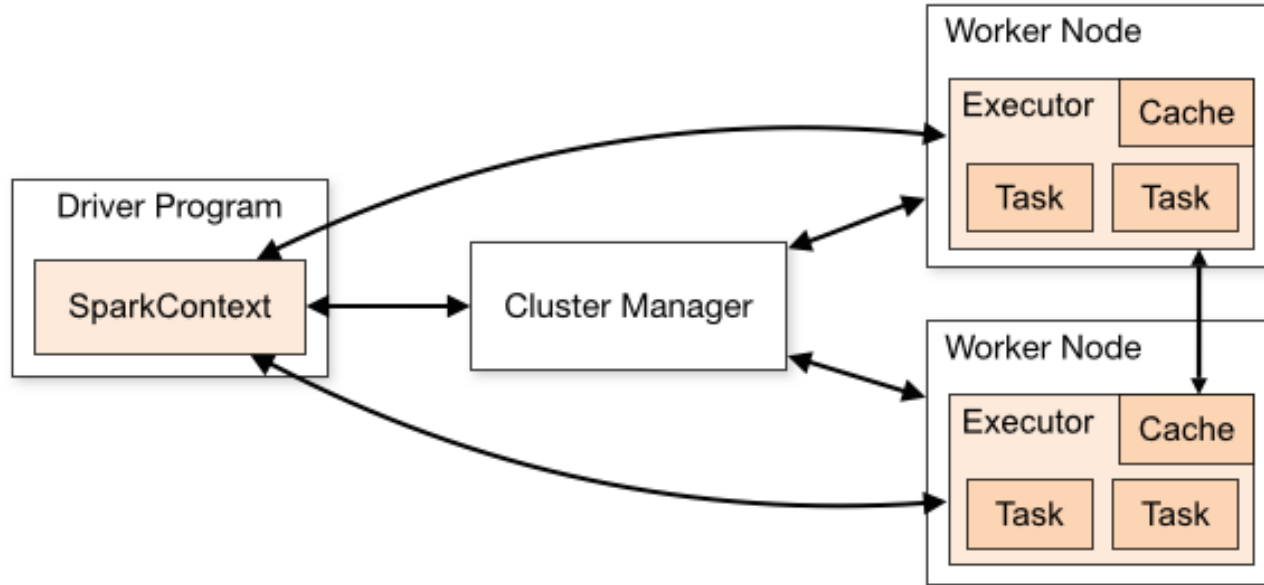
Les gestionnaires de clusters



Les plateformes pour Apache Spark



Vue d'Ensemble de l'Architecture Apache Spark



Rôle du Driver

Le Rôle du Driver

Définition : Le Driver est le processus central qui exécute le programme principal de votre application Spark.

Fonctions :

- Il transforme le programme Spark en tâches.
- Il maintient des informations sur le contexte Spark et les sessions utilisateur.

Driver & SparkUI



3.4.0-amzn-0

Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

Spark shell application UI

Executors

[Show Additional Metrics](#)

Summary

	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(7)	2556	156.6 GiB / 169.8 GiB	156.6 GiB / 169.8 GiB	0.0 B / 0.0 B	459.9 GiB	96	0	0	3104	3104	11.4 h (46 min)	127.6 GiB	62.4 KiB	62.4 KiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(7)	2556	156.6 GiB / 169.8 GiB	156.6 GiB / 169.8 GiB	0.0 B / 0.0 B	459.9 GiB	96	0	0	3104	3104	11.4 h (46 min)	127.6 GiB	62.4 KiB	62.4 KiB	0

Executors

Show entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Peak JVM Memory OnHeap / OffHeap	Peak Execution Memory OnHeap / OffHeap	Peak Storage Memory OnHeap / OffHeap	Peak Pool Memory Direct / Mapped	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
driver	ip-172-31-3-68.eu-west-1.compute.internal:41281	Active	0	66.9 KiB / 10.5 GiB	66.9 KiB / 10.5 GiB	0.0 B / 0.0 B	920.2 MiB / 274.8 MiB	0.0 B / 0.0 B	1012.2 KiB / 0.0 B	6.5 MiB / 0.0 B	0.0 B	0	0	0	0	0	18 min (0.8 s)	0.0 B	0.0 B	0.0 B		Thread Dump
1	ip-172-31-3-170.eu-west-1.compute.internal:46697	Active	473	25.8 GiB / 26.6 GiB	25.8 GiB / 26.6 GiB	0.0 B / 0.0 B	43.8 GiB / 142.3 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.6 GiB / 1.7 GiB	91.4 GiB	16	0	0	679	679	2.1 h (7.6 min)	22.5 GiB	0.0 B	10.6 KiB	stdout stderr	Thread Dump
2	ip-172-31-3-222.eu-west-1.compute.internal:44907	Active	466	26.2 GiB / 26.6 GiB	26.2 GiB / 26.6 GiB	0.0 B / 0.0 B	40.6 GiB / 141.5 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.3 GiB / 1.8 GiB	89.1 GiB	16	0	0	701	701	2.1 h (7.3 min)	23.9 GiB	0.0 B	11.4 KiB	stdout stderr	Thread Dump
3	ip-172-31-3-196.eu-west-1.compute.internal:39167	Active	473	26.2 GiB / 26.6 GiB	26.2 GiB / 26.6 GiB	0.0 B / 0.0 B	41.6 GiB / 141.5 MiB	0.0 B / 0.0 B	26.5 GiB / 0.0 B	2.2 GiB / 1.1 GiB	91.4 GiB	16	0	0	680	680	2.1 h (8.0 min)	22.7 GiB	0.0 B	10.9 KiB	stdout stderr	Thread Dump
4	ip-172-31-3-36.eu-west-1.compute.internal:46023	Active	482	26.1 GiB / 26.6 GiB	26.1 GiB / 26.6 GiB	0.0 B / 0.0 B	42.6 GiB / 144.3 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	2.2 GiB / 1.7 GiB	94.3 GiB	16	0	0	674	674	2.1 h (9.2 min)	23.9 GiB	62.4 KiB	11.4 KiB	stdout stderr	Thread Dump
5	ip-172-31-3-244.eu-west-1.compute.internal:43799	Active	317	26.3 GiB / 26.6 GiB	26.3 GiB / 26.6 GiB	0.0 B / 0.0 B	41.8 GiB / 131.7 MiB	0.0 B / 0.0 B	26.6 GiB / 0.0 B	1.7 GiB / 754 MiB	44.3 GiB	16	0	0	178	178	1.4 h (7.7 min)	16.8 GiB	0.0 B	8.7 KiB	stdout stderr	Thread Dump
6	ip-172-31-3-39.eu-west-1.compute.internal:43111	Active	345	26 GiB / 26.6 GiB	26 GiB / 26.6 GiB	0.0 B / 0.0 B	43.6 GiB / 130.3 MiB	0.0 B / 0.0 B	26.5 GiB / 0.0 B	1.9 GiB / 1.1 GiB	49.5 GiB	16	0	0	192	192	1.4 h (6.2 min)	17.8 GiB	0.0 B	9.4 KiB	stdout stderr	Thread Dump

Showing 1 to 7 of 7 entries

Previous **1** Next

Rôle du Executors

Les Executors

Définition : Les Executors sont des processus distribués responsables de l'exécution des tâches assignées par le Driver.

Fonctions :

- Exécutent les calculs et stockent les données pour les applications Spark.
- Chaque Executor est un processus JVM distinct.

Show Additional Metrics

- ☐ Select All
- ☐ On Heap Memory
- ☐ Off Heap Memory

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Blacklisted
Active(3)	0	5.9 KiB / 1.1 GiB	0.0 B	2	0	0	5	5	4 s (0.2 s)	0.0 B	0.0 B	0.0 B	0
Total(3)	0	5.9 KiB / 1.1 GiB	0.0 B	2	0	0	5	5	4 s (0.2 s)	0.0 B	0.0 B	0.0 B	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0

Executors

Show20entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
1	10.12.221.27:55834	Active	0	2 KiB / 366.3 MiB	0.0 B	1	0	0	3	3	2 s (0.1 s)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump
0	10.12.221.27:55835	Active	0	2 KiB / 366.3 MiB	0.0 B	1	0	0	2	2	2 s (94.0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump
driver	10.12.221.27:55827	Active	0	2 KiB / 366.3 MiB	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B		Thread Dump

Rôle du DAG (Directed Acyclic Graph)

Le DAG (Directed Acyclic Graph)

Définition : Un « DAG » est un graphe des opérations à effectuer, où les nœuds représentent les opérations, et les arêtes représentent les dépendances entre ces opérations.

Catalyst Optimizer

Fonctionnement :

- Spark crée un « DAG » pour chaque job, permettant une optimisation et une planification efficace des « Tasks ».
- Permet de récupérer partiellement en cas de défaillance.

Details for Job 0

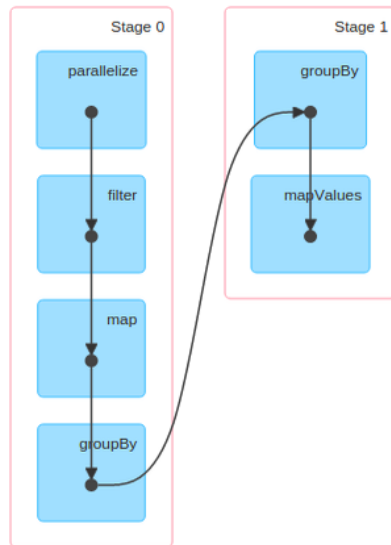
Status: RUNNING

Active Stages: 1

Pending Stages: 1

▶ Event Timeline

▼ DAG Visualization



Stage

Active Stages (1)

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read
0	groupBy at DAGTest.java:53 (kill) +details	2016/08/24 12:45:38	Unknown	0/2			

DAG
&
SparkUI

Introduction à Apache Spark

RDD *vs* DataFrame *vs* Dataset



RDD vs DataFrame

Abstraction

Collection distribuée
d'objets Java/Scala



Type de Données

N'importe quel type
d'objet Java/Scala



API

Disponible en Java,
Scala, Python



Abstraction

Collection distribuée de
données tabulaires
organisées en colonnes



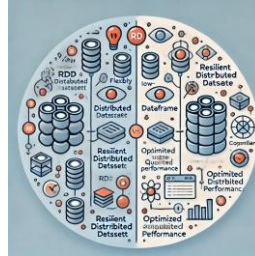
Type de Données

Données structurées
avec schéma



API

Disponible en Java,
Scala, Python



RDD vs DataFrame

RDD

vs

DataFrame

Non

Schéma

Oui, implicite

Moins optimisé

Performances

Optimisé (Catalyst Optimizer)

Non

Interrogation
SQL

Oui (via Spark SQL)

Manuelle

Optimisation

Automatique (Catalyst
Optimizer)

RDD vs DataFrame

01

Sérialisation

Coûteuse (Java/Scala serialization)

02

Fonctions Avancées

Supporte les transformations et actions basiques (map, filter, reduce)

RDD



DataFrame

01

Sérialisation

Moins coûteuse (optimisé pour le stockage en colonnes)

02

Fonctions Avancées

Supporte des opérations plus complexes et des agrégations (select, groupBy, join)



DataFrame vs Dataset



DataFrame

01

Utilisation

Idéal pour les opérations SQL et le traitement des données structurées

02

Schéma & Type

Implicite à l'exécution

03

API

Disponible en Java, Scala, Python, R



Dataset

01

Utilisation

Idéal pour les opérations typées et le traitement des données structurées et semi-structurées

02

Schéma & Type

Explicite à la compilation

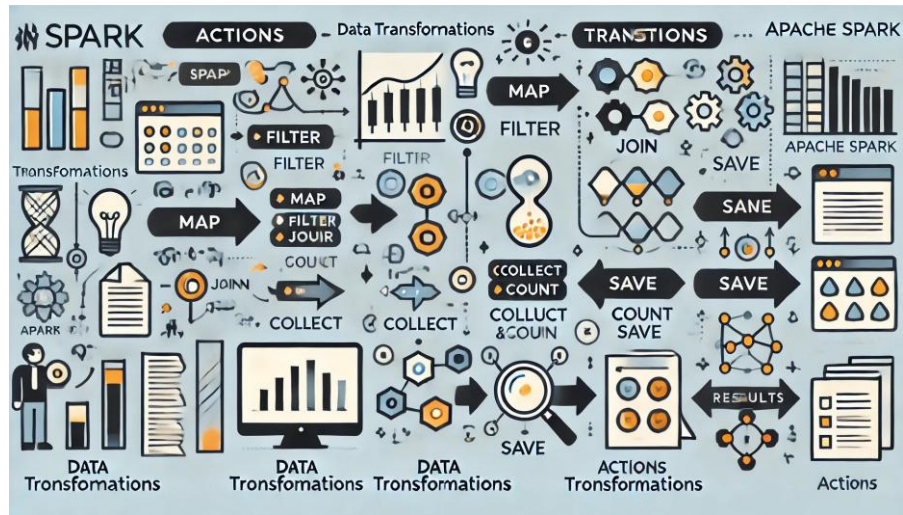
03

API

Disponible en Java et Scala

Introduction à Apache Spark

Actions & Transformations



Actions

- Les **actions** en Apache Spark déclenchent l'exécution des transformations précédentes et renvoient un résultat au driver.
 - **count()** : Compte le nombre d'éléments dans le DataFrame/RDD.
 - **show()** : Affiche les premières lignes du DataFrame.
 - **saveAsTextFile(path)** : Sauvegarde les données du RDD en tant que fichier texte à l'emplacement spécifié.
 - **write** : Écrit les données du DataFrame dans un format spécifié (parquet, csv, json, etc.) à un emplacement donné.

Transformation

Toute opération qui **transforme** un DataFrame/RDD en un autre. Étant donné que les objets DataFrame/RDD sont immuables, ces opérations aboutissent à un **nouveau** DataFrame.

Exemples de Transformations

1. select(cols)

- **Description** : Sélectionne un ensemble de colonnes dans un DataFrame.
- **Exemple** :

```
val selectedDf = df.select("name", "age")
```

2. join(df2, joinExprs)

- **Description** : Joint deux DataFrames sur des colonnes spécifiées.
- **Exemple** :

```
val joinedDf = df1.join(df2, df1("id") === df2("id"))
```

Types Transformations

Il faut savoir qu'ils existent deux types de transformations :

- ***Narrow Transformations (étroites)***
- ***Wide Transformations (Larges)***

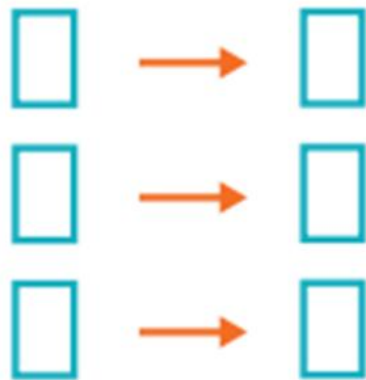
Narrow Transformations

Sont des transformations où chaque **partition** des données de sortie dépend d'une **seule** partition des données d'entrée.

map(func) : Applique une fonction à chaque élément du RDD/DataFrame.

```
val rdd1 = sc.parallelize(Seq(1, 2, 3, 4))  
val rdd2 = rdd1.map(x => x * 2)
```

Narrow Transformations
1 to 1



Wide Transformations

sont des transformations où chaque **partition** des données de sortie peut dépendre de **plusieurs** partitions des données d'entrée.

agg(agggregationExprs) : Applique des fonctions d'agrégation à un DataFrame, ce qui peut impliquer un shuffle pour regrouper les données en fonction des clés d'agrégation.

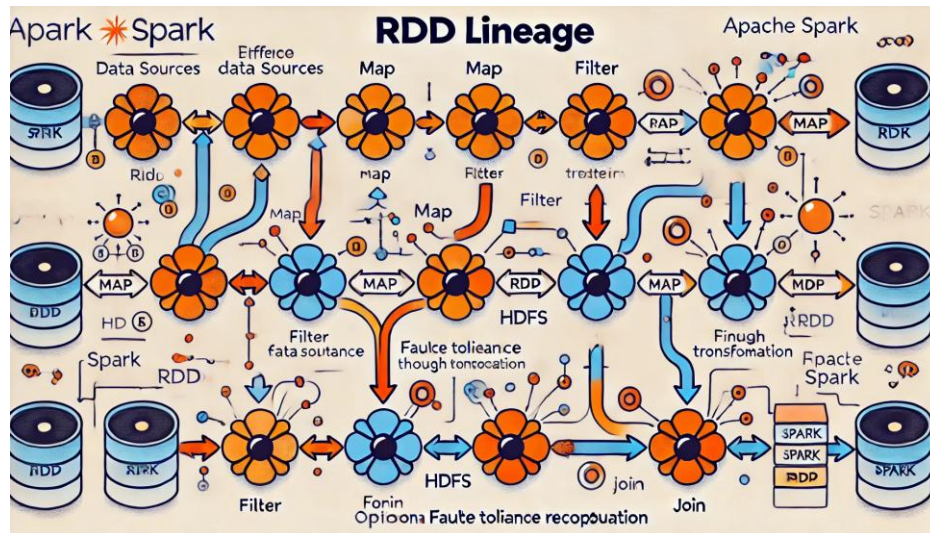
```
val aggregatedDf = df.groupBy("category").agg(sum("value"))
```

Wide Transformations (shuffles)
1 to N



Introduction à Apache Spark

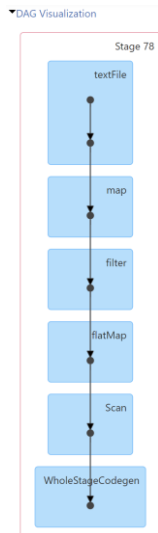
RDD Lineage & Fault Tolerance



RDD Lineage

RDD **Lineage** est un mécanisme de suivi des transformations appliquées à un RDD pour arriver à un RDD **final**.

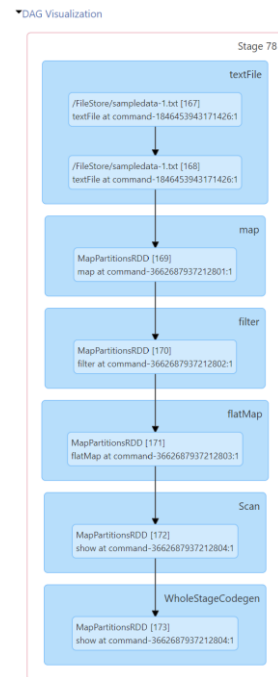
DAG



Fault Tolerance



Optimisation DAG



Fault Tolerance



- RDD (Resilient Distributed Dataset)
- Recalcul et Ré-exécution des Tâches
- Tasks Idempotentes



- Gestion de la Mémoire et Fault **Tolerance**
 - ❖ RDD en Mémoire
 - ❖ Persisting et Caching



- Fault **Tolerance** au Niveau des Jobs et des Stages
 - ❖ Stages & Tasks
 - ❖ Checkpointing

Closure



Closure



Sérialisation des Closures



Problèmes
d'Encapsulation



Performance

Example of a closure

Ici, Spark doit envoyer la valeur du facteur à chaque exécuteur → c'est cela qu'on appelle la *closure*.



```
factor = 5
```

```
rdd = sc.parallelize([1,2,3,4])
```

```
result = rdd.map(lambda x: x * factor) # "factor" is inside closure
```

Use Case : Détection de Transactions Frauduleuses en Temps Réel dans le Domaine Bancaire

Supposons que vous avez un ensemble de données contenant des transactions avec des informations telles que :

- Montant de la transaction
- Type de carte (crédit/débit)
- Pays d'origine de la transaction
- Heure de la transaction

Vous disposez également d'une liste de seuils de risque externes que vous souhaitez appliquer à chaque transaction pour calculer un score de risque.

- https://github.com/WyTaSoft/wytaSoft_training_academy_data/tree/features/uc_input_data

Use Case : Détection de Transactions Frauduleuses en Temps Réel dans le Domaine Bancaire

- **Solution avec Closure en PySpark :**
- **Données :** Une liste de transactions contenant des informations de base sur chaque transaction.
- **Facteur de risque :** Un dictionnaire contenant des seuils de risque externes basés sur le type de carte ou le pays d'origine, que vous utiliserez pour calculer un score de risque pour chaque transaction.
- **Objectif :** Utiliser PySpark pour appliquer les facteurs de risque à chaque transaction et signaler les transactions avec un score de risque élevé.

```
# Données de transactions
transactions = [
    {"id": 1, "montant": 500, "carte": "crédit", "pays": "US", "heure": "10:00"},
    {"id": 2, "montant": 2000, "carte": "débit", "pays": "FR", "heure": "15:30"},
    {"id": 3, "montant": 1500, "carte": "crédit", "pays": "IN", "heure": "11:00"},
    {"id": 4, "montant": 300, "carte": "débit", "pays": "US", "heure": "09:45"}
]

# Paralléliser les transactions
rdd = sc.parallelize(transactions)

# Seuils de risque externes (closure)
facteurs_risque = {
    "crédit": {"seuil_montant": 1000, "risque": 1.2},
    "débit": {"seuil_montant": 500, "risque": 1.5},
    "US": {"risque": 1.1},
    "IN": {"risque": 2.0},
    "FR": {"risque": 1.0}
}
```

.....
.....
.....
.....



```
# Fonction qui calcule le score de risque pour chaque transaction
def calculer_risque(transaction):
    carte = transaction["carte"]
    pays = transaction["pays"]
    montant = transaction["montant"]
```

Use Case : Détection de Transactions Frauduleuses en Temps Réel dans le Domaine Bancaire

- **Objectif** : Utiliser PySpark pour appliquer les facteurs de risque à chaque transaction et signaler les transactions avec un score de risque élevé.

Quiz : Introduction à Apache Spark

<https://wytasoft.com/quizz/introduction-apache-spark>



1.3

Use case & Tuning



Use case & Tuning



3. Opération RDD

1. w001 - Analyse des Transactions Bancaires



4. Java, Scala, Python & Apache Spark

Update icon

2. w002 - Détection de comportements suspects chez les clients bancaires



5. Opération DataFrame

3. w003 - ventes par produit prioritaire

Opération RDD



Opération **RDD**

- **Use Case : Analyse des Transactions et Gestion des Comptes avec PySpark**
 1. Création et Affichage de RDD : Analyse des Transactions
 2. Opérations sur les RDD : Prise d'Éléments et Comptage
 3. Exploration de l'Immutabilité et de la Transformation des RDD
 4. Jointure et Tri des RDD : Associer Comptes et Transactions
 5. Débogage des RDD avec `toString()`

Création et Affichage de RDD : Analyse des Transactions

- Nous avons deux jeux de données :
- **Transactions** : Une liste de transactions où chaque ligne contient l'ID de la transaction, l'ID du client, le montant, et la date de la transaction.
- **Comptes Clients** : Une liste contenant les informations des comptes clients (ID du client, nom, et type de compte).

```
# Données des transactions
transactions = [
    (1, 1001, 500, "2023-09-01"),
    (2, 1002, 1500, "2023-09-01"),
    (3, 1001, 250, "2023-09-02"),
    (4, 1003, 700, "2023-09-03")
]

# Données des comptes clients
comptes = [
    (1001, "Alice", "Épargne"),
    (1002, "Bob", "Courant"),
    (1003, "Charlie", "Courant")
]
```

Création et Affichage de **RDD** : Analyse des Transactions

- **sc.parallelize()** : La fonction `parallelize()` est une méthode de `SparkContext (sc)` utilisée pour créer un RDD.

```
# Création des RDD à partir des données
rdd_transactions =
sc.parallelize(transactions)
rdd_comptes = sc.parallelize(comptes)

# Affichage du contenu des RDDs
print("Transactions RDD:")
rdd_transactions.collect()

print("Comptes Clients RDD:")
rdd_comptes.collect()
```



```
# Affiche chaque élément du RDD dans les partitions  
rdd_transactions.foreach(lambda x: print(x))
```

Création et Affichage de **RDD** : Analyse des Transactions

- Pour afficher les éléments d'un **RDD** sans utiliser `collect()` dans PySpark,
- Qui n'agrège pas tous les éléments du RDD sur le driver

Opérations sur les RDD : Prise d'Éléments et Comptage

Prendre des éléments : Utilisez une fonction pour afficher les trois premières transactions.



```
print("Premières transactions :")  
print(rdd_transactions.take(3))
```

Opérations sur les RDD : Exploration de l'Immutabilité et de la Transformation des RDD

Filtrer les transactions
supérieures à 500 :

```
● ● ●  
rdd_transactions_grandes = rdd_transactions.filter(lambda t: t[2] > 500)  
  
print("Transactions de plus de 500 :")  
rdd_transactions_grandes.collect()
```

Jointure et Tri des RDD : Associer Comptes et Transactions

Nous voulons lier les transactions avec les informations de comptes en utilisant une **jointure**

```
# Jointure basée sur l'ID client  
rdd_jointure = rdd_transactions.map(lambda t: (t[1], t)).join(rdd_comptes)
```

Tri par montant de transaction :

```
rdd_triee = rdd_jointure.sortBy(lambda x: x[1][0][2], ascending=False)
```




```
print(rdd_triee.toDebugString())
```

Debug des RDD

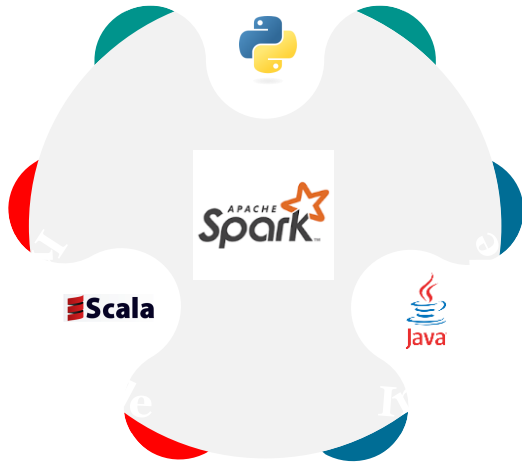
Afficher le plan d'exécution

Use case & Tuning

Java, Scala, Python & Apache Spark



Java, Scala, Python & Apache Spark



Java

Convient aux **développeurs Java expérimentés** qui souhaitent intégrer Spark dans un environnement d'entreprise, avec une bonne performance, mais une syntaxe plus verbeuse



Scala

Meilleur choix pour les **développeurs Spark natifs** qui cherchent des performances maximales, une syntaxe concise et un accès complet à toutes les fonctionnalités de **Spark**.



Python

Idéal pour les **data scientists** et les tâches d'analyse de données grâce à sa simplicité et son écosystème riche en bibliothèques.

Java, Scala, Python & Apache Spark



Datasets

Spark propose des Datasets, qui sont des collections distribuées similaires aux RDDs

Typed Transformations

Les transformations typées sont disponibles avec les Datasets en Scala et Java.

Serialization

Possibilité de passer à la sérialisation Kryo, qui est beaucoup plus rapide et plus efficace en termes d'utilisation de la mémoire

```
case class Transaction(id: Int, amount: Double)
val ds: Dataset[Transaction] = spark.read.json("transactions.json")
ds.filter(_.amount > 500).show()
```



Typed Transformations

```
val ds: Dataset[String] = spark.read.textFile("file.txt")
val lengthDs: Dataset[Int] = ds.map(line => line.length)
```

Use case & Tuning

Opération
DataFrame



Création d'un DataFrame à partir d'un fichier CSV

- Dans certains cas, il est nécessaire de créer un schéma explicite lors de la lecture de fichiers dans Spark

```
DataFrame

# Lire le fichier CSV avec le schéma explicite
df = spark.read.csv("path/to/transactions.csv", header=True, schema=schema)
# Afficher le schéma du DataFrame pour vérifier
df.printSchema()
# Afficher les premières lignes du DataFrame
df.show(5)
```

```
DataFrame

# Définir un schéma explicite
schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("client_id", IntegerType(), True),
    StructField("montant", DoubleType(), True),
    StructField("date", StringType(), True)
])
```

Filtrage et regroupement des données dans un DataFrame

- Appliquer un **filtrage** pour extraire des sous-ensembles spécifiques de données, puis utiliser un **groupBy** pour agréger les données par certaines colonnes.

```
DataFrame

# Filtrer les transactions dont le montant est
# supérieur à 500
df_filtre = df.filter(df["montant"] > 500)
# Afficher les transactions filtrées
df_filtre.show()
```

```
DataFrame

# Regrouper par client_id et calculer la somme des montants des transactions
df_grouped = df.groupBy("client_id").sum("montant")
# Renommer la colonne résultante
df_grouped = df_grouped.withColumnRenamed("sum(montant)", "total_montant")
# Afficher le résultat
df_grouped.show()
```

Traitement de données dans un DataFrame

- **Objectif** : Pour chaque client et par type de compte, calculer le nombre de transactions, le montant total et la moyenne mensuelle.
- Fonction Spark : regroupements (groupBy), agrégations (agg), jointure SQL entre comptes et transactions.

Use case « spark it » start - w001 - Analyse des Transactions Bancaires

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._

object NonOptimizedJob {
  def main(args: Array[String]): Unit = {

    val df = spark.read
      .format("csv")
      .option("header", "true")
      .load("transactions.csv")

    val filteredDf = df.filter(col("amount") != "")
    val castedDf = filteredDf
      .withColumn("amount", col("amount").cast("double"))

    val groupedDf = castedDf
      .groupBy("client_id", "transaction_type")
      .agg(sum("amount").as("total_amount"))

    groupedDf.write
      .format("parquet")
      .save("output/transactions_by_client")
  }
}
```

Le bon nombre de coalesce(n)

La méthode pour déterminer le bon nombre de coalesce(n) (ou repartition(n)) dépend de ce que tu veux optimiser :

- le nombre de fichiers en sortie,
- les performances d'écriture/lecture,
- ou la taille cible des fichiers Parquet (recommandée : 100–300 Mo par fichier).

Le bon nombre de coalesce(n)

- Objectif courant : des fichiers ~200 Mo
- Étapes concrètes pour choisir le bon coalesce(n) :

1. Exécute sans coalesce pour mesurer la taille totale

```
val df = spark.read.parquet("ton_path")  
df.write.mode("overwrite").parquet("output_path") // sans coalesce ni partitionBy
```

Puis, utilise %fs ou %sh dans un notebook pour voir la taille :

```
%fs ls /FileStore/output_path
```

Note la **taille totale** (T) et **nombre de fichiers** (F) => moyenne = T / F .

Le bon nombre de coalesce(n)

- Décide de la taille cible des fichiers
 - Choisissons 200 Mo comme valeur cible.

Calcule le nombre optimal de fichiers :

$$\text{Nombre_optimal_fichiers} = \frac{\text{Taille_totale_en_Mo}}{200}$$

$$\text{coalesce}(n) = \left\lceil \frac{T}{200} \right\rceil$$

Use case «spark it»

start - w002 - Détection de comportements suspects chez les clients bancaires

Règles Métier :

1. Le montant cumulé des retraits sur 7 jours glissants est **supérieur à 10 000 €**.
2. Le nombre de transactions dans un pays étranger est **supérieur à 5** en 3 jours.
3. Un écart de **plus de 1000 km** entre deux transactions à moins de 2h d'intervalle.

```
import org.apache.spark.sql.functions._
import org.apache.spark.sql.Column

// Lecture du fichier CSV
val df = spark.read
  .option("header", "true")
  .option("inferSchema", "true")
  .csv("/dbfs/FileStore/transactions.csv")
  .withColumn("timestamp", col("timestamp").cast("timestamp"))

// Règle 1 : cumul des retraits sur 7 jours > 10 000 €
val retraitDf = df.filter(col("transaction_type") === "retrait")
val retraitWithWindow = retraitDf.withColumn("window_7d", window(col("timestamp"), "7 days"))
val rule1 = retraitWithWindow.groupBy("client_id", "window_7d")
  .agg(sum("amount").alias("total_retrait"))
  .filter(col("total_retrait") > 10000)

// Règle 2 : plus de 5 transactions à l'étranger en 3 jours
val foreignDf = df.filter(col("country") !== "FR")
val foreignWithWindow = foreignDf.withColumn("window_3d", window(col("timestamp"), "3 days"))
val rule2 = foreignWithWindow.groupBy("client_id", "window_3d")
  .agg(count("*").alias("nb_tx"))
  .filter(col("nb_tx") > 5)

// Règle 3 : déplacement > 1000 km en moins de 2h
val dfWithAlias = df
  .select("client_id", "timestamp", "latitude", "longitude")
  .withColumnRenamed("timestamp", "ts")
  .withColumnRenamed("latitude", "lat")
  .withColumnRenamed("longitude", "lon")

val a = dfWithAlias.alias("a")
val b = dfWithAlias.alias("b")

val joined = a.join(b, $"a.client_id" === $"b.client_id" && $"a.ts" < $"b.ts")
  .filter(unix_timestamp($"b.ts") - unix_timestamp($"a.ts") < 7200) // < 2h
  .withColumn("distance", haversine($"a.lat", $"a.lon", $"b.lat", $"b.lon"))
  .filter($"distance" > 1000)

val rule3 = joined.select($"a.client_id".alias("client_id")).distinct()

// Fusion des clients suspects
val suspects = rule1.select("client_id")
  .union(rule2.select("client_id"))
  .union(rule3.select("client_id"))
  .distinct()

// Sauvegarde des résultats
suspects.write.mode("overwrite").csv("/dbfs/FileStore/output/suspects")

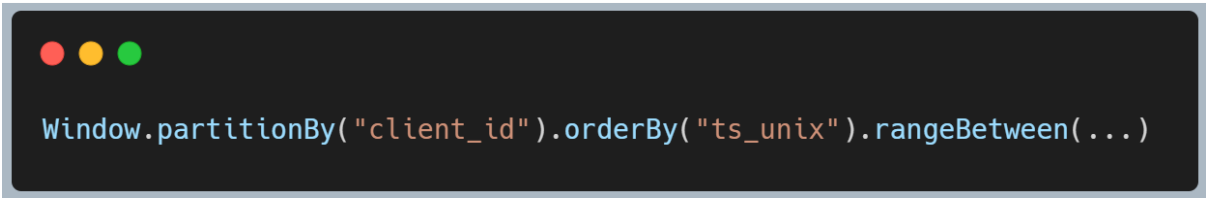
// Fonction Haversine
def haversine(lat1: Column, lon1: Column, lat2: Column, lon2: Column): Column = {
  val R = 6371.0 // rayon moyen de la Terre en km
  val dLat = radians(lat2 - lat1)
  val dLon = radians(lon2 - lon1)
  val a = pow(sin(dLat / 2), 2) +
    cos(radians(lat1)) * cos(radians(lat2)) * pow(sin(dLon / 2), 2)
  val c = lit(2) * atan2(sqrt(a), sqrt(lit(1) - a))
  lit(R) * c
}
```

Utilisation des window

Avant :

- Utilisation de window(...) basé sur groupBy — nécessite un shuffle massif.

Après :



```
Window.partitionBy("client_id").orderBy("ts_unix").rangeBetween(...)
```

Optimisation : rangeBetween évite les jointures implicites et réduit fortement les shuffles, ce qui améliore les performances en streaming ou batch.

Jointure intelligente sur timestamps

- Avant :



```
a.join(b, a.ts < b.ts)
```

Moins explicite, plus coûteux.

- Après :



```
.filter($"ts_unix2" > $"ts_unix1" && $"ts_unix2" - $"ts_unix1" <= 7200)
```

- **Optimisation** : filtrage précis dans la jointure, évite des paires inutiles, réduit le nombre de lignes post-jointure.

Use case « spark it »

start - w003 - ventes par
produit prioritaire

Règles Métier :

1. Filtrer les ventes sur une liste prioritaire de produits (par exemple : produits à forte marge).
2. Calculer pour ces produits le total vendu par catégorie.
3. Réutiliser les données pour faire un top 5 des clients ayant acheté ces produits.

Alors comment choisir !?

- contains / isin avec Broadcast Set
- join avec un DataFrame

contains / isin avec Broadcast Set



```
val prioritySet = spark.sparkContext.broadcast(Set("P001", ..., "P010"))  
df.filter(row => prioritySet.value.contains(row.getAs[String]("product_id")))
```

- **Avantages :**
 - Très rapide si la liste est très petite (quelques centaines de valeurs max).
 - Évite une jointure et peut être plus performant pour des petits ensembles en mémoire
 - Pas de shuffle (opération locale à chaque partition).
- **Inconvénients :**
 - Pas expressif (pas facile à maintenir).
 - Ne fonctionne pas bien avec des DataFrames distribués (pas de filter(\$"product_id".isin(...)) si dynamique ou énorme).
 - Le broadcast manuel peut causer des problèmes de gestion mémoire si mal utilisé.

join avec un DataFrame



```
val priorityDF = Seq("P001", ..., "P010").toDF("product_id")
df.join(broadcast(priorityDF), Seq("product_id"))
```

- **Avantages :**
 - Plus lisible et SQL-compatible.
 - Meilleur pour les cas dynamiques (ex : si la liste vient d'un fichier ou d'une table).
 - Spark peut automatiquement optimiser avec un broadcast join si la table de droite est petite.
 - Plus flexible et maintenable dans des pipelines complexes.
- **Inconvénients :**
 - Moins performant que Set.
 - contains si la liste est vraiment minuscule et statique.
 - Nécessite un join, donc potentiellement une étape de shuffle si non-broadcastée.

02

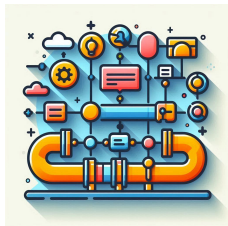
Spark Dev + SQL



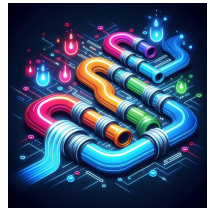
Spark Dev + SQL



1. Développement Spark en production



2. Introduction à Spark SQL



3. Exploitation de Spark SQL

Spark Dev + SQL

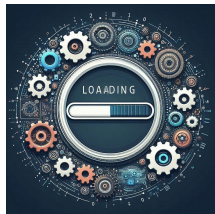
Développement Spark en production



Développement **Spark** en production



1. Spark Scala avec Maven



3. Scala Overview

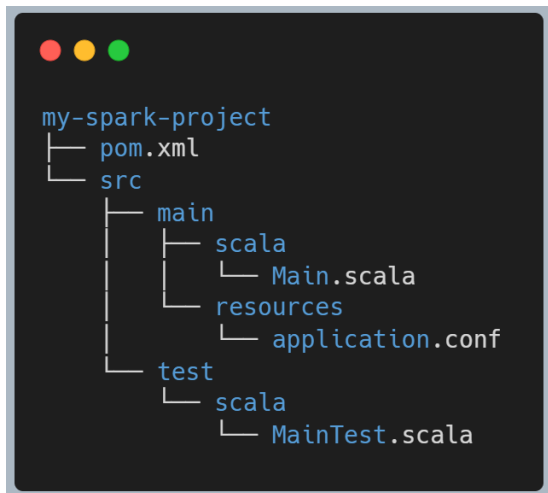


2. Poetry & PySpark

Initialisation d'un Projet Spark Scala avec Maven

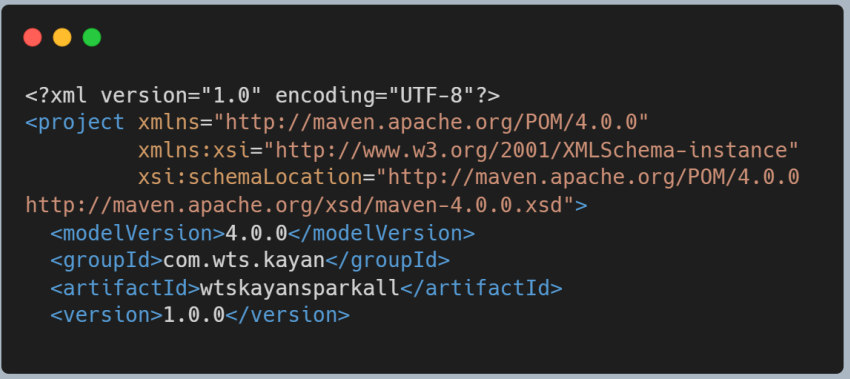
- Objectif : Configurer un projet Spark utilisant Scala et Maven.
- Création d'une structure de projet Maven pour organiser le code et gérer les dépendances.
- Pourquoi Maven : gestion de dépendances, modularité, et facilité de build pour les projets Scala.

Création d'une Structure de Projet Maven



- **Objectif :** Organiser le code de manière claire et efficace pour les applications Spark en Scala.
- **Structure Maven typique :**
 - **src/main/scala :** Contient le code source principal en Scala.
 - **src/test/scala :** Contient les tests unitaires.
 - **src/main/resources :** Emplacement pour les fichiers de configuration ou de données statiques, comme les fichiers CSV ou application.conf.
- **Fichier pom.xml :** Le fichier central de configuration Maven pour gérer les dépendances, les plugins, et les configurations de build.
- **Avantages d'une structure Maven :** Standardisation, gestion simplifiée des dépendances, compatibilité avec divers outils de CI/CD.

Présentation du fichier `pom.xml` pour un projet Spark Scala



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.wts.kayan</groupId>
  <artifactId>wtstkayansparkall</artifactId>
  <version>1.0.0</version>
```

- Le fichier `pom.xml` est le fichier de configuration principal de Maven pour la gestion des dépendances, des plugins, et des configurations de build.
- Il définit des informations globales sur le projet, comme le `groupId`, `artifactId`, et `version`.
- Les propriétés facilitent les modifications, comme la version de Scala, Spark, Java, et les noms des archives.

Les dépendances

```
<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-hdfs</artifactId>
  <version>${hadoop}</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-common</artifactId>
  <version>${hadoop}</version>
  <scope>provided</scope>
</dependency>
```

- **Dépendances principales** pour le projet Spark : Scala (scala-library) : Version du langage Scala utilisée.
- **Spark Core** : Base de Spark, incluant les fonctionnalités de traitement des données.
- **Spark SQL** : Module SQL pour le traitement des données structurées.
- **Hadoop** : Nécessaire pour les opérations de lecture/écriture avec HDFS.
- **Config** : Gestion des fichiers de configuration.
- **Dépendances de test** : JUnit, ScalaTest, Specs2.

Propriétés du Projet

```
<properties>
  <java.version>1.8</java.version>
  <scala.version>2.11.12</scala.version>
  <maven-jar-plugin>2.6</maven-jar-plugin>
  <maven-compiler-plugin>3.1</maven-compiler-plugin>
  <maven-release-plugin>2.5.3</maven-release-plugin>
  <scala-maven-plugin>3.2.2</scala-maven-plugin>
  <spark-version>2.4.0</spark-version>
  <scala.dep.version>2.11</scala.dep.version>
  <hadoop>2.7.3</hadoop>
  <pom.scala.mainclass>com.wts.kayan.app.job.MainDriver</pom.scala.mainclass>
  <maven-assembly-plugin.version>3.0</maven-assembly-plugin.version>
  <jar-name>wts-training-spark</jar-name>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

- **Définition des versions clés** : java.version, scala.version, spark-version, hadoop.
- **Propriété principale** pom.scala.mainclass : Spécifie la classe de démarrage pour Spark.
- **Importance des propriétés** pour la flexibilité du projet.

Plugins Maven

```
<plugin>
  <artifactId>maven-assembly-plugin</artifactId>
  <version>2.5.4</version>
  <configuration>
    <archive>
      <manifest>
        <mainClass>${pom.scala.mainclass}</mainClass>
      </manifest>
    </archive>
    <finalName>${jar-name}</finalName>
    <appendAssemblyId>>false</appendAssemblyId>
    <descriptors>
      <descriptor>assembly/assembly.xml</descriptor>
    </descriptors>
    <descriptorRefs>
      <descriptorRef>jar-with-dependencies</descriptorRef>
    </descriptorRefs>
  </configuration>
  <executions>
    <execution>
      <id>make-assembly</id>
      <phase>package</phase>
      <goals>
        <goal>single</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

- **maven-compiler-plugin** : Utilisé pour compiler le code Java dans la version spécifiée.
- **scala-maven-plugin** : Permet la compilation de code Scala.
- **maven-assembly-plugin** : Génère le JAR avec toutes les dépendances, ce qui facilite l'exécution sur un cluster Spark.

Exemple de Création et Exécution du JAR

- **Commandes Maven** pour compiler et packager le projet :
- **mvn clean package** : Compile et crée le JAR.
- **spark-submit** pour exécuter le JAR sur Spark.
- **Utilité de jar-with-dependencies** pour garantir que toutes les dépendances sont incluses dans le JAR.

Introduction à Poetry

PYTHON PACKAGING AND DEPENDENCY MANAGEMENT MADE EASY

Poetry

Ajouter des Dépendances



```
poetry add pyspark
```


Utilisation des Environnements Virtuels



```
poetry shell
```



```
poetry install
```

Déploiement et Production avec Poetry



```
poetry build
```



```
poetry publish
```

Scala & Spark

Scala est le langage natif pour Spark.

Offre une **syntaxe concise** et une intégration naturelle avec l'API Spark.

Fonctionnalités uniques :

- **Case Class** : Modélisation simple et immuable des données.
- **Fonctions anonymes** : Simplifie les transformations.

Case Class : Une Force Unique

Définition : Structure immuable pour modéliser les données.

- Génère automatiquement des méthodes comme toString, equals, et hashCode.



```
case class Transaction(id: Int, amount: Double, customer: String)
```



```
val transactionsRDD = sc.parallelize(Seq(Transaction(1, 100.0, "Alice")))
val transactionsDF = transactionsRDD.toDF()
```

Fonctions Anonymes

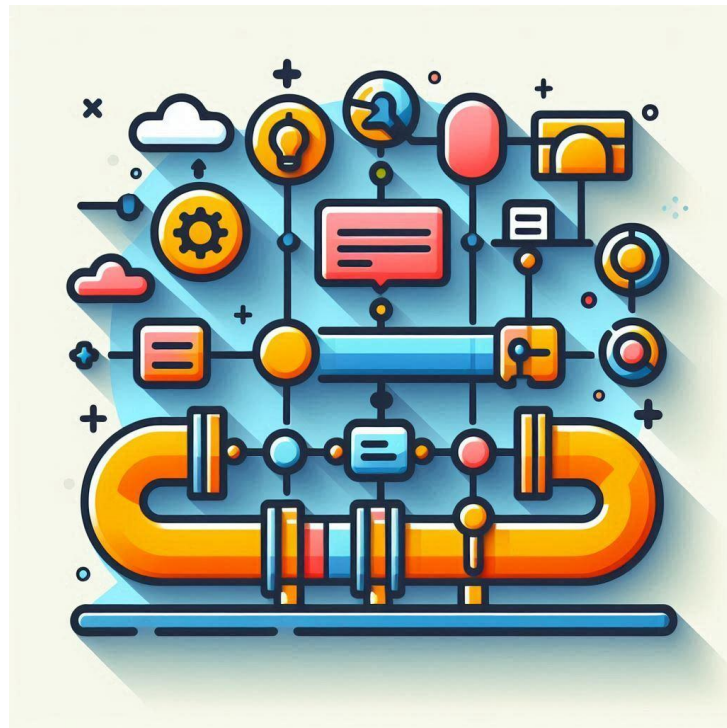
Scala simplifie les transformations avec des **fonctions anonymes**



```
val rdd = sc.parallelize(Seq(1, 2, 3, 4, 5))  
val result = rdd.map(_ * 2).filter(_ > 5).reduce(_ + _)
```

Spark Dev + SQL

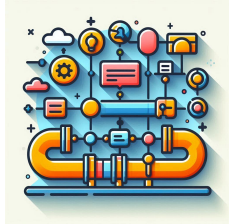
Introduction à Spark SQL



Introduction à Spark SQL



1. Batch Spark



2. Apache Spark – Scala Use Case

Définition du chargement de données

Qu'est-ce que le chargement de données ?

- Première étape du pipeline de traitement des données.
- Extraction des données depuis diverses sources pour les rendre disponibles dans Spark.

Sources de données supportées par Spark

- Types de sources de données supportées par Spark :
 - Fichiers plats pour CSV, TSV
 - Formats semi-structurés pour JSON, XML
 - Formats colonnes pour Parquet, ORC
 - Bases de données : JDBC (SQL, NoSQL).
 - Stockage en cloud : Amazon S3, Azure Blob, Google Cloud Storage.
 - Streaming : Kafka, Flume.
 - APIs externes.

Exemple de chargement de données avec Spark

Code exemple de chargement de données depuis un fichier Parquet :



Spark read

```
df = spark.read.format("parquet").load("/path/to/data.parquet")
```

Chargement depuis une base de données SQL



Spark read

```
jdbcDF = spark.read \  
    .format("jdbc") \  
    .option("url", "jdbc:mysql://localhost:3306/db") \  
    .option("dbtable", "tablename") \  
    .option("user", "root") \  
    .option("password", "password") \  
    .load()
```

Défis liés au chargement de données

Défis rencontrés lors du chargement de données :

- Formats de données hétérogènes.
- Volumes de données importants.
- Qualité des données (données manquantes, incohérentes).
- Connexion aux sources en temps réel.

Bonnes pratiques pour l'ingestion des données

- Validation de la qualité des données dès l'ingestion.
- Gestion des erreurs et des exceptions (données corrompues, formats inattendus).
- Scalabilité : utilisation des connecteurs natifs de Spark.
- Partitionnement pour des performances optimales.
- Optimisation du format de stockage (Parquet, ORC pour les grandes données).

Introduction aux Traitements Batch

- Définition du traitement batch.
- Avantages du traitement batch avec Spark (scalabilité, tolérance aux pannes, gestion de grands volumes de données).
- Cas d'utilisation typiques : traitement de journaux, calculs d'agrégation, intégration de données

Les étapes d'un traitement **Batch**

1. Ingestion de données :

- Chargement de données depuis plusieurs sources (HDFS, S3, bases de données, fichiers locaux).
- Utilisation de `spark.read`.

2. Transformation des données :

- Application de transformations sur les DataFrames (filtrage, jointures, agrégations, etc.).
- Utilisation des APIs de transformation de Spark (`map`, `filter`, `join`, `groupBy`)

3. Calculs distribués :

- Les calculs sont distribués sur plusieurs nœuds du cluster Spark.
- Optimisation des performances grâce aux partitions et aux mécanismes de "lazy evaluation" de Spark.

4. Écriture des résultats :

- Enregistrement des résultats dans différents formats (`parquet`, `JDBC`, etc.).
- Utilisation de `write`

Introduction à Spark SQL

Apache Spark – Scala Use Case



Optionnel : configuration du **workspace**

Étape 1 : Installer le JDK 8

Téléchargement : Allez sur le site officiel d'Oracle pour télécharger le JDK 8.

Étape 2 : Installer IntelliJ IDEA Community Edition

Téléchargement : Téléchargez IntelliJ IDEA Community Edition depuis le site officiel de JetBrains.

Étape 3 : Installer le Plugin Scala

Ouvrez IntelliJ IDEA.

Allez dans File > Settings (sur Windows) ou IntelliJ IDEA > Preferences (sur macOS).

Sélectionnez Plugins dans le menu de gauche.

Cliquez sur l'onglet Marketplace, recherchez Scala, puis cliquez sur Install.

Redémarrez IntelliJ IDEA pour activer le plugin.

Optionnel : configuration du **workspace**

Étape 4 : Installation de Git

Téléchargez Git depuis le site officiel de Git.

Étape 5 : Configuration de Git sur IntelliJ

Dans IntelliJ, allez dans File > Settings (Windows) ou IntelliJ IDEA > Preferences (macOS).

Sélectionnez Version Control > Git.

Cliquez sur le bouton Test pour s'assurer qu'IntelliJ reconnaît correctement Git. Si ce n'est pas le cas, spécifiez le chemin vers l'exécutable Git.

Optionnel : Installation Apache Spark sur Windows

Étape 1 : Télécharger Apache Spark version 2.4.5

- Le fichier à télécharger : [spark-2.4.5-bin-hadoop2.7.tgz](#)
- <https://archive.apache.org/dist/spark/spark-2.4.5/>

Étape 2 : Télécharger winutils

- Le fichier à télécharger : winutils.exe
- <https://github.com/cdarlint/winutils/blob/master/hadoop-2.7.7/bin/winutils.exe>

Étape 3 : Créer les répertoires suivants :

- C:\spark
- C:\hadoop\bin

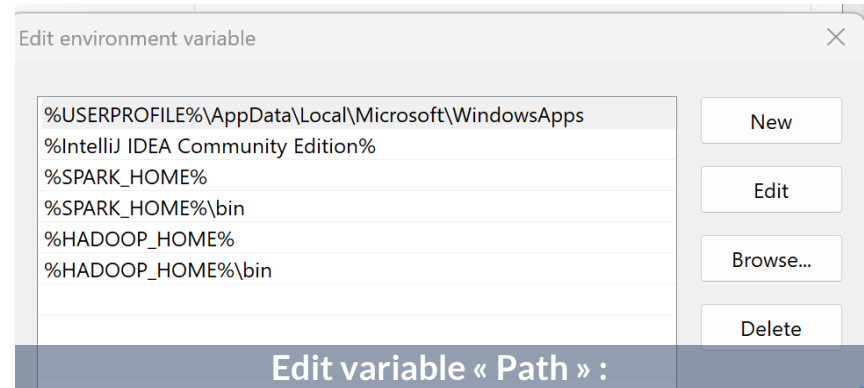
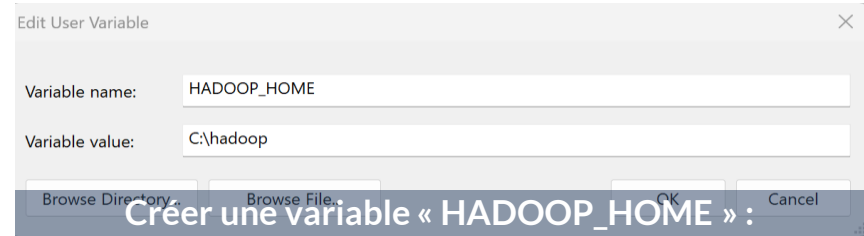
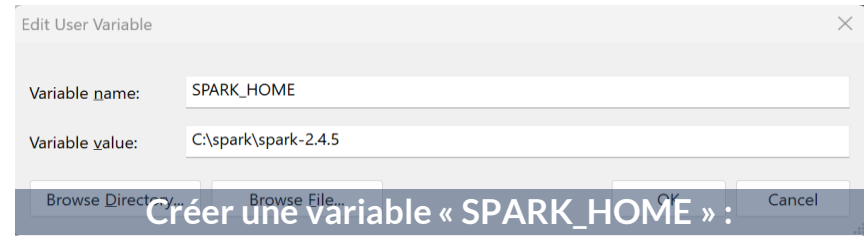
Étape 4 : Fichiers à copier

- A Dézipper et renommer spark-2.4.5-bin-hadoop2.7.tgz → spark-2.4.5
- spark-2.4.5 → C:\spark\
- winutils.exe → C:\hadoop\bin\

Étape 5 : Création des variables d'environnement windows

- Créer une variable « SPARK_HOME »
- Créer une variable « HADOOP_HOME » :
- Edit variable « Path » :

Optionnel : Installation Apache Spark sur Windows





```
C:\>C:\spark\spark-2.4.5\bin\spark-shell
```

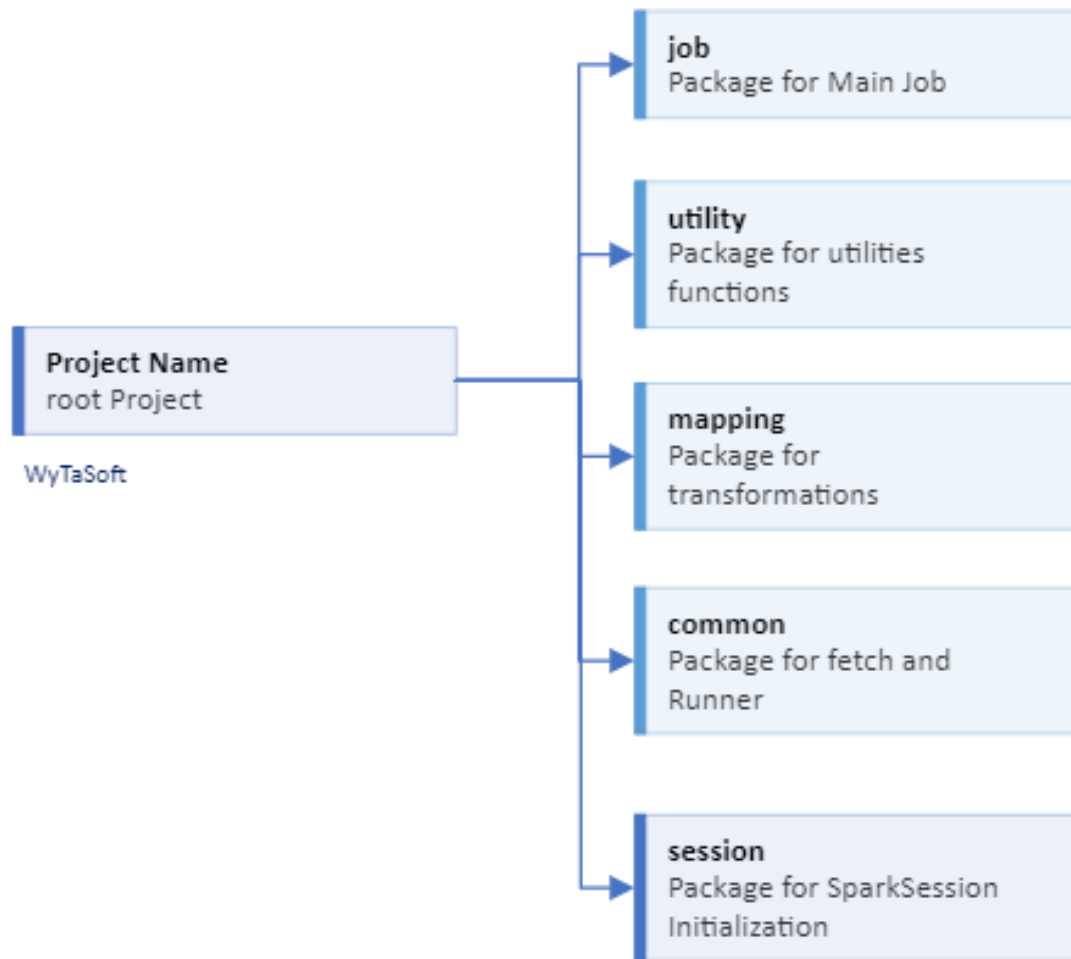
Optionnel : Installation Apache Spark sur Windows

- Pour exécuter Spark il faudrait lancer la commande suivante :
 - **C:\spark\spark-2.4.5\bin\spark-shell**

Use Case: Analyse des Commandes Clients

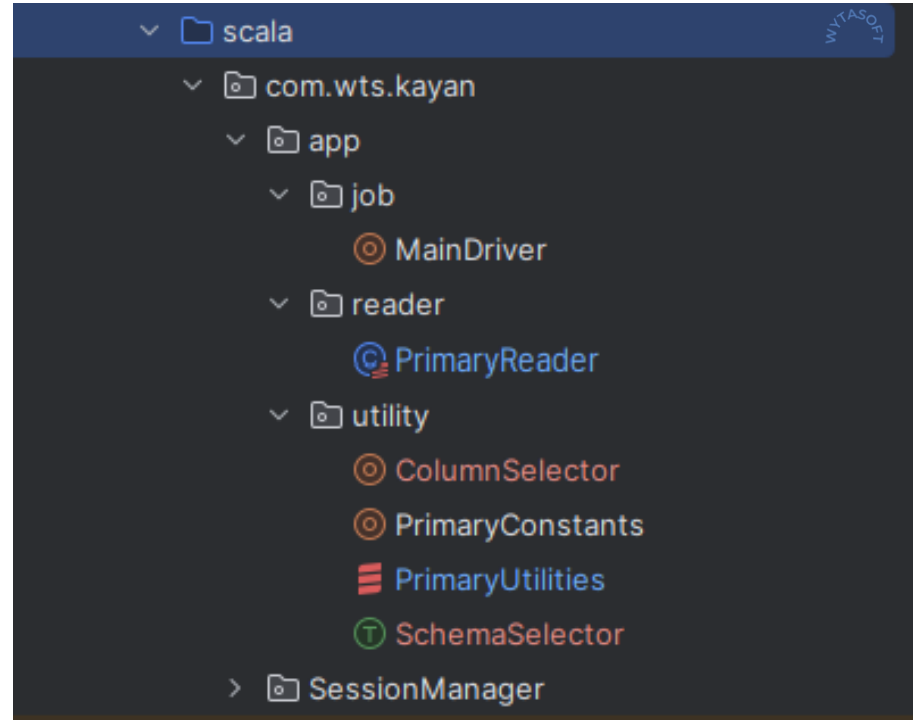
- **Objectif:** Combinez les informations sur les clients avec leurs commandes, filtrez les commandes de plus de 100€, calculez le total des commandes par client, et écrivez le résultat dans une nouvelle table.
- **Étapes :**
 1. **Lecture des Tables**
 - Table des clients (clients): contient clientId, name, et location.
 - Table des commandes (orders): contient orderId, clientId, amount, et date.
 2. **Jointure des Tables**
 - Jointure sur clientId pour associer chaque commande à son client.
 3. **Transformation et Filtres**
 - Filtrez pour ne garder que les commandes de plus de 100€.
 - Groupez par clientId et calculez le total des commandes par client.
 4. **Écriture du Résultat**
 - Écrivez le résultat dans une nouvelle table

Structure du Projet Scala/Spark



Use Case: Analyse des Commandes Clients

1. Read data
using Spark



PrimaryReader

- Initialisation des DataFrames avec Injection de Dépendances dans Spark
 - Notion « Extends » d'un « trait »
 - Lazy Initialization
 - Implicit Parameters

```
class PrimaryReader()(implicit sparkSession: SparkSession, env: String, config: Config ) extends SchemaSelector {  
    private lazy val clients =  
        readDataFrame(PrimaryConstants.CLIENTS, clientsSchema)  
  
    private lazy val orders =  
        readDataFrame(PrimaryConstants.ORDERS, ordersSchema)
```


Notion « **Extends** » d'un « **trait** »



Abstraction et Modularité



Réutilisabilité du Code



Extensibilité

Notion « Lazy Initialization »



Économie de Ressources



Amélioration des Performances au Démarrage

« Implicit » Parameters



Simplicité du code



Injection de Dépendances

PrimaryUtilities

- Calcul de la partition à lire
 - Réduction de la Charge I/O
 - Utilisation Optimisée des Ressources

```
18 def getMaxPartition(path: String, columnPartitioned: String = "date")(
19   spark: SparkSession): String = {
20   val fs = org.apache.hadoop.fs.FileSystem
21   .get(spark.sparkContext.hadoopConfiguration)
22
23   try {
24     val listOfInsertDates: Array[String] = fs
25       .listStatus(new Path(s"$path"))
26       .filter(_._isDirectory)
27       .map(_._getPath)
28       .map(_._toString)
29     val filterPartitionFolders =
30       listOfInsertDates.filter(_._contains(s"$columnPartitioned="))
31     val insertDateStr = filterPartitionFolders.map(_._split(s"$columnPartitioned=")(1))
32
33     if (insertDateStr.length > 1) {
34       val dateFormatted =
35         insertDateStr.map(d => convertStringToDate(d, "yyyy-MM-dd"))
36       val maxDate = dateToStrNdReformat(dateFormatted.max, "yyyy-MM-dd")
37       log.info(s"\n**** max $columnPartitioned $maxDate ****\n")
38       maxDate
39     } else if (insertDateStr.length == 0) {
40       log.info(s"\n**** there are no partition by $columnPartitioned in $path ****\n")
41       "2999-01-01"
42     } else {
43       log.info(s"\n**** max $columnPartitioned ${insertDateStr(0)} ****\n")
44       insertDateStr(0)
45     }
46   } catch {
47     case _: Throwable =>
48       println("Fatal Exception: Check Src-View Data")
49       throw new ArrayIndexOutOfBoundsException
50   }
51 }
52 }
```

Réduction de la Charge I/O



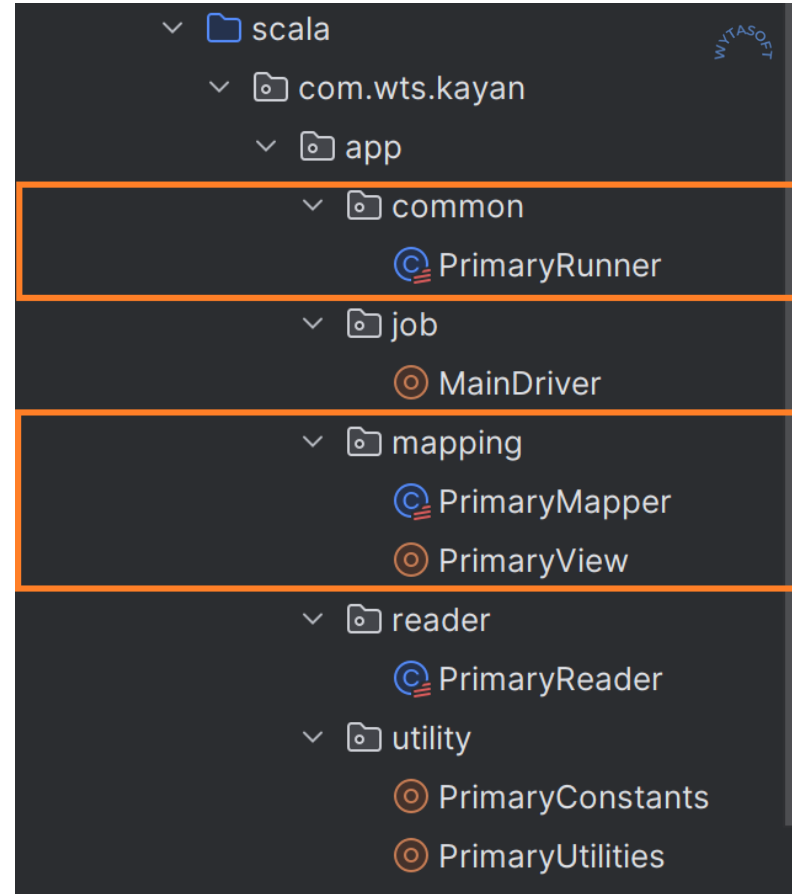
Réduction des Temps de Lecture



Allègement du Traitement

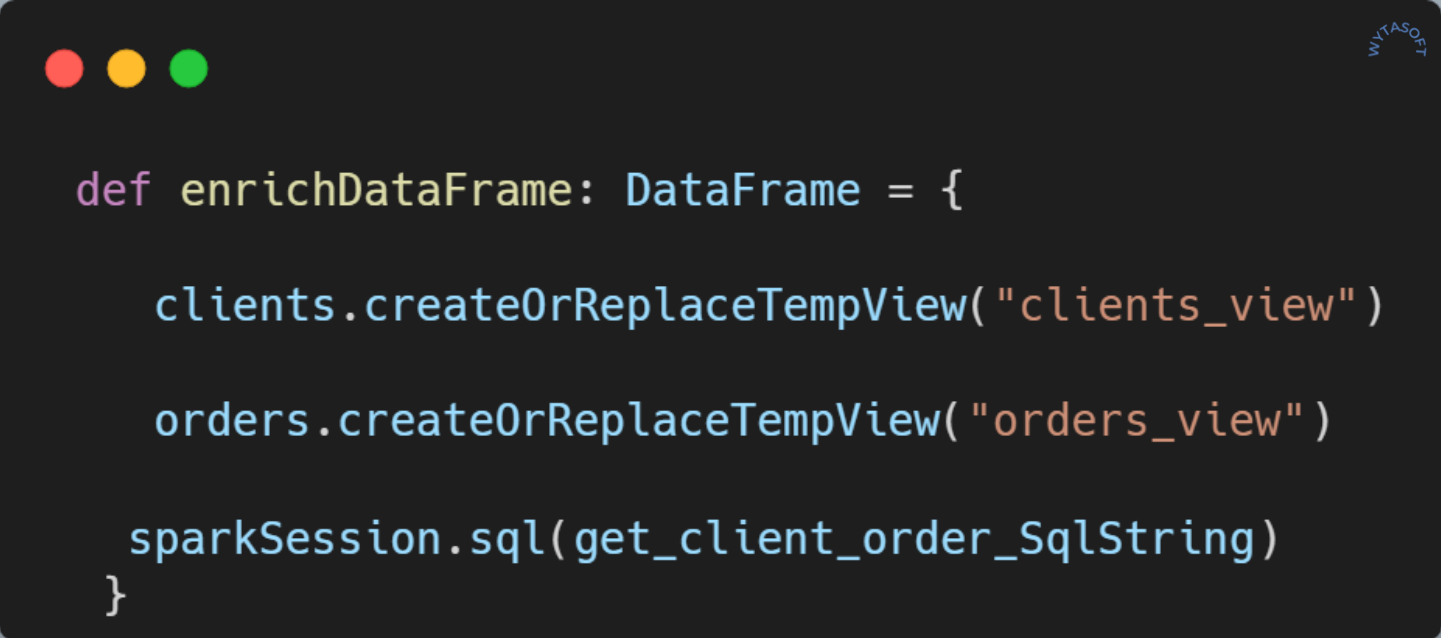
Use Case: Analyse des Commandes Clients

2.Join
dataframe
using
SparkSQL



Join dataframe using **SparkSQL**

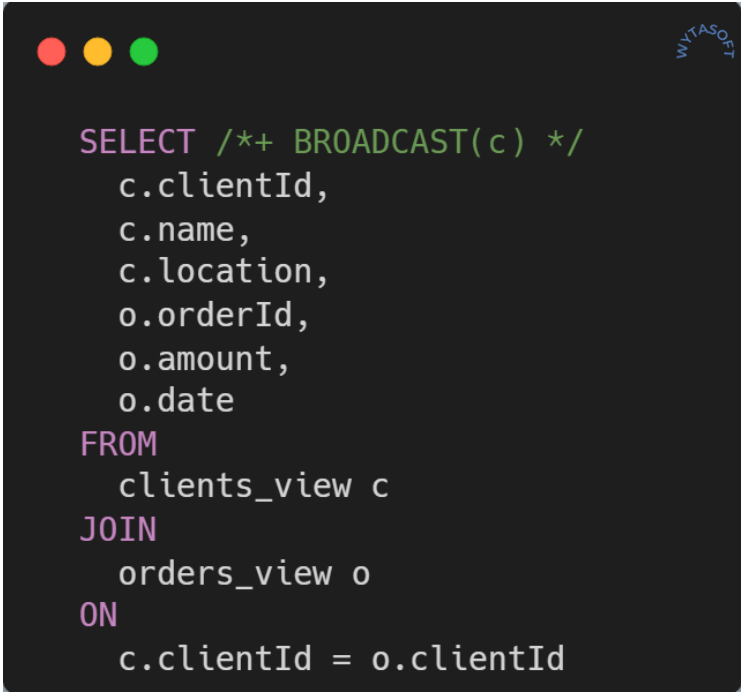
Enregistrer le DataFrame sous une vue temporaire



```
def enrichDataFrame: DataFrame = {  
    clients.createOrReplaceTempView("clients_view")  
    orders.createOrReplaceTempView("orders_view")  
    sparkSession.sql(get_client_order_SqlString)  
}
```

Join dataframe using SparkSQL

Jointure entre la table « client » et la table « order »



```
SELECT /*+ BROADCAST(c) */
  c.clientId,
  c.name,
  c.location,
  o.orderId,
  o.amount,
  o.date
FROM
  clients_view c
JOIN
  orders_view o
ON
  c.clientId = o.clientId
```


Quelques features de SparkSQL



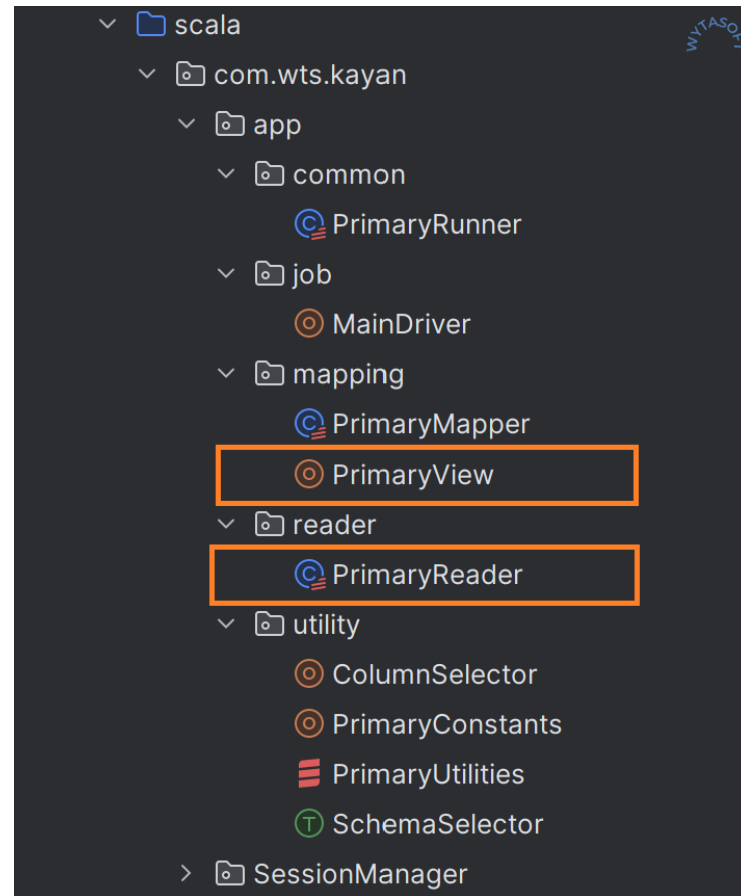
Les vues temporaires



Filtre « Where » avec « DataFrame »

Use Case: Analyse des Commandes Clients

3. Filter dataframe using SparkSQL



Filter dataframe using SparkSQL

Agrégation en utilisant « Windows function »

WYTASOFT

```
SELECT
  c.clientId,
  c.name,
  c.location,
  o.orderId,
  o.amount,
  o.date,
  sub.totalAmountByClient
FROM
  clients_view c
LEFT JOIN
  orders_view o
ON
  c.clientId = o.clientId
LEFT JOIN
  (SELECT
    clientId,
    SUM(amount) AS totalAmountByClient
  FROM
    orders_view
  GROUP BY clientId
  ) sub
ON
  c.clientId = sub.clientId
```



Moins de Shuffles

Simplicité et Lisibilité

Optimisation Intégrée

WYTASOFT

```
SELECT /*+ BROADCAST(c) */
  c.clientId,
  c.name,
  c.location,
  o.orderId,
  o.amount,
  o.date,
  SUM(o.amount) OVER(PARTITION BY
    c.clientId) AS totalAmountByClient
FROM
  clients_view c
LEFT JOIN
  orders_view o
ON
  c.clientId = o.clientId
```

Filter dataframe using SparkSQL

Filtre en utilisant « Where »



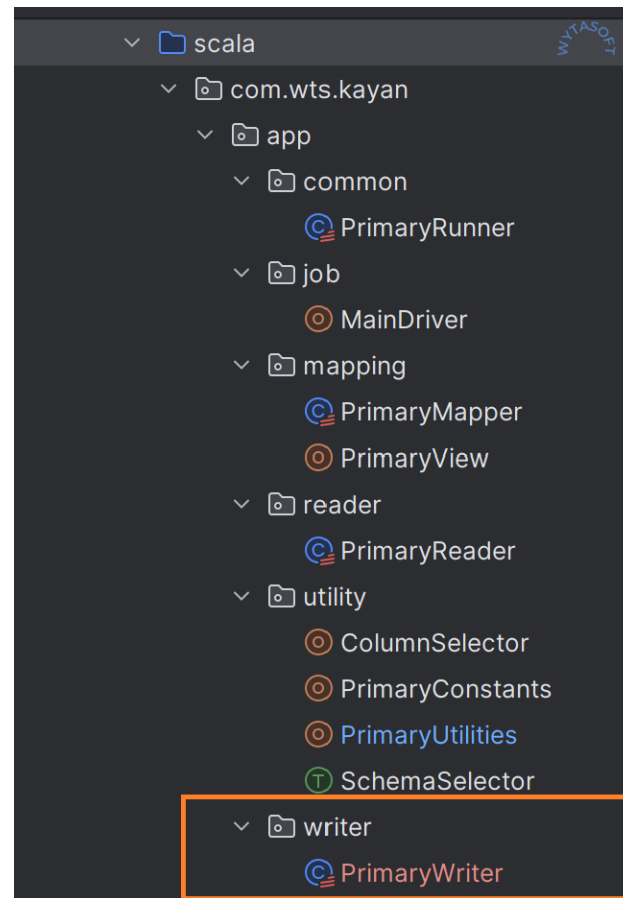
```
val dataframe: DataFrame = sparkSession.read
  .schema(schema)
  .parquet(s"$inputPath${tableName.toLowerCase}/")
  .selectExpr(ColumnSelector.getColumnSequence(sourceName):_*)
  .where(effectiveCondition)
```



Use Case: Analyse des Commandes Clients

4. Write dataframe result

1. Location & Storage
2. Partitionnement et Répartition
3. Gestion du Format
4. Configuration et Options



Write dataframe result

écrire un DataFrame Spark dans le datalake



```
dataFrame
  .coalesce(numPartition)
  .write
  .format("parquet")
  .partitionBy("location")
  .mode(s"$mode")
  .save(s"/$env/project/datalake/clients_orders")
```



Spark Dev + SQL

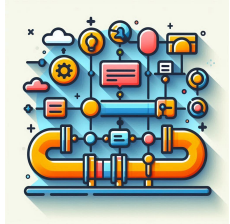
Exploitation de Spark SQL



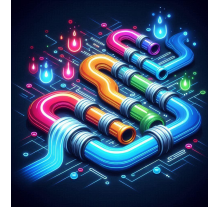
Exploitation de Spark SQL



1. Lecture de fichiers



2. inferSchema



3. Use Case Analyse des Données de Vols

Lecture de fichiers CSV depuis Amazon S3

- ☆ Configuration des accès S3 avec Spark.
- ☆ Lecture des fichiers CSV directement depuis S3.
- ☆ Application de l'option inferSchema pour déduire le schéma des données.

Read From S3

```
# Configurer l'accès à S3
spark._jsc.hadoopConfiguration().set("fs.s3a.access.key", "YOUR_ACCESS_KEY")
spark._jsc.hadoopConfiguration().set("fs.s3a.secret.key", "YOUR_SECRET_KEY")
spark._jsc.hadoopConfiguration().set("fs.s3a.endpoint", "s3.amazonaws.com")

# Lecture d'un fichier CSV depuis S3
df = spark.read.csv("s3a://bucket-name/project/file.csv", header=True, inferSchema=True)
```



Read From S3

```
# Lecture d'un fichier CSV depuis S3
df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load("s3a://your-bucket-name/project/transactions.csv")
```

Lecture de fichiers CSV depuis Amazon S3

Read en utilisant le load :

- Flexibilité
- Options Personnalisables
- Lecture de fichiers multiples

L'utilisation de l'option inferSchema

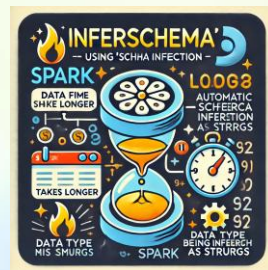


Avantages de inferSchema

Facilité d'Utilisation

Flexibilité

Simplicité pour des Jeux de Données
Simples



Inconvénients de inferSchema

Performance

Précision et Fiabilité

Contrôle Limité

Quand Utiliser inferSchema

Volume

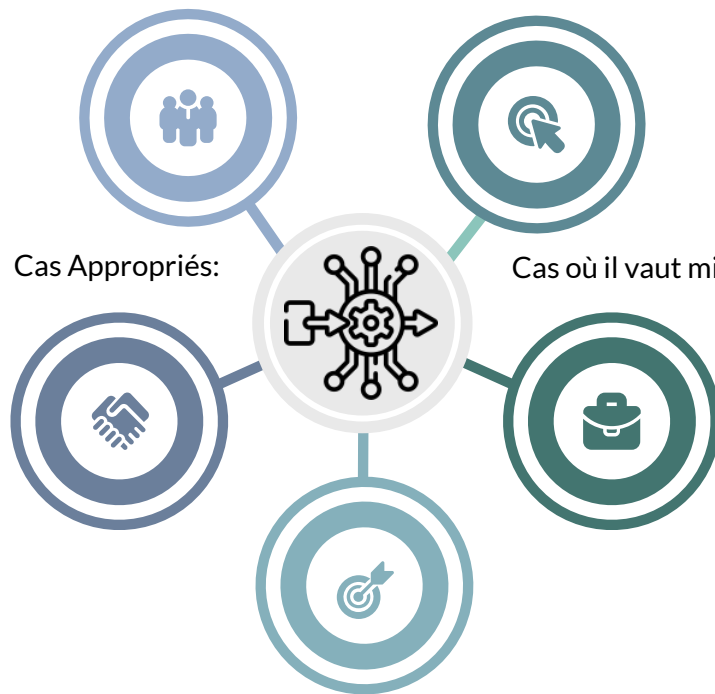
Jeux de données de petite à moyenne taille.

Analytique

Analyses exploratoires ou prototypes rapides où le schéma n'est pas critique.

simplicité

Scénarios où la simplicité et la rapidité d'exécution priment sur la performance et le contrôle.



Volume

Fichiers très volumineux, où la surcharge de performance peut être significative.

Data Quality

Données critiques où le schéma doit être précis et contrôlé.

Affichage des Colonnes, Schéma et Statistiques Usuelles

- Affichage des colonnes et des types de données.
- Affichage du schéma des données.
- Calcul et affichage des statistiques de base.



Affichage des Colonnes

```
# Afficher les colonnes et le schéma
df.printSchema()
df.columns
# Afficher les statistiques de base
df.describe().show()
```

Analyse des Données de Vols avec Apache Spark

Utilisation de Jointures pour Combiner les Données

- Combiner différentes tables de données liées aux vols (informations des vols, compagnies aériennes) à l'aide de jointures.
- Créer un ensemble de données enrichi pour une analyse approfondie.

Création et Utilisation d'une Vue Temporaire

- Créer une vue temporaire pour simplifier l'accès et permettre des requêtes SQL plus complexes.
- Faciliter la manipulation et l'analyse des données structurées.

Calcul du Taux d'Annulation des Vols

- Utiliser des fonctions SQL pour calculer le taux d'annulation des vols par compagnie aérienne.
- Identifier les tendances et les performances des compagnies en matière d'annulations.

Agrégation et Analyse des Données avec SQL

- Appliquer des fonctions SQL telles que count, distinct, et group by pour obtenir des statistiques détaillées.
- Analyser le nombre total de vols, les vols uniques, et d'autres mesures pertinentes.

Visualisation des Résultats

- Présenter les résultats sous forme de tableaux et de graphiques pour visualiser les tendances et les statistiques.
- Interpréter les données de manière claire et convaincante.

Analyse des Données de Vols avec Apache Spark

Quelles étaient les conditions météorologiques au moment du départ de chaque vol ?

Les conditions météorologiques (par exemple, vent fort) ont-elles un impact sur les retards des vols ?

Utilisation de Jointures pour Combiner les Données

Utilisation de Jointures pour Combiner les Données

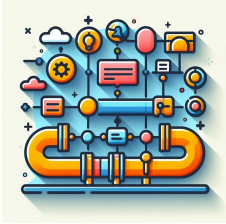
- Combiner différentes tables de données liées aux vols (informations des vols, compagnies aériennes) à l'aide de jointures.
- Créer un ensemble de données enrichi pour une analyse approfondie.
 - Architecture Data layer
 - Architecture Medaillon : modélisation one big table

03

Tuning Spark: From Internals to Deployment



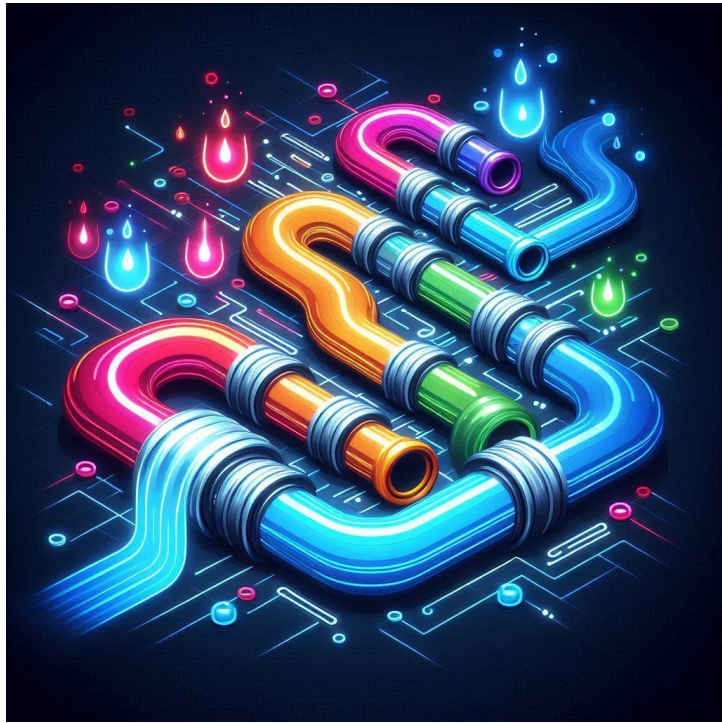
Tuning Spark: From **Internals** to **Deployment**



1. Monitoring : Spark UI Deep Dive

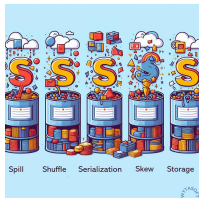
Tuning Spark: From **Internals** to **Deployment**

Monitoring : Spark UI Deep Dive



Monitoring : Spark UI Deep Dive

1. Les 5S Apache Spark



2. Optimisation Allocation mémoire

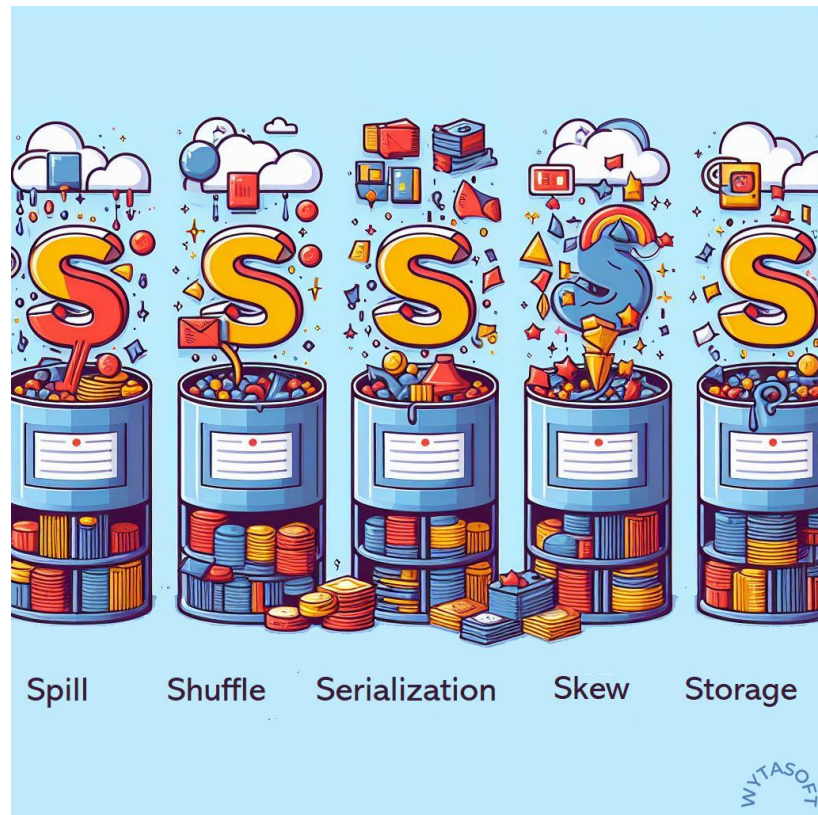


Monitoring : Spark UI

Deep Dive

Les 5S

Apache Spark

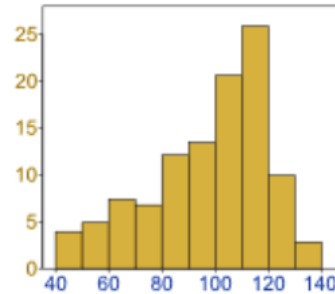


Les 5 S Spark

Spill

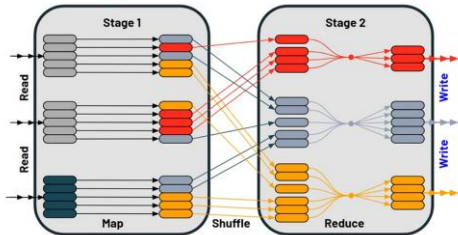


Skew

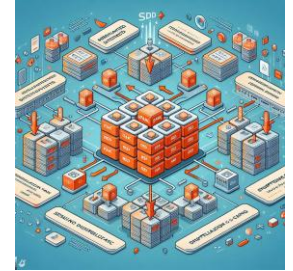


Negative Skew

Shuffle



Serialization

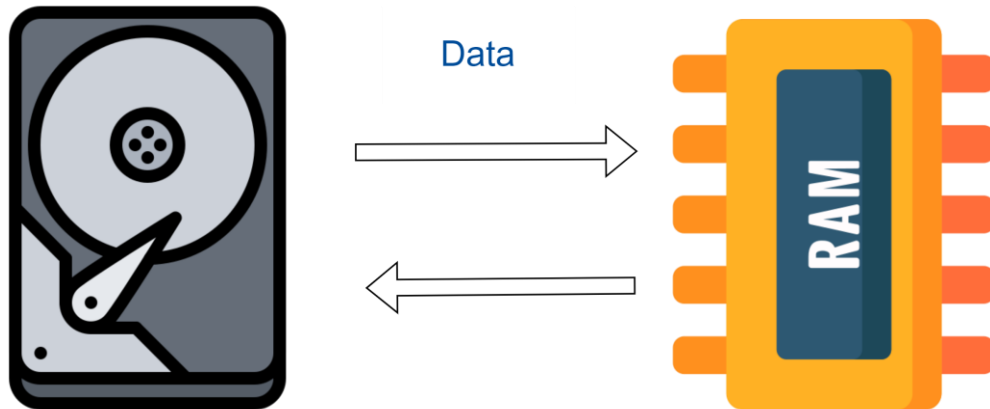


Storage



Spill

- C'est quoi le « Spill »
- Structure de Mémoire dans un Cluster Spark
- Actions Susceptibles de Provoquer un Spill
- Monitoring Spill



Spill

- Spill est causé par l'écriture de fichiers temporaires sur le disque lorsqu'il n'y a plus de mémoire (une partition est trop grande pour tenir dans la RAM).
- Dans ce cas, un RDD est d'abord déplacé de la RAM vers le disque, puis de nouveau vers la RAM.
- Afin d'éviter les erreurs de type "Out Of Memory" (OOM).

Structure de Mémoire dans un Cluster Spark

Worker Node

Executor

Mémoire de Travail
`spark.executor.memoryOverhead`

Mémoire vive
`spark.executor.memory`

Mémoire d'Exécution

Mémoire utilisable * `spark.executor.fraction`
* `spark.executor.storageFraction`

Mémoire de Stockage

Mémoire utilisable * `spark.executor.fraction` *
(1 - `spark.executor.storageFraction`)

Mémoire Utilisateur

Mémoire utilisable * (1 - `spark.executor.fraction`)

Mémoire réservée

300MB

Actions Susceptibles de Provoquer un **Spill**

- Toute « **action** » dans un Job Spark peut entraîner une augmentation de la taille des données au-delà de la capacité de la mémoire.

RDD

DataFrame

Lazy

***Narrow
Transformations***

***Wide
Transformations***

Use case Optimisation « Spill »



Use case « `shuffle.partitions` »



Use case « `only on the driver` »

Use case « shuffle.partitions »

- ☆ La réduction de « spark.sql.shuffle.partitions » entraîne une augmentation de la taille des fichiers par partition.

- ☆ Shuffle Partitions

- ☆ Taille des partitions

Shuffle Partitions

La configuration « `spark.sql.shuffle.partitions` » détermine le **nombre** de **partitions** à utiliser lors du **Shuffle** des données pour les jointures ou les agrégations. Par défaut, elle est fixée à 200.

Taille des Partitions

Lorsque vous réduisez le nombre de **partitions** de **Shuffle**, chaque partition finit par contenir plus de données. En effet, la même quantité de données est répartie entre moins de partitions.

data lake, data lakehouse, data mesh

data layer ou l'architecture médaillon

raw layer, silver layer

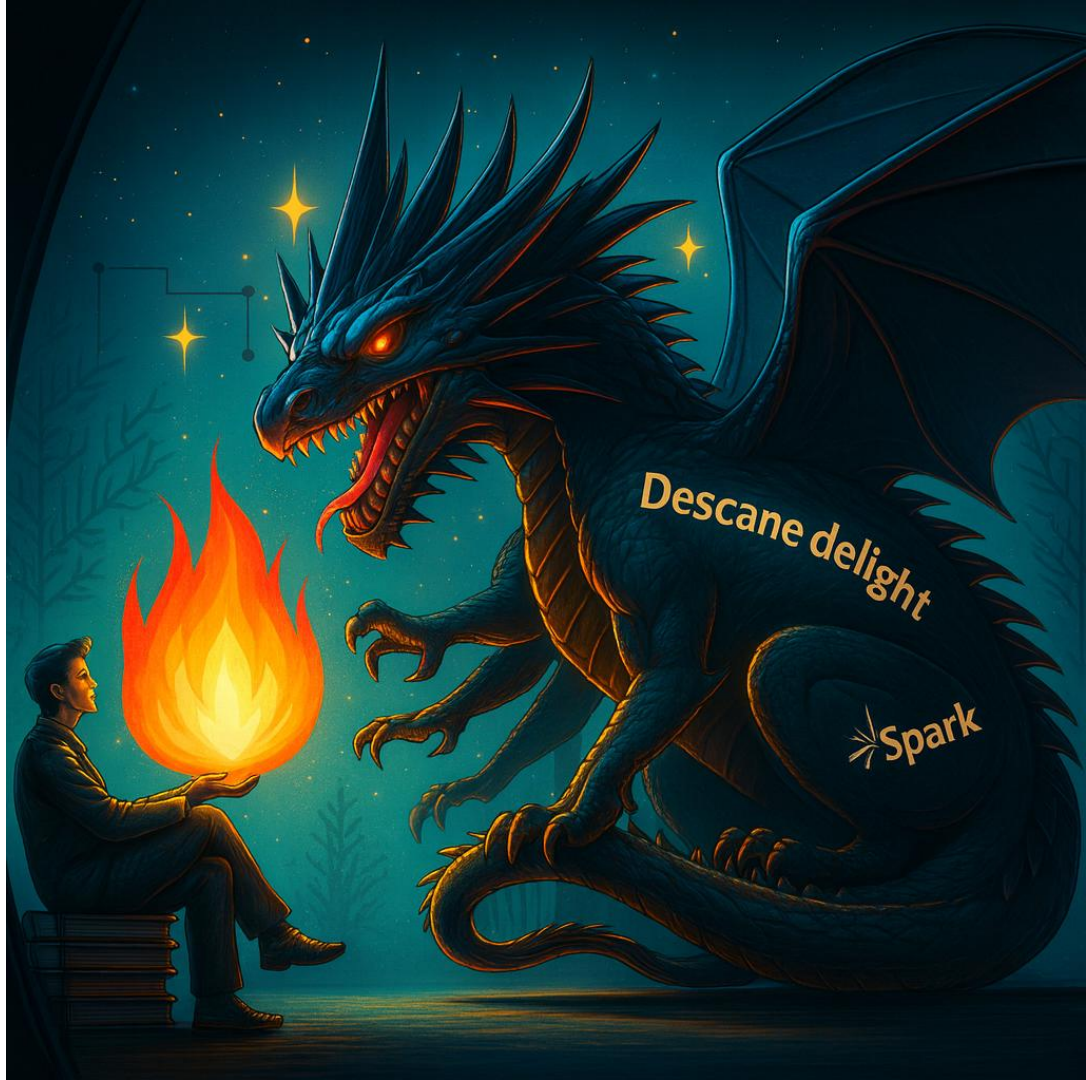
Use case « only on the driver »

- ☆ Certaines actions sont exécutées uniquement sur le driver node, ce qui signifie qu'elles collectent des données à partir (RDD, DataFrame ou Dataset) et les renvoient au driver. Ces actions sont souvent utilisées pour récupérer un petit sous-ensemble de données ou pour déclencher des calculs et des agrégations du dataset

☆ Les 10 actions à éviter

Les 10 actions à éviter

- `collect()`
- `take(n)`
- `first()` / `head()`
- `count()`
- `reduce(func)`
- `fold(zeroValue, func)`
- `aggregate(zeroValue, seqOp, combOp)`
- `sum()`, `mean()`, `max()`, `min()`
- `foreach(func)`
- `toLocalIterator()`



Practice : exemples et alternatives



Use case : sum() vs alternative

- **Use case :**
 - ☆ Calcul des données des employés pour trouver le département avec le salaire moyen le plus élevé.
- **Solution :**
 - ☆ La solution pourrait être d'utiliser les fonctions windows

sum() solution

```
val employees = Seq(
  (1, "Engineering", 60000),
  (2, "Engineering", 70000),
  (3, "HR", 50000),
  (4, "HR", 60000),
  (5, "Marketing", 40000),
  (6, "Marketing", 45000)
).toDF("employee_id", "department", "salary")

val avgSalariesByDept =
  employees.groupBy("department")
    .agg(avg("salary").alias("avg_salary"))

val employeesAboveAvg = employees
  .join(avgSalariesByDept, "department")
  .where($"salary" > $"avg_salary")
  .select("employee_id", "department",
    "salary")
display(employeesAboveAvg)
```

Window function solution



```
val windowSpec = Window.partitionBy("department")

// Add a new column with the average salary for each department
val dfWithAvgSalary = employees.withColumn("avg_salary", avg(col("salary")).over(windowSpec))

// Filter employees who earn more than the average in their department
val employeesAboveAvg = dfWithAvgSalary.filter(col("salary") > col("avg_salary"))
    .select("employee_id", "department", "salary", "avg_salary")
```

Use case : Calcul Max partition

- **Use case :**
 - ★ Lire les données d'une partition bien définie, par exemple la partition contenant la valeur maximum/minimum.
- **Solution :**
 - ★ La solution pourrait être de lire les partitions via hdfs et de calculer la valeur max de celles-ci.

Calcul Max partition

```
def getMaxPartition(path: String, columnPartitioned: String = "partitioned_date", formatDate: String = "yyyy-MM-dd") =
{
  val log = LoggerFactory.getLogger(getClass.getName)
  val fs = FileSystem.get(spark.sparkContext.hadoopConfiguration)

  Try {
    val status = fs.listStatus(new Path(path))
    val partitionPaths = status.filter(_.isDirectory)
                                .map(_.getPath.getName)
                                .filter(_.startsWith(s"$columnPartitioned="))
                                .map(_.split("=")(1))

    val dateFormatted: Array[Date] = partitionPaths
      .map(path => Try(convertStringToDate(path, formatDate)))
      .collect { case Success(date) => date } // Ensure we only work with successful conversions

    if (dateFormatted.nonEmpty) {
      val maxDate: Date = dateFormatted.maxBy(_.getTime)
      dateToStrNdReformat(maxDate, formatDate)
    } else {
      log.info(s"**** There are no partitions by $columnPartitioned in $path ****")
      "2999-01-01"
    }
  } match {
    case Success(maxDate) => maxDate
    case Failure(e) =>
      log.error("Exception occurred while fetching the max partition: ", e)
      "2999-01-01"
  }
}
```

Comment faire un call de la fonction

Calcul Max
partition

```
val maxPartitionDate =  
getMaxPartition("/FileStore/tables/employees/", "hiring_date")  
println(s"Max partition date: $maxPartitionDate")
```


Calcul Max partition

Comment utiliser le résultat de la fonction

```
val emp_by_date = spark.read
    .format("parquet")
    .load(s"/FileStore/tables/employees/hiring_date=$maxPartitionDate")
```

Identifier Spill via SparkUI

Summary Metrics for 24 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	3 s	5 s	5 s	5 s	6 s
GC Time	26.0 ms	44.0 ms	46.0 ms	47.0 ms	58.0 ms
Spill (memory)	320 MB	640 MB	640 MB	640 MB	640 MB
Spill (disk)	23.1 MB	41.3 MB	43.8 MB	45.1 MB	46.4 MB
Input Size / Records	78.1 MB / 571413	128.1 MB / 929564	128.1 MB / 951878	128.1 MB / 973369	128.1 MB / 988570
Shuffle Write Size / Records	52.1 MB / 1142826	79.3 MB / 1858328	82.3 MB / 1903756	84.8 MB / 1946778	87.1 MB / 1977140

Showing 1 to 6 of 6 entries

Aggregated Metrics by Executor

Show 20 entries

Executor ID	Logs	Address	Task Time	Total Tasks	Failed Tasks	Killed Tasks	Succeeded Tasks	Excluded	Input Size / Records	Shuffle Write Size / Records	Spill (Memory)	Spill (Disk)
driver		ubuntu.fritz.box:49569	2.1 min	24	0	0	24	false	3 GiB / 22507139	1.9 GiB / 45014278	14.7 GiB	1017 MB

Showing 1 to 1 of 1 entries

Tasks (24)

Show 20 entries

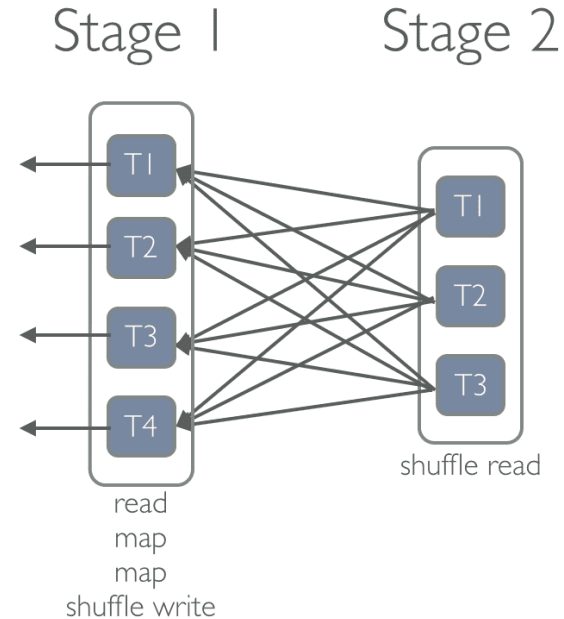
Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Input Size / Records	Shuffle Write Size / Records	Spill (Memory)	Spill (Disk)	Errors
0	29	0	SUCCESS	PROCESS_LOCAL	driver	ubuntu.fritz.box		2021-05-07 20:00:22	6 s	48.0 ms	128.1 MB / 988570	79.3 MB / 1977140	640 MB	40.7 MB	
1	30	0	SUCCESS	PROCESS_LOCAL	driver	ubuntu.fritz.box		2021-05-07 20:00:28	5 s	46.0 ms	128.1 MB / 929564	81.8 MB / 1858328	640 MB	45.9 MB	
2	31	0	SUCCESS	PROCESS_LOCAL	driver	ubuntu.fritz.box		2021-05-07 20:00:33	5 s	46.0 ms	128.1 MB / 965040	80 MB / 1930080	640 MB	41.3 MB	

Practice : Spill



Shuffle

- Fonctionnement du Shuffle
- Stratégies d'Optimisation
- Monitoring Shuffle

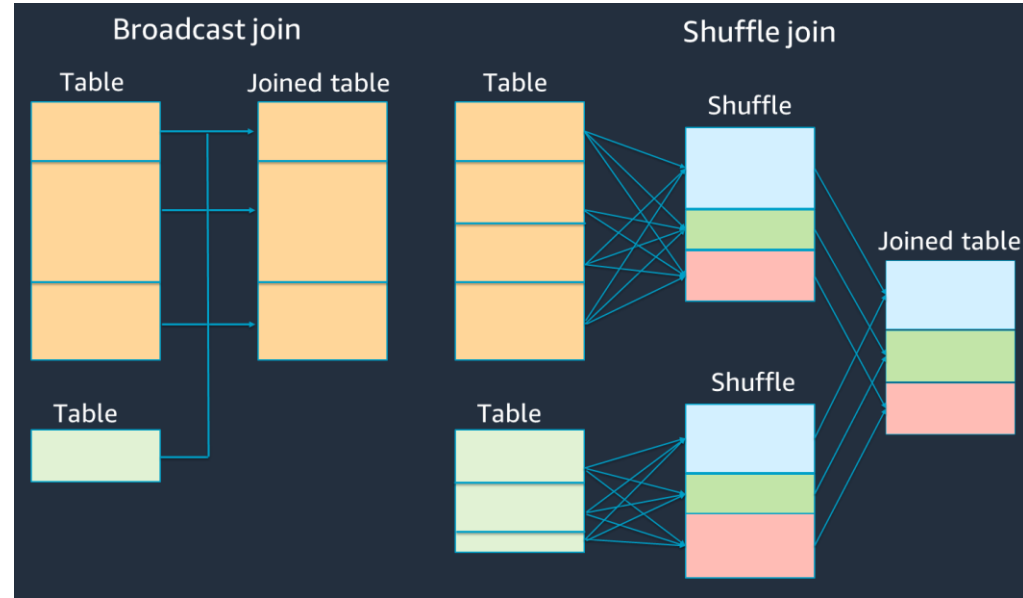


Shuffle

- Le shuffle dans Spark est un processus de réorganisation des données à travers les différents nœuds du cluster.
- Il se produit lors d'opérations qui nécessitent de regrouper ou de trier les données, comme les jointures (join), les agrégations (groupBy) ou le rééquilibrage des données (repartition).
- Cependant, le shuffle est gourmand en ressources car il implique des transferts de données sur le réseau et des écritures/lectures sur disque.

Broadcast join/Shuffle join

- Shuffle hash join combine deux tables sans tri et distribue la jointure entre les deux tables.
- Sort-merge join distribue les deux tables à joindre par clé et les trie avant de les joindre.
- Broadcast hash join envoie le plus petit RDD ou table sur chaque nœud de travail. Ensuite, une combinaison côté map avec chaque partition



Use case « join() or crossJoin() »

- ☆ Si l'une des tables dans la jointure est suffisamment petite, Spark peut choisir d'utiliser un **broadcast join**, où la petite table est envoyée à tous les nœuds du cluster.

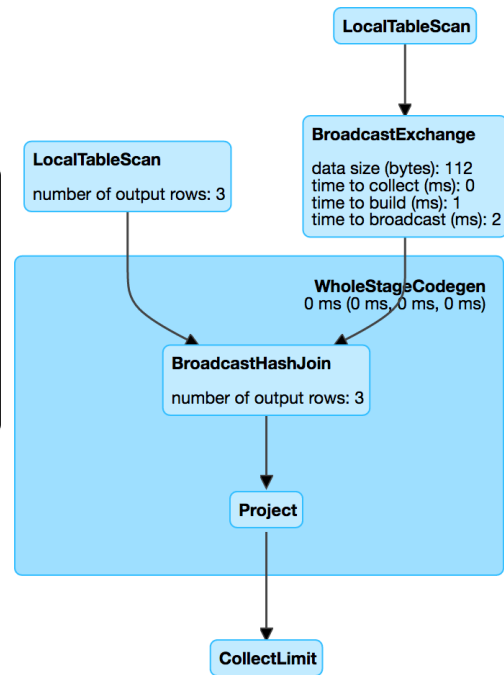
☆ Stratégies du **broadcast join**

Broadcast Hash Join

Broadcast Join: Explicitement indiquer à Spark de diffuser le DataFrame « salaries ».

```
import org.apache.spark.sql.functions.broadcast
```

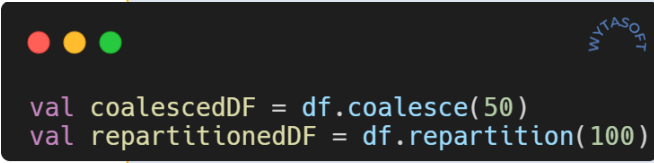
```
// Broadcast Join: Explicitement indiquer à Spark de diffuser le DataFrame `salaries`.  
val broadcastJoinDF = employees.join(broadcast(salaries), "id")  
broadcastJoinDF.show()
```



Repartition & coalesce

- **repartition** : Utilisez-le quand vous devez augmenter le nombre de partitions ou quand vous devez redistribuer les données de manière uniforme.

- **coalesce** : Utilisez-le pour diminuer le nombre de partitions, surtout quand vous savez que les données sont déjà distribuées de manière assez équilibrée.

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner and a 'WYTASOFT' logo in the top-right corner. It contains two lines of Scala code.

```
val coalescedDF = df.coalesce(50)  
val repartitionedDF = df.repartition(100)
```

Identifier Shuffle via SparkUI

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Blacklisted
Active(7)	194	282.9 MB / 9.6 GB	0.0 B	12	0	0	1497	1497	16 min (27 s)	487.3 MB	101.7 MB	101.7 MB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B	0
Total(7)	194	282.9 MB / 9.6 GB	0.0 B	12	0	0	1497	1497	16 min (27 s)	487.3 MB	101.7 MB	101.7 MB	0

Executors

Show 20 entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
0	10.53.20.24:42018	Active	38	36.5 MB / 1.5 GB	0.0 B	2	0	0	51	51	1.4 min (4 s)	66.1 MB	7.4 MB	13.9 MB	stdout stderr	Thread Dump
driver	in-ibmibm5429.persistent.co.in:42934	Active	0	30.4 MB / 384.1 MB	0.0 B	0	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B		Thread Dump
1	10.53.20.24:40185	Active	25	68.6 MB / 1.5 GB	0.0 B	2	0	0	43	43	1.6 min (3 s)	115.1 MB	53.8 MB	55.4 MB	stdout stderr	Thread Dump
2	10.53.20.0:45915	Active	52	53.6 MB / 1.5 GB	0.0 B	2	0	0	668	668	4.9 min (6 s)	119.6 MB	13.2 MB	8.7 MB	stdout stderr	Thread Dump
3	10.53.20.0:41765	Active	51	53.8 MB / 1.5 GB	0.0 B	2	0	0	684	684	4.9 min (8 s)	103.2 MB	11.6 MB	8.3 MB	stdout stderr	Thread Dump
4	10.53.16.14:45840	Active	14	19.9 MB / 1.5 GB	0.0 B	2	0	0	26	26	1.5 min (3 s)	41.2 MB	7.6 MB	7.7 MB	stdout stderr	Thread Dump
5	10.53.16.14:36592	Active	14	20.1 MB / 1.5 GB	0.0 B	2	0	0	25	25	1.5 min (3 s)	42 MB	8.1 MB	7.7 MB	stdout stderr	Thread Dump

Showing 1 to 7 of 7 entries

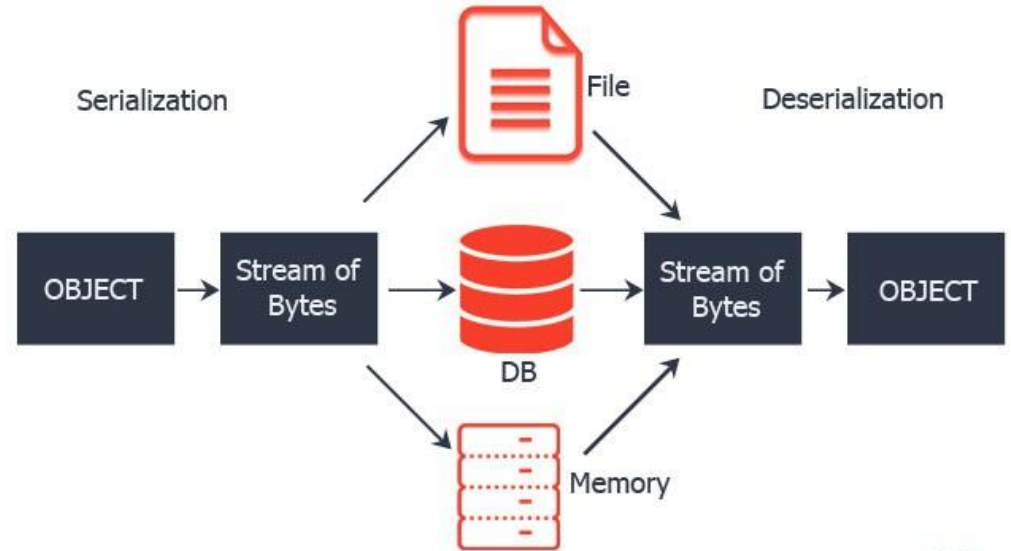
[Previous](#) 1 [Next](#)

Practice : Shuffle



Serialization

- Fonctionnement du Serialization
- Stratégies d'Optimisation
- Monitoring Serialization



Serialization

- La **sérialisation** dans Apache Spark est le processus de conversion des objets en un flux de bytes pour le stockage ou la transmission à travers le réseau :
- **Transfert de Données sur le Réseau** : Lors des shuffles ou lorsque les données sont envoyées à travers les nœuds du cluster, les objets doivent être sérialisés.
- **Stockage sur Disque** : Les données persistées ou les données qui débordent de la mémoire (spill) sont stockées sur disque et doivent donc être sérialisées.

Kryo serialization

Enregistrement des Classes : Pour obtenir les meilleures performances avec Kryo, il est recommandé d'enregistrer toutes les classes que vous prévoyez de sérialiser.



```
sparkConf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
sparkConf.set("spark.kryo.registrationRequired", "true") // Enforce registration

// Enregistrement explicite des classes utilisées
sparkConf.registerKryoClasses(Array(
  classOf[MyClass1],
  classOf[MyClass2],
  // ajoutez d'autres classes ici
))
```



Persist



```
// Création d'un DataFrame
val data = Seq((1, "Alice"), (2, "Bob"), (3, "Charlie"))
val df = spark.createDataFrame(data).toDF("id", "name")

// Utilisation de persist avec un niveau de stockage spécifique
df.persist(StorageLevel.MEMORY_AND_DISK)

// Actions qui bénéficieront de la mise en cache
df.show()
df.count()

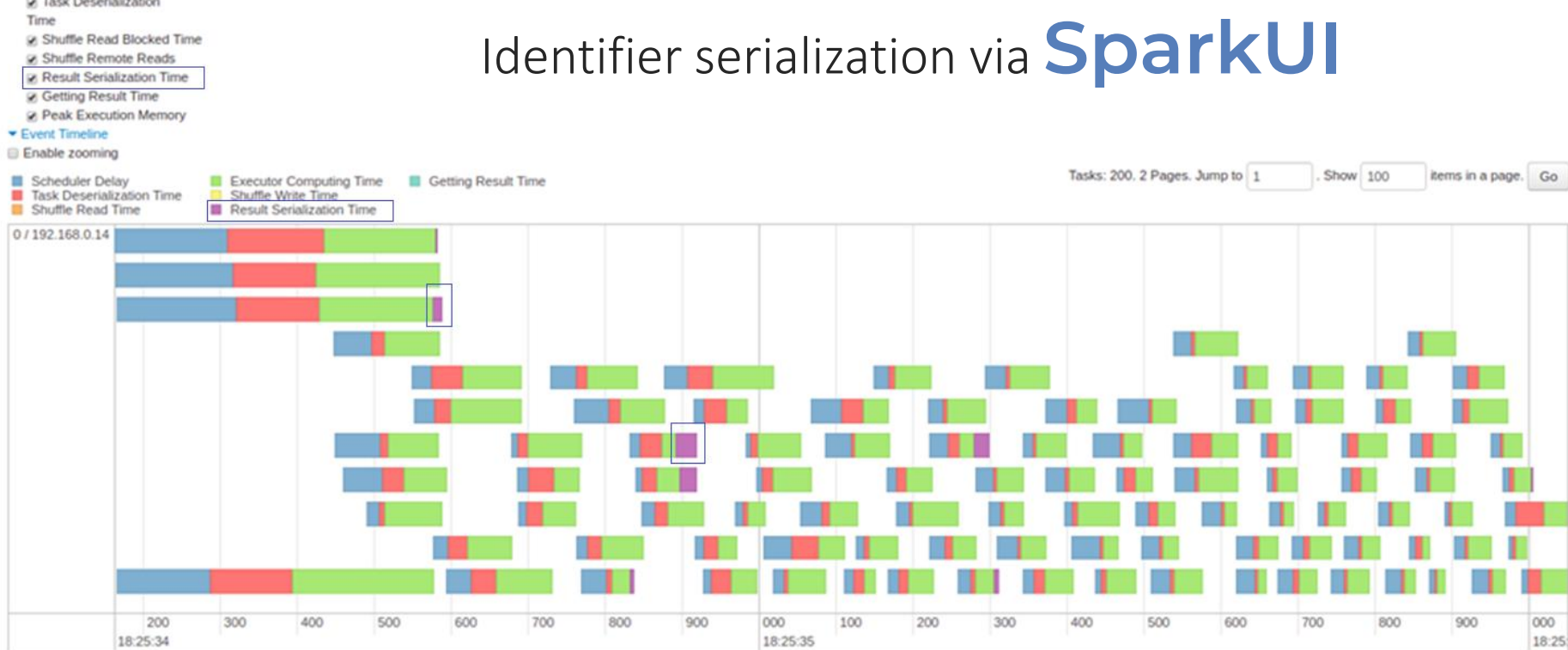
// Libération des ressources
df.unpersist()
```

- MEMORY_ONLY
- MEMORY_AND_DISK
- MEMORY_ONLY_SER
- MEMORY_AND_DISK_SER

Les options du « Persist »

Niveau de Stockage	Description	Avantages	Inconvénients
MEMORY_ONLY	Stocke l'objet en mémoire. Si l'espace est insuffisant, certaines partitions seront recalculées à la demande.	Accès rapide aux données.	Peut nécessiter un recalcul si l'espace en mémoire est insuffisant.
MEMORY_AND_DISK	Stocke l'objet en mémoire; stocke sur disque si l'espace en mémoire est insuffisant.	Garantit que l'objet est toujours disponible sans recalcul.	Accès plus lent aux données stockées sur disque.
MEMORY_ONLY_SER	Stocke les objets en mémoire sous forme sérialisée.	Utilise moins d'espace mémoire.	Nécessite une désérialisation avant l'accès.
MEMORY_AND_DISK_SER	Stocke les objets en mémoire de manière sérialisée, et sur disque si nécessaire.	Économise l'espace mémoire et garantit la disponibilité.	Accès plus lent et nécessite désérialisation.
DISK_ONLY	Stocke l'objet uniquement sur le disque.	Ne consomme pas de mémoire vive.	Accès aux données le plus lent.
MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc.	Stocke les objets avec réplication (e.g., deux copies en mémoire ou mémoire et disque).	Augmente la tolérance aux pannes.	Utilisation plus élevée des ressources (mémoire, disque).
OFF_HEAP	(Expérimental) Stocke l'objet dans la mémoire hors tas gérée par Tungsten.	Ne pas affecter le garbage collector de la JVM.	Nécessite une configuration appropriée et gère la sérialisation.

Identifier serialization via SparkUI



Summary Metrics for 200 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Task Deserialization Time	2.0 ms	4.0 ms	5.0 ms	13.0 ms	0.1 s
Duration	6.0 ms	16.0 ms	24.0 ms	36.0 ms	0.2 s
GC Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	11.0 ms
Result Serialization Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	27.0 ms
Getting Result Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Scheduler Delay	5.0 ms	11.0 ms	16.0 ms	23.0 ms	0.2 s
Peak Execution Memory	8.1 MiB	8.1 MiB	8.1 MiB	8.1 MiB	8.1 MiB

Practice : Serialization



Storage

- Fonctionnement du Storage
- Stratégies d'Optimisation
- Monitoring Storage

Different Data File Format in Big Data



JSON



CSV



Parquet



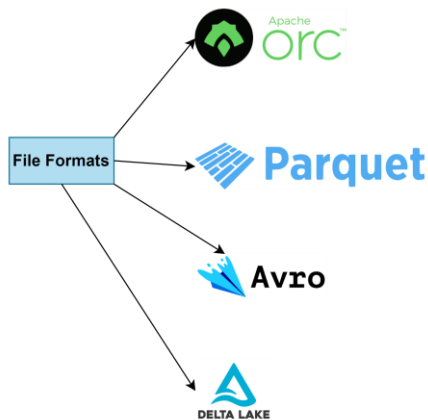
Avro



ORC

Storage

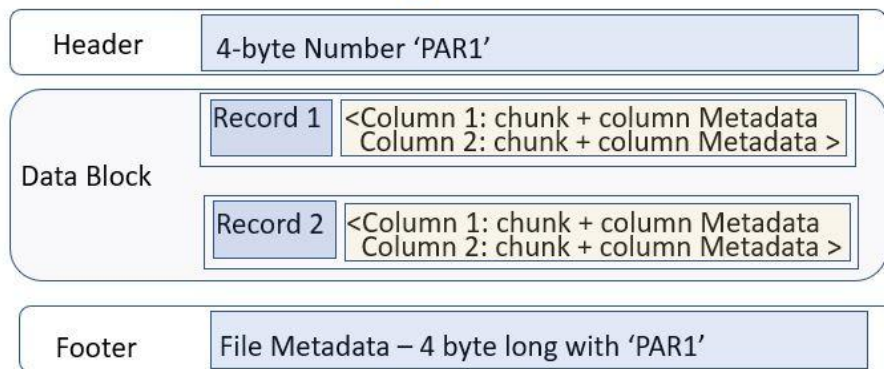
- Le principal avantage de Spark est sa capacité à stocker les données intermédiaires en mémoire vive (RAM) entre les étapes de calcul.
- Cela permet de réduire considérablement le temps d'accès aux données par rapport à la lecture depuis le disque, accélérant ainsi les calculs répétitifs ou les chaînes de transformations.



Caractéristique	Parquet	ORC	Avro	Delta Lake
Type de Stockage	Colonne	Colonne	Ligne	Colonne (basé sur Parquet)
Optimisation	Requêtes analytiques, compression	Requêtes Hive, compression	Sérialisation, intégration Kafka	Transactions, gestion des versions
Compression	Excellente	Excellente	Bonne	Excellente (héritée de Parquet)
Utilisation	Analyse de données à grande échelle	Analyse de données avec Hive	Échange de messages, évolution de schéma	Lacs de données transactionnels
Support des Schémas	Oui	Oui	Oui (avec évolution)	Oui (avec contraintes ACID)
Points Forts	Performance de lecture, faible consommation de stockage	Performance de requête, indexation	Flexibilité, évolution facile des schémas	Cohérence, récupération d'erreur
Limitations	Moins optimal pour les petits fichiers	Spécifique à l'écosystème Hadoop	Moins compressé que Parquet/ORC	Nécessite Spark pour l'exploitation

Formats de Fichiers

Décryptage de Apache Parquet



WyTaSoft

1. **En-tête (Header)** : est généralement minimal ou même absent dans les fichiers Parquet.

2. **Corps (Body)** :

Rôle : Le corps du fichier contient les données réelles, organisées en blocs ou en groupes de lignes (**row groups**).

Groupes de Lignes (Row Groups) : Chaque groupe de lignes contient des colonnes, qui sont elles-mêmes divisées en **pages**.

Pages : Les données au sein d'une colonne sont organisées en pages.

3. **Pied de Page (Footer)** :

Rôle : Le pied de page contient les métadonnées pour lire les données du body.

4. **Fin du Fichier (EOF)** :

Magic Number : À la toute fin du fichier, un "magic number" (PAR1)

Offset du Footer : Un offset indique la position du début du footer dans le fichier.

Compression Apache Parquet

- **Taille Storage** : réduit significativement la taille des données stockées.
- **Lecture Optimisée** : La compression au niveau des colonnes améliore le taux de compression.

Algorithme	Taux de Compression	Vitesse de Compression	Vitesse de Décompression	Utilisation CPU
Snappy	Moyen	Rapide	Très Rapide	Faible
GZIP	Élevé	Lent	Moyen	Élevée
LZ4	Moyen à Élevé	Très Rapide	Très Rapide	Faible à Moyenne

```
import org.apache.spark.sql.SparkSession

val spark = SparkSession.builder()
  .appName("Compression Parquet Example")
  .getOrCreate()

// Configuration pour utiliser GZIP comme algorithme de compression
spark.conf.set("spark.sql.parquet.compression.codec", "snappy")

val df = spark.read.csv("path/to/your/data.csv")

// Écrire le DataFrame en format Parquet avec compression snappy
df.write.parquet("path/to/output/myproject/")
```

Optimisation Storage

 `spark.sql.files.maxPartitionBytes` : 128MB

Le nombre maximal d'octets à regrouper dans une seule partition lors de la lecture de fichiers.

 `spark.sql.parquet.filterPushdown` : true

Active l'optimisation Predicate pushdown

 `spark.sql.parquet.enableVectorizedReader` : true

Active le décodage parquet vectorisé.

 `spark.sql.parquet.cacheMetadata` : true

Active la mise en cache des métadonnées de schéma Parquet. Peut accélérer l'interrogation des données statiques.

Quiz : Les 5S

Apache Spark

<https://www.wysoft.com/wysoft-group/index.php/wysoftacademy/>



Monitoring : Spark UI Deep Dive

Optimisation
Allocation
mémoire



Optimisation Allocation mémoire



1. L'allocation avec **logique** et **méthode**



2. L'allocation en **use case**

Use case 14 – Allocation de mémoire

Estimation du volume de données

Disons que ton job traite :

- **10 Go** de données non compressées
- En mémoire, Spark peut consommer **~2 à 3×** la taille brute (shuffle, cache, etc.)

➡ **Besoin mémoire total estimé : ~25 à 30 Go**

Use case 14 – Allocation de mémoire

- Supposons que le job Spark traite **10 Go de données non compressées**.
- En pratique, Spark ne travaille pas seulement avec la taille brute des données. Il génère :
 - des **fichiers temporaires** (shuffle),
 - du **cache** mémoire,
 - et d'autres **structures intermédiaires**.
- Il faut donc **multiplier la taille brute par un facteur de 2 à 3** pour estimer la mémoire réellement consommée.
- On arrive à une estimation mémoire totale de **25 à 30 Go** pour le job.

Use case 14 – Allocation de mémoire

- Supposons que le job Spark traite **10 Go de données non compressées**.
- En pratique, Spark ne travaille pas seulement avec la taille brute des données. Il génère :
 - des **fichiers temporaires** (shuffle),
 - du **cache** mémoire,
 - et d'autres **structures intermédiaires**.
- Il faut donc **multiplier la taille brute par un facteur de 2 à 3** pour estimer la mémoire réellement consommée.
- On arrive à une estimation mémoire totale de **25 à 30 Go** pour le job.

Use case 14 – Calcul du nombre de tâches

On part d'un volume total estimé de 30 Go

Chaque **task** est censée traiter entre 300 Mo et 500 Mo.

$$\text{Nombre de tâches} = \frac{\text{Taille totale}}{\text{Taille par tâche}}$$

Prenons une moyenne raisonnable : 400 Mo par task.

$$\frac{30 \text{ Go}}{0,4 \text{ Go}} = 75 \text{ tâches}$$

Use case 14 – Définir le nombre de cores

Recommandation :

- 1 task = 1 core
- Et 2 à 5 cores par executor

Nous pourrions viser, par exemple :

15 executors × **5 cores** = **75 cores**

→ qui traiteront les **75 tasks** en parallèle

→ donc traitement en **une seule vague de tasks**, ce qui est optimal.

Use case 14 – Déduire la mémoire totale

- Chaque core consomme 300 à 500 Mo de données, donc il faut dimensionner en conséquence.

Nous pourrions viser, par exemple :

Prenons **500 Mo de données par task** → donc **500 Mo par core** minimum.

Ajoutons une marge pour cache/shuffle → on prend **~1 Go par core**.

Donc chaque executor (5 cores) devrait avoir **~5 Go de mémoire**, voire 6 Go si on est prudent.

Use case 14 – Configuration finale suggérée

- Chaque core consomme 300 à 500 Mo de données, donc il faut dimensionner en conséquence.

Nous pourrions viser, par exemple :

Prenons **500 Mo de données par task** → donc **500 Mo par core** minimum.

Ajoutons une marge pour cache/shuffle → on prend **~1 Go par core**.

Donc chaque executor (5 cores) devrait avoir **~5 Go de mémoire**, voire 6 Go si on est prudent.

Use case 14 – Configuration finale suggérée

Paramètre	Valeur
Nombre de tasks	75
Executor cores	5
Nombre d'executors	15
Executor memory	5G
Total memory	$15 \text{ executors} \times 5 \text{ Go} = 75 \text{ Go}$ (à ajuster selon infra)

Use case 14 – Variante compacte

Paramètre	Valeur
Nombre de tasks	75
Executor cores	2
Nombre d'executors	5
Executor memory	5G
Total memory	15 executors × 5 Go = 75 Go (à ajuster selon infra)

- **5 executors × 2 cores = 10 cores**
- Donc Spark exécutera **75 tâches en 7 à 8 vagues**

Cela fonctionne aussi, mais **avec moins de parallélisme** → **plus de temps de traitement**.

Use case 15 – Allocation mémoire d'une application Spark

Une startup Fintech veut analyser **90 Go de transactions bancaire** (non compressés) quotidiennement pour :

- Détecter les transactions frauduleuses,
- Calculer des kpi en fonction de location et type de transaction.
- Sauvegarder ces éléments au niveau du datalakehouse

Ce traitement est orchestré via un **Spark job sur Kubernetes**, lancé chaque nuit.

Use case 15 – Étapes de calcul pour l'allocation

Estimation du volume réel

Données brutes : **90 Go**

Overhead Spark (shuffle, cache, etc.) → facteur de x2 à x3

► Volume estimé total à traiter : **~225 Go**

Définir le découpage en tâches

Règle : 1 tâche traite 300 à 500 Mo

Choix médian : 400 Mo par tâche

► Nombre de tâches = $225 \text{ Go} / 0.4 \text{ Go} = 562$ tâches

Use case 15 – + Hypothèse ajoutée

:

- Le job Spark comporte 3 actions distinctes (ex : count, groupBy, write)
- Ces actions ne s'exécutent pas en même temps
- Nous pourrions donc répartir le traitement en **3 vagues de ~187 tâches**

Use case 15 – Allocation des ressources par wave

1. Tâches par vague :

$$562 \text{ tâches} \div 3 \text{ vagues} \approx 187 \text{ tâches par vague}$$

2. Cores nécessaires par wave :

$$1 \text{ tâche} = 1 \text{ core} \Rightarrow 187 \text{ cores par vague}$$

3. Executors :

- 5 cores par executor
- $187 / 5 = 37,4 \rightarrow$ donc 38 executors par vague

Use case 15 – Mémoire nécessaire par executor

Go par tâche + overhead

→ 5 tâches par executor = 5 Go utiles

Ajoute overhead : $\text{memoryOverhead} = 1250\text{m}$

Use case 15 – Configuration finale

Paramètre	Valeur
Nombre d'executors	38
Cores par executor	5
Mémoire par executor	5 Go
Overhead	1.25 Go
Mémoire totale par executor	6.25 Go
Mémoire totale estimée	$38 \times 6.25 = \sim 238$ Go (gérée par Kubernetes)

Use case 15 – Variante 1 : config «fine grain»

- 38 executors × 5 cores
- 5 Go utiles par executor → 190 Go utiles
- Haute granularité (38 tasks parallèles)
- Souple pour Kubernetes (plus facile à placer sur plusieurs nœuds)

Use case 15 – Variante 1 : config «coarse grain »

- 20 executors × 10 cores
- 10 Go utiles par executor
- Toujours 200 Go utiles
- Besoin de nœuds avec 10 vCPU + 12.5 GiB chacun

4.1

Optimisation Spark Avancée



Optimisation Spark Avancée



1. Optimisations Spark - Part 1



2. Optimisations Spark - Part 2

Optimisation
Spark Avancée

Optimisations Spark - Part 1



Optimisation Spark - Part 1



1. Optimisation write data



3. KryoRegistrar



2. Configuration de KryoSerializer



4. Partitionnement

Use case 1 - Optimisation write data

Nous souhaiterions analyser la durée moyenne de connexion par emplacement géographique pour mieux comprendre l'utilisation du service en fonction des zones géographiques

Données :

Le fichier **user_connections.csv** contient les informations suivantes :

- **user_id** : Identifiant unique de l'utilisateur.
- **location** : Emplacement géographique de l'utilisateur.
- **connection_duration** : Durée de la connexion (en minutes).
- **service_type** : Type de service utilisé (par exemple, "standard", "premium").
- **connection_date** : Date et heure de la connexion.
- **cancelled** : Statut de la connexion (1 si annulée, 0 sinon).

Use case 1 - Optimisation write data

Étapes de Traitement

1. **Chargement des Données** : Les données sont chargées en utilisant un schéma structuré pour assurer la qualité des types de données.
2. **Agrégation** : Calcul de la durée moyenne de connexion par emplacement (location) en utilisant l'agrégation Spark.

Recommandation

spark.default.parallelism

`spark.default.parallelism` = 2 * nombre total de cœurs sur le cluster



```
spark.conf.set("spark.default.parallelism", 32)
```

Environ **1 partition pour chaque tranche de 128 Mo** de données est une règle de base. Ainsi, pour un dataset de 1 Go, le nombre de partitions recommandé serait autour de 8.

Use case 2 - Optimisation Serializer

JavaSerializer (par défaut)

- Utilise le mécanisme de sérialisation intégré de Java pour convertir les objets en un format pouvant être transmis ou stocké.

KryoSerializer (recommandé pour les charges lourdes)

- Spark propose également le **KryoSerializer**, qui est une alternative plus rapide et plus efficace.

Use case 2 - Configuration de KryoSerializer



```
spark = SparkSession.builder \  
    .appName("Optimized Serialization") \  
    .config("spark.serializer", "org.apache.spark.serializer.KryoSerializer") \  
    .getOrCreate()
```

Use case 2 - Quand passer à KryoSerializer ?

- Grands volumes de données
- Objets complexes
- Latence réseau
- Gestion consommation mémoire

Use case 3 – c'est quoi KryoRegistrar ?

- Permet de personnaliser la sérialisation en enregistrant manuellement des classes spécifiques pour Kryo.
- Évite les coûts liés à la sérialisation automatique de Spark, particulièrement pour des objets complexes.

Use case 3 – c'est quoi KryoRegistrar ?



```
--conf spark.serializer=org.apache.spark.serializer.KryoSerializer  
--conf spark.kryo.registrator=com.example.MyKryoRegistrator
```

Use case 3 – c'est quoi KryoRegistrar ?



```
import org.apache.spark.serializer.KryoRegistrar
import com.esotericsoftware.kryo.Kryo

class MyKryoRegistrar extends KryoRegistrar {
  override def registerClasses(kryo: Kryo): Unit = {
    kryo.register(classOf[MyCustomClass])
    kryo.register(classOf[AnotherClass])
  }
}
```

Use case 3 – PySpark & KryoRegistrar

Pourquoi PySpark ne prend pas en charge KryoRegistrar ?

- **Interopérabilité Python-JVM** : PySpark repose sur Py4J pour interagir avec la JVM.

Use case 4 – Introduction au Partitionnement de Données

Rôle du partitionnement :

- Optimiser le parallélisme dans les traitements Spark.
- Gérer efficacement les ressources (CPU, mémoire).
- Réduction des opérations de shuffle
- Optimisation des performances des opérations d'entrée/sortie
- Prévention des problèmes de déséquilibre (Data Skew)

Différence clé :

- `spark.default.parallelism` : Pour les **RDD**.
- `spark.sql.shuffle.partitions` : Pour les **DataFrames** et **SQL**.

Use case 4 – spark.default.parallelism

- Définit le nombre de partitions pour les RDD.
- Calcul par défaut :
 - **2 × le nombre de cœurs CPU** disponibles sur le cluster.
- Cas d'utilisation :
 - Transformations RDD telles que **map()**, **reduce()**.
- Effet d'un mauvais paramétrage :
 - Trop peu de partitions : surcharge des tâches.
- Trop de partitions : overhead inutile.



```
val rdd = spark.parallelize(Seq(1, 2, 3, 4), 4) // Partitionnement manuel
```

Use case 4 – spark.sql.shuffle.partitions

- Définit le nombre de partitions créées après une opération de **shuffle**.
- Valeur par défaut : **200** partitions.
- Cas d'utilisation :
 - Opérations **join**, **groupBy**, **aggregate** sur des DataFrames.
- Effet d'un mauvais paramétrage :
 - Trop de partitions : ralentissement dû à la gestion des métadonnées.
 - Trop peu de partitions : surcharge des nœuds.



```
spark.conf.set("spark.sql.shuffle.partitions", 50)
val df = spark.read.parquet("/path/to/data").groupBy("column").count()
```

Use case 4 – Quand utiliser `repartition()`

- **Caractéristiques :**
 - Augmente ou réduit le nombre de partitions.
 - Crée un **shuffle** complet des données.
- **Utilisation typique :**
 - Uniformiser la distribution des données entre les partitions.
 - Adapter les partitions aux ressources disponibles.



```
val dfRepartitioned = df.repartition(10)
```

Use case 4 – Quand utiliser `coalesce()`

- **Caractéristiques :**
 - Réduit le nombre de partitions sans shuffle.
 - Combine des partitions adjacentes.
- **Utilisation typique :**
 - Préparer les données pour une écriture optimisée.
 - Réduire les partitions inutiles après une opération.



```
val dfCoalesced = df.coalesce(2)
```


Use case 4 – répartition vs coalesce

Critère	repartition()	coalesce()
Shuffle	Oui	Non
Augmenter les partitions	Oui	Non
Réduire les partitions	Oui	Oui
Performance	Plus lent à cause du shuffle	Plus rapide, car sans shuffle

Adaptive Query Execution (AQE) – Spark 3.0

Active par défaut dans Spark 3.2+

- Changement dynamique du **plan d'exécution** :
 - **Join strategy switching** (e.g. broadcast ↔ shuffle)
 - **Skew join handling** (division de partitions déséquilibrées)
 - Optimisation de nombre de partitions de shuffle



```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

Use case 5 – Qu'est-ce que le Dynamic Partition Pruning (DPP)

⚠ Problème avec le Static Partition Pruning

Static pruning fonctionne uniquement quand la valeur de la partition est connue à la compilation :

```
SELECT * FROM ventes
JOIN pays ON ventes.pays = pays.nom
WHERE pays.nom = 'FR'
```

➔ Ici Spark peut pruner (filtrer) à la compilation la partition pays='FR'.

Mais si la valeur de la partition vient d'un résultat de jointure, on ne peut pas la deviner à l'avance :

```
SELECT * FROM ventes
JOIN clients ON ventes.client_id = clients.id
WHERE clients.region = 'Europe'
```

Spark ne peut pas savoir à la compilation quelles partitions de ventes sont nécessaires.

Résultat : toutes les partitions sont scannées = 🤖 performances.

Use case 5 – Qu'est-ce que le Dynamic Partition Pruning (DPP)

- Spark exécute d'abord le plan à gauche de la jointure.
- Il collecte les valeurs nécessaires à la jointure (ex: les client_id ou region).
- Il utilise ces valeurs pour filtrer les partitions à droite avant de lire la table partitionnée.



```
SELECT v.*  
FROM ventes v  
JOIN clients c  
  ON v.client_id = c.id  
WHERE c.region = 'Europe'
```

Use case 5 – DPP & Tips et limitations

Fonctionne avec :

- Tables partitionnées Parquet, ORC, Delta
- DataFrames & SQL
- Joins
- Filtres dans WHERE ou JOIN condition

⚠ Ne fonctionne pas avec :

- Filtres complexes (UDFs, LIKE, IS NULL)
- Partitions multi-colonnes mal exploitées
- API RDD ou code Spark mal structuré (si la relation join n'est pas claire)

Optimisation
Spark Avancée

Optimisations Spark - Part 2



Optimisation Spark - Part 2



1. **cache & persist**



2. **Shuffles**



3. **Garbage Collector**



4. **Broadcast et
Accumulateurs**



5. **Join**

Use case 1 – cache & persist

- **Définition :** cache() et persist() permettent de stocker des RDDs/Datasets fréquemment utilisés en mémoire pour améliorer les performances.

Use case 1 – Différence entre cache & persist

Aspect	cache()	persist()
Comportement	Stocke les données en mémoire (par défaut).	Permet de spécifier différents niveaux de stockage.
Flexibilité	Pas de choix de niveau.	Choix entre mémoire, disque, ou les deux.
Utilisation	Simplification rapide.	Contrôle plus précis des ressources.

Use case 1 – Les Niveaux de Persistance (**persist**)

Niveau	Description
MEMORY_ONLY	Stocke les données en mémoire. Si la mémoire est insuffisante, les données sont recalculées.
MEMORY_AND_DISK	Stocke en mémoire, et bascule sur disque en cas de manque de mémoire.
DISK_ONLY	Stocke uniquement sur disque.
MEMORY_ONLY_SER	Comme MEMORY_ONLY, mais sérialise les données pour économiser de la mémoire.
MEMORY_AND_DISK_SER	Comme MEMORY_AND_DISK, mais avec sérialisation.

Use case 1 – MEMORY_ONLY

- **Description :** Stocke les données en mémoire. Si la mémoire est insuffisante, les données sont recalculées.
- **Cas d'utilisation :**
- **Petits ensembles de données fréquemment réutilisés.**
 - Exemple : Vous avez un DataFrame de moins de 1 Go utilisé dans plusieurs étapes d'un pipeline, et votre cluster a suffisamment de mémoire.
- **Étapes intermédiaires coûteuses mais non volumineuses.**
 - Exemple : Calcul d'un DataFrame après un filtrage ou une agrégation légère.

Use case 1 – MEMORY_AND_DISK

- **Description :** Stocke en mémoire et bascule sur disque en cas de manque de mémoire.
- **Cas d'utilisation :**
- **Données volumineuses qui ne tiennent pas entièrement en mémoire.**
 - Exemple : Un DataFrame agrégé représentant plusieurs gigaoctets.
- **Traitement de pipelines longs avec étapes réutilisant les données.**
 - Exemple : Analyse des logs utilisateurs sur plusieurs jours avec plusieurs jointures.

Use case 1 – DISK_ONLY

- **Description :** Stocke uniquement sur disque.
- **Cas d'utilisation :**
- **Contrainte mémoire importante.**
 - Exemple : Votre cluster est utilisé par plusieurs jobs avec des ressources limitées.
- **Données rarement utilisées mais coûteuses à recalculer.**
 - Exemple : Un DataFrame résultant d'une opération de jointure complexe utilisée uniquement à la fin d'un pipeline.

Use case 1 – MEMORY_ONLY_SER

- **Description :** Comme MEMORY_ONLY, mais sérialise les données pour économiser de la mémoire.
- **Cas d'utilisation :**
- **Données volumineuses et réutilisées plusieurs fois.**
 - Exemple : Un DataFrame contenant des logs utilisateur avec des champs complexes (objets, chaînes longues).
- **Cluster limité en mémoire mais avec une bonne capacité CPU.**
 - Exemple : Analyse des clics publicitaires nécessitant de nombreuses transformations.

Use case 1 – MEMORY_AND_DISK_SER

- **Description :** Comme MEMORY_AND_DISK, mais avec sérialisation.
- **Cas d'utilisation :**
- **Gros volumes de données et contrainte mémoire importante.**
 - Exemple : Analyse de données IoT, avec plusieurs dizaines de gigaoctets de capteurs à traiter.
- **Données complexes réutilisées dans plusieurs étapes.**
 - Exemple : Un DataFrame contenant des structures JSON imbriquées pour une analyse de logs d'applications.

Use case 1 – Load, persist & write

```
// Read a DataFrame from a file
val df = spark.read
  .format("csv")
  .option("header", "true")
  .option("inferSchema", "true")
  .load("/path/to/data.csv")

// Persist the DataFrame in MEMORY_AND_DISK
df.persist(StorageLevel.MEMORY_AND_DISK)

// First Action: Perform a transformation and write to disk
val aggregatedDf = df.groupBy("category").count()
aggregatedDf.write
  .format("parquet")
  .mode("overwrite")
  .save("/path/to/output/aggregated_data")

// Second Action: Perform another transformation and write to disk
val filteredDf = df.filter("value > 100")
filteredDf.write
  .format("parquet")
  .mode("overwrite")
  .save("/path/to/output/filtered_data")

// Unpersist the DataFrame to release resources
df.unpersist()
```


Use case 1 – Load, persist & write

Étape	Sans persist	Avec persist
Lecture des données	Lecture du fichier CSV à chaque transformation	Lecture unique du fichier CSV
Première transformation	Relecture complète des données	Utilisation des données persistées
Seconde transformation	Relecture complète des données	Utilisation des données persistées
Temps d'exécution	Plus long	Plus court
Utilisation des ressources	Inefficace : accès disque répété	Efficace : données en mémoire ou disque temporaire

Use case 1 – Unpersist: Free Your Memory, Save Your Resources!

- Méthode utilisée pour libérer les ressources (mémoire et disque) allouées à un DataFrame ou RDD après qu'il a été mis en cache ou persisté.



```
val df = spark.read.format("csv").load("/path/to/data.csv")
df.persist()

// Étape 1 : Action utilisant la persistance
val result1 = df.filter("value > 100").count()

// Étape 2 : Une fois terminé, libérer les ressources
df.unpersist()

// Étape 3 : Nouvelle étape sans surcharge mémoire
val result2 = df.filter("value < 50").count()
```

Use case 2 – Shuffles

- Les **shuffles** sont des étapes coûteuses où Spark déplace des données entre partitions.
- Impact direct sur la **latence** et l'**utilisation des ressources**.
- Nécessité d'optimiser pour réduire les **I/O** et le **temps de traitement**.

Use case 2 – Minimiser les Shuffles

- Les **shuffles** sont des étapes coûteuses où Spark déplace des données entre partitions.
- Impact direct sur la **latence** et l'**utilisation des ressources**.
- Nécessité d'optimiser pour réduire les **I/O** et le **temps de traitement**.

Use case 2 – reduceByKey vs groupByKey

reduceByKey : regroupe et réduit en même temps, limitant les données échangées.

groupByKey : déplace toutes les données avant toute agrégation.



```
val rdd = sc.parallelize(List(("a", 1), ("b", 2), ("a", 3)))  
val reducedRdd = rdd.reduceByKey(_ + _)
```

Use case 2 – reduceByKey & DataFrames/Datasets

Pour remplacer reduceByKey, on utilise souvent :

- **groupBy** suivi d'une fonction d'agrégation :

```
val result = df.groupBy("key").agg(sum("value").alias("sum_value"))
```

agg pour des agrégations multiples :

```
val result = df.groupBy("key").agg(  
    sum("value").alias("sum_value"),  
    count("value").alias("count_value")  
)
```

Use case 2 – les Paramètres de Shuffle

`spark.shuffle.compress :`

- Active la compression des données échangées (par défaut : true).
- Réduit la quantité de données déplacées, économisant le réseau.

Use case 2 – les Paramètres de Shuffle

`spark.shuffle.file.buffer` :

- Définit la taille du buffer pour écrire les fichiers shuffle.
- Par défaut : 32 Ko, augmenter peut réduire le nombre d'écritures disque.

Use case 2 – les Paramètres de Shuffle

Exemple pratique :

Par exemple, si vous exécutez une jointure entre deux énormes datasets dans Spark, l'augmentation de la taille du tampon avec :



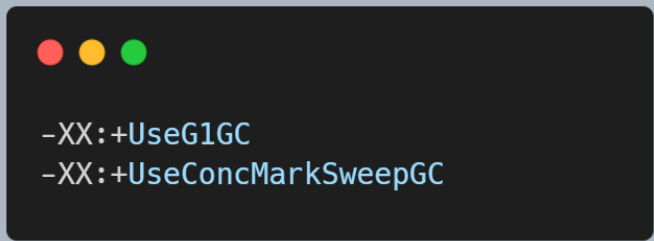
```
spark.conf.set("spark.shuffle.file.buffer", "64k")
```

Permettra de réduire la charge sur le disque pendant le shuffle. Au lieu d'écrire 10 000 blocs de 32 Ko, Spark n'écrira que 5 000 blocs de 64 Ko, économisant ainsi du temps.

Use case 3 – Garbage Collector (GC)

- **Rôle du Garbage Collector** : Gestion automatique de la mémoire.
- **Importance en Spark** : Réduction des pauses liées au nettoyage mémoire, essentiel pour les applications Big Data.

Use case 3 – Utilisation du GC Concurrent (G1GC ou CMS)



```
-XX:+UseG1GC  
-XX:+UseConcMarkSweepGC
```

- **Concurrent Mark Sweep (CMS)** : Nettoie en parallèle, réduit les pauses.
- **G1GC (Garbage First GC)** : Idéal pour les applications à faible latence.
- **Avantages :**
 - Moins de "stop-the-world".
 - Nettoyage plus régulier.

Use case 3 – Avantages de l'Optimisation du GC

- Réduction des pauses pendant le nettoyage mémoire.
- Amélioration des performances pour les jobs Spark gourmands en mémoire.
- Utilisation efficace des ressources CPU et RAM.
- Réduction des erreurs liées à la mémoire (OutOfMemoryError).

Use case 3 – Recommendations Pratiques



```
-XX:+PrintGCDetails  
-XX:+PrintGCDateStamps
```

- Activez le GC concurrent (G1GC ou CMS) pour les workloads à faible latence.
- Testez différents paramètres comme SurvivorRatio et NewSize.

Use case 4 – Broadcast et Accumulateurs

- **Variables Broadcast :**
 - Partagent de grandes données de référence entre les nœuds sans les répliquer.
 - Réduisent la consommation réseau.
 - Utilisées pour des données inchangées, comme des tables de recherche ou des configurations globales.
- **Accumulateurs :**
 - Permettent d'effectuer des agrégats globaux (exemple : compteur, somme).
 - Utiles pour collecter des métriques pendant l'exécution (exemple : nombre d'erreurs).

Use case 4 – Variables Broadcast

Définition : Partage efficace de données immuables sur tous les exécuteurs.

Pourquoi l'utiliser ?

- Évite de répéter l'envoi des mêmes données sur le réseau.
- Optimise les opérations nécessitant une table de recherche ou des données statiques.



```
val broadcastVar = spark.sparkContext.broadcast(Set("A", "B", "C"))  
val filteredRdd = rdd.filter(row => broadcastVar.value.contains(row.value))
```

Use case 4 – Accumulateurs

Définition :

Variables globales pour suivre des agrégats pendant le traitement.

Pourquoi l'utiliser ?

Suivre des métriques comme les erreurs ou anomalies.

Collecter des informations sans impacter le traitement des données.



```
val errorCount = spark.sparkContext.longAccumulator("Error Counter")
val resultRdd = rdd.map(row => {
  if (row.isError) errorCount.add(1)
  process(row)
})
println(s"Total errors: ${errorCount.value}")
```


Use case 4 – Avantages des Broadcast et Accumulateurs

Broadcast :

- Réduction des échanges réseau.
- Optimisation des traitements impliquant des données statiques.

Accumulateurs :

- Visibilité des métriques globales en temps réel.
- Faible impact sur les performances.

Use case 4 – Bonnes Pratiques

Broadcast :

- Ne pas utiliser pour des données volumineuses (> 2 Go). Définir des données immuables et souvent réutilisées.

Accumulateurs :

- Limiter leur utilisation aux métriques nécessaires.



Attention : les accumulateurs ne garantissent pas de cohérence en cas d'échec d'une tâche.

Use case 4 – Exemple Pratique

Cas concret :

Analyse des ventes avec un filtre basé sur une liste de produits autorisés (broadcast) et un compteur d'erreurs (accumulateur).



```
val allowedProducts = spark.sparkContext.broadcast(List("Product1", "Product2"))
val errorCount = spark.sparkContext.longAccumulator("Error Counter")

val filteredData = salesData.filter(row => {
  if (!allowedProducts.value.contains(row.product)) {
    errorCount.add(1)
    false
  } else true
})

println(s"Errors detected: ${errorCount.value}")
```

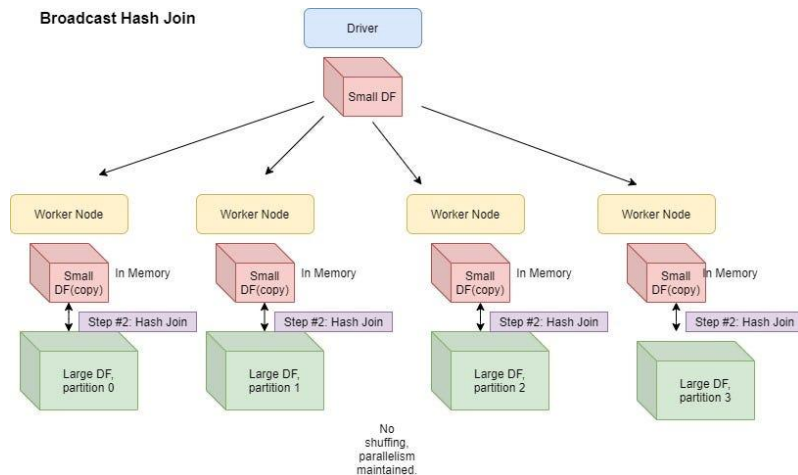
Use case 5 – Optimisation des Joins

- Les joins sont des opérations coûteuses en ressources (mémoire, CPU, réseau).
- Un **broadcast join** peut améliorer considérablement les performances en évitant le shuffle.
- **Broadcast Join :**
 - Diffuse une petite table à tous les nœuds.
 - Réduit les échanges réseau.

Use case 5 – Fonctionnement des Broadcast Joins

- Spark diffuse une table **petite et immuable** à tous les exécuteurs.
- Applicable lorsque l'une des tables est suffisamment petite pour tenir en mémoire.
- Les exécuteurs utilisent directement la table broadcastée, évitant des déplacements inutiles de données.

Use case 5 – Broadcast Hash Join (BHJ)



- Type de join par défaut lorsque l'une des tables est diffusée.
- Spark génère une table de hachage en mémoire pour la petite table (broadcast).
- Les nœuds de calcul effectuent une recherche rapide sur cette table de hachage pour joindre les enregistrements.

Cas d'utilisation :

- Lorsque l'une des tables est petite et peut tenir en mémoire.
- Idéal pour les joins simples et rapides, tels que des recherches (lookups).

Use case 5 – Broadcast Nested Loop Join (BNLJ)

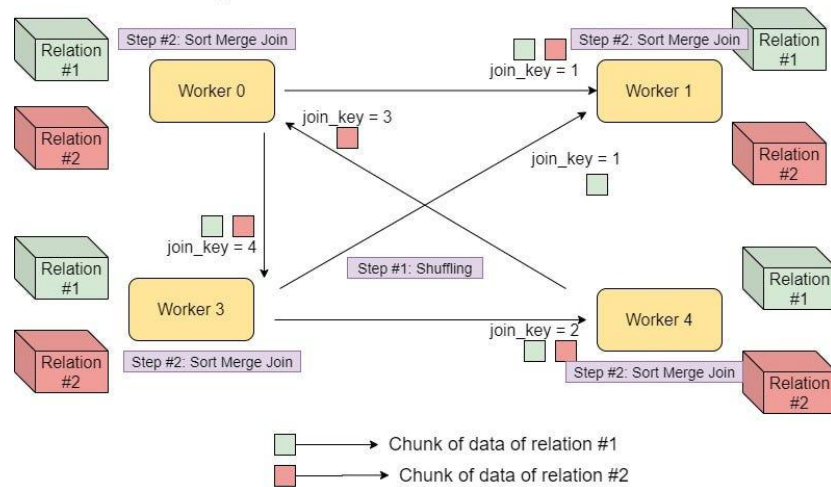
- Utilisé lorsque les conditions de jointure sont complexes ou lorsque les colonnes de jointure ne peuvent pas être utilisées pour un hachage efficace.
- Spark parcourt chaque ligne de la table principale et la compare à chaque ligne de la table broadcastée.

Cas d'utilisation :

- Conditions de jointure complexes (exemple : non-égalité ou conditions avec plusieurs colonnes).
- Recherches qui ne peuvent pas utiliser un hachage classique.

Use case 5 – Broadcast Sort Merge Join (BSMJ)

Shuffle Sort-Merge Join



- Combine un tri et une diffusion pour des performances optimales.
- Les deux tables sont triées sur les colonnes de jointure, puis une recherche par fusion est effectuée.

Cas d'utilisation :

- Lorsque la table broadcastée est petite, mais que la table principale est volumineuse et déjà triée.
- Recommandé pour des jointures avec des tables très grandes et une diffusion rapide.

Use case 5 – Broadcast Semi Join

- Utilisé pour vérifier l'existence ou la non-existence d'une correspondance dans une autre table.
- Ne retourne que les lignes de la table principale qui trouvent une correspondance.

Cas d'utilisation :

- Cas d'existence avec « IN » ou « EXISTS ».
- Exemple : Sélectionner uniquement les clients qui ont effectué des commandes.

```
df1.join(broadcast(df2), "key", "left_semi")
```

Use case 5 – Broadcast Anti Join

- Similaire au semi-join, mais retourne uniquement les lignes de la table principale qui **n'ont pas** de correspondance.

Cas d'utilisation :

- Cas de non-existence avec « NOT IN » ou « NOT EXISTS ».
- Exemple : Identifier les utilisateurs n'ayant pas encore utilisé une certaine fonctionnalité.

```
df1.join(broadcast(df2), "key", "left_anti")
```

Use case 5 – Configuration Automatique du Broadcast Join

- **Seuil de broadcast automatique :** Spark décide automatiquement d'utiliser un broadcast join si la table est inférieure au seuil défini par :
- **Seuil de broadcast automatique :** Spark décide automatiquement d'utiliser un broadcast join si la table est inférieure au seuil défini par :

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10 * 1024 * 1024)
```

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
```

Use case 5 – Résumé Broadcast Join

Type	Cas d'utilisation	Points forts	Limites
Broadcast Hash Join	Petite table, conditions simples	Rapide, efficace	Limité à des tailles de tables
Broadcast Nested Loop	Conditions complexes (exemple : non-égalité)	Flexibilité	Moins performant
Broadcast Sort Merge	Tables triées, grandes données	Performant pour grandes tables	Nécessite un tri
Broadcast Semi Join	Vérification d'existence (IN ou EXISTS)	Efficace pour les sous-ensembles	Non adapté aux résultats complets
Broadcast Anti Join	Vérification de non-existence (NOT IN)	Identification d'exceptions	Non adapté aux résultats complets

Use case 5 – Compatibilité avec AQE (Adaptive Query Execution)

Avec l'introduction d'AQE dans Spark 3.0, les Broadcast Joins sont devenus encore plus dynamiques :

- Spark peut **déterminer au moment de l'exécution** si une table doit être diffusée ou non, en fonction des statistiques collectées.
- Permet d'éviter des choix sous-optimaux basés sur des estimations incorrectes.



```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

Use case 6 – Cartesian Joins

- **Définition :** Un Cartesian Join (produit cartésien) associe chaque ligne d'un DataFrame avec toutes les lignes d'un autre DataFrame.
- **Caractéristiques :**
 - Très coûteux en termes de temps et de ressources.
 - Provoque une explosion du volume de données : $n \times m \times n \times m$ lignes.

Use case 6 – Comment Spark gère les Cartesian Joins

- Par défaut, *spark.sql.crossJoin.enabled* est réglé sur false.
- Si une condition de jointure est omise :
 - Spark lève une exception pour prévenir un produit cartésien accidentel.
- Si activé explicitement :
 - Syntaxe : *.crossJoin()*
 - Nécessite de définir *spark.sql.crossJoin.enabled* sur true.

Use case 7 – AQE what else ?

Adaptive Query Execution (AQE) : Optimisation encore plus fine des requêtes en cours d'exécution.

Optimisation du Data Skew : Gestion avancée des déséquilibres de données dans les tâches distribuées.

Compression optimisée : Meilleure gestion des données compressées avec Parquet et ORC.

Use case 8 – New in SQL

- Support amélioré pour le Dynamic Partition Pruning (DPP).
- **Nouvelles fonctions SQL** : MAP_ZIP_WITH, ANY_VALUE, ARRAY_INTERSECT, MAKE_DATE



```
SELECT ARRAY_INTERSECT(ARRAY(1, 2, 3), ARRAY(2, 3, 4)) AS result
```



```
SELECT MAKE_DATE(2024, 12, 25) AS result
```

Use case 9 – Gestion des Ressources

- **Améliorations sur Kubernetes :** Support natif pour les clusters Kubernetes 1.25 et au-delà.
- **Allocation dynamique :** Optimisation pour les workloads élastiques.
- **Meilleure intégration avec les clouds :** GCP, AWS et Azure.

Use case 10 – Dynamic Resource Allocation sur Kubernetes (Spark 3.1+)

- Comme avec YARN, Spark peut désormais ajuster dynamiquement le nombre d'executors en fonction de la charge.
- Nécessite d'activer le shuffle service interne :



```
--conf spark.dynamicAllocation.enabled=true  
--conf spark.shuffle.service.enabled=true
```

Use case 10 – Support amélioré des images Docker custom

- Support amélioré des images Docker custom



```
./bin/docker-image-tool.sh -r monrepo -t latest build  
./bin/docker-image-tool.sh -r monrepo -t latest push
```

Use case 10 – Pod template support (spark 3.x)

- Pod template support



```
--conf spark.kubernetes.executor.podTemplateFile=/templates/executor-pod.yaml
```

Use case 10 – Monitoring avec Prometheus + Grafana

- → Visualisation possible dans Grafana pour tous les pods Spark



```
--conf spark.metrics.conf=metrics.properties
```

Use case 10 – Résumé des apports clés de Spark 3 sur Kubernetes

Amélioration	Depuis	Impact
GA / support prod complet	3.1	✓ Prêt pour prod
Dynamic Allocation	3.1	✓ Économie de ressources
Pod Templates	3.x	🔧 Personnalisation avancée
Docker custom + push	3.x	🐳 Déploiement facile
Spark History Server	3.1	🔍 Suivi post-mortem
Secret / Volume Mount	3.1	🔒 Sécu + config
Retry driver auto	3.4	🔄 Résilience
Prometheus Export	3.x	📊 Monitoring avancé

Use case 10 – Exemple de spark-submit sur Kubernetes

```
spark-submit \  
  --master k8s://https://<K8S_API_SERVER>:6443 \  
  --deploy-mode cluster \  
  --name mon-job-spark \  
  --class com.wytasoft.MonAppSpark \  
  --conf spark.executor.instances=4 \  
  --conf spark.executor.memory=4g \  
  --conf spark.executor.cores=2 \  
  --conf spark.driver.memory=2g \  
  --conf spark.kubernetes.container.image=wytasoft/spark-app:latest \  
  --conf spark.kubernetes.namespace=data-apps \  
  --conf spark.kubernetes.authenticate.driver.serviceAccountName=spark-sa \  
  --conf spark.kubernetes.executor.deleteOnTermination=true \  
  --conf spark.kubernetes.driver.label.app=spark-job \  
  --conf spark.kubernetes.executor.label.app=spark-job \  
  --conf spark.dynamicAllocation.enabled=true \  
  --conf spark.shuffle.service.enabled=true \  
  --conf spark.eventLog.enabled=true \  
  --conf spark.eventLog.dir=s3a://mon-bucket/spark-logs/  
  --conf spark.hadoop.fs.s3a.access.key=AKIAxxxxx \  
  --conf spark.hadoop.fs.s3a.secret.key=xxxxxxxxxxx \  
  --conf spark.hadoop.fs.s3a.endpoint=s3.eu-west-3.amazonaws.com \  
  --conf spark.kubernetes.file.upload.path=s3a://mon-bucket/spark-jars/ \  
  local:///opt/spark/applications/mon-job.jar
```


Use case 11 – Join Hints améliorés



```
SELECT /*+ COALESCE(3) */ * FROM t;  
  
SELECT /*+ REPARTITION(3) */ * FROM t;  
  
SELECT /*+ REPARTITION(c) */ * FROM t;  
  
SELECT /*+ REPARTITION(3, c) */ * FROM t;  
  
SELECT /*+ REPARTITION_BY_RANGE(c) */ * FROM t;  
  
SELECT /*+ REPARTITION_BY_RANGE(3, c) */ * FROM t;  
  
SELECT /*+ REBALANCE */ * FROM t;  
  
SELECT /*+ REBALANCE(3) */ * FROM t;  
  
SELECT /*+ REBALANCE(c) */ * FROM t;  
  
SELECT /*+ REBALANCE(3, c) */ * FROM t;
```

Use case 12 – Coalesce & AQE

- Coalesce automatique des partitions après shuffle :
- Combine automatiquement les partitions trop petites après une opération de shuffle.



```
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

Use case 13 – skew & AQE

- Identifie les jointures déséquilibrées (data skew) et applique des stratégies d'optimisation comme le partitionnement dynamique.



```
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Use case 13 – skew & AQE

- Replanification des Broadcast Joins
- Permet de redéfinir un broadcast join si une table broadcast est trop grande.

```
spark.conf.set("spark.sql.adaptive.localShuffleReader.enabled", "true")
```

5.1

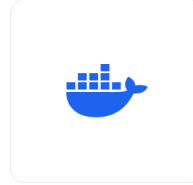
Overview Spark Shell & docker



Overview Spark **Shell & docker**



1. Connexion à Spark Shell



2. Introduction à Docker

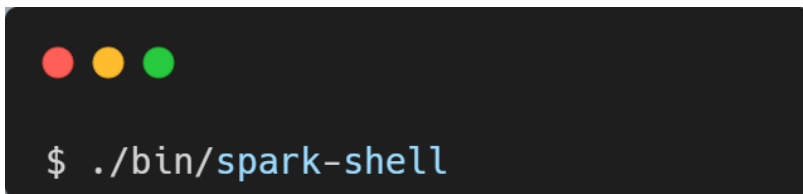
Prise en Main Docker

Connexion à Spark Shell



Connexion à Spark Shell

- Démarrer avec Spark Shell
 - Lancement de Spark Shell : Pour démarrer Spark Shell, vous devez avoir Spark installé. Une fois cela fait, exécutez la commande suivante dans votre terminal :

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The command `./bin/spark-shell` is entered at the prompt.

```
$ ./bin/spark-shell
```


Connexion à Spark Shell

- Démarrer avec Spark Shell
 - Lancement de Spark Shell : Pour démarrer Spark Shell, vous devez avoir Spark installé. Une fois cela fait, exécutez la commande suivante dans votre terminal :



```
$ ./bin/spark-shell
```

- Python (PySpark Shell) : Si vous souhaitez utiliser Python (via PySpark), utilisez la commande suivante :



```
./bin/pyspark
```

Hdfs avec Spark Shell

- **hadoop fs** est l'interface pour interagir avec le système de fichiers distribué HDFS via des commandes Hadoop. Cela vous permet de manipuler des fichiers, d'effectuer des opérations comme la copie, la suppression, la lecture, ou l'écriture de fichiers dans HDFS.
- **-put** est l'option qui permet de copier des fichiers ou des répertoires locaux vers HDFS. C'est similaire à la commande `cp` sous Linux, mais elle transfère les fichiers depuis votre système de fichiers local vers le système distribué HDFS.



```
hadoop fs -put "C:/Users/20210706/file.csv" "/project/training/20210706/"
```

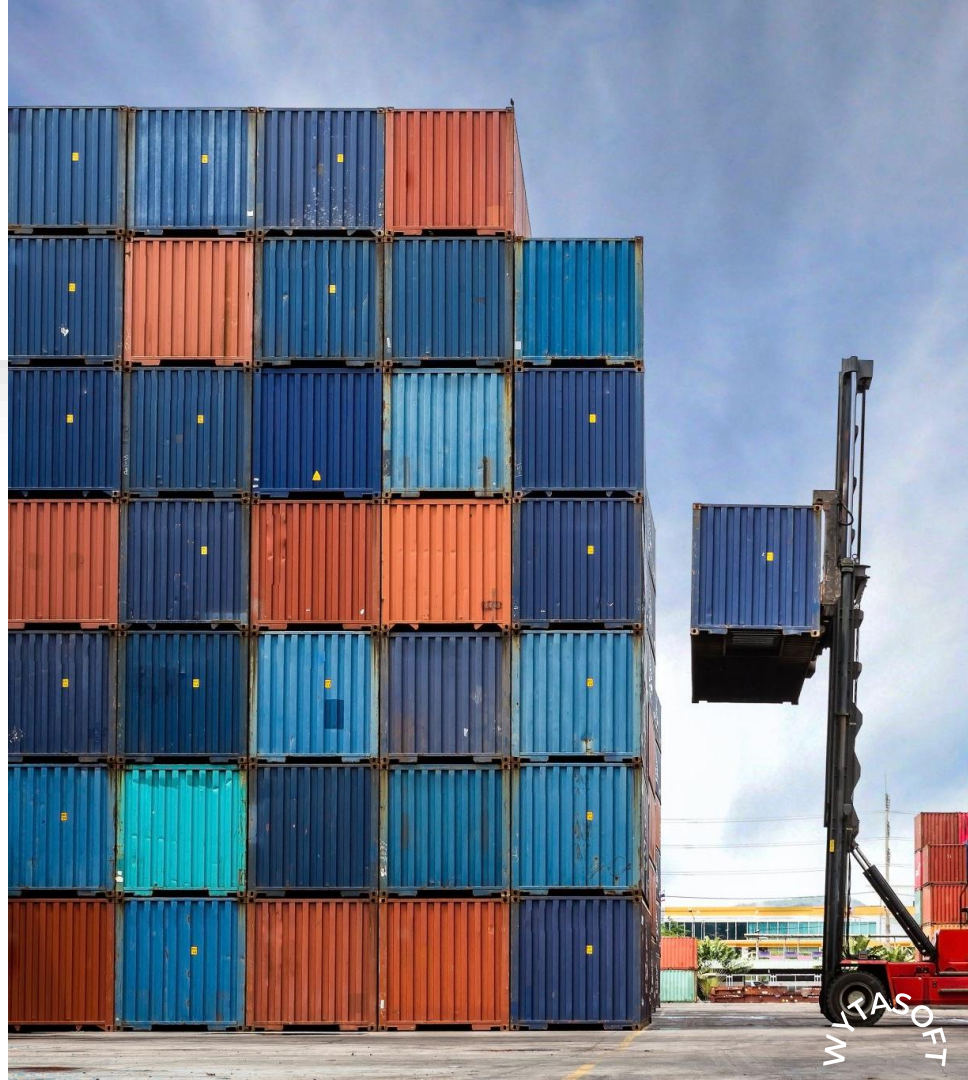
Prise en Main
Spark Shell



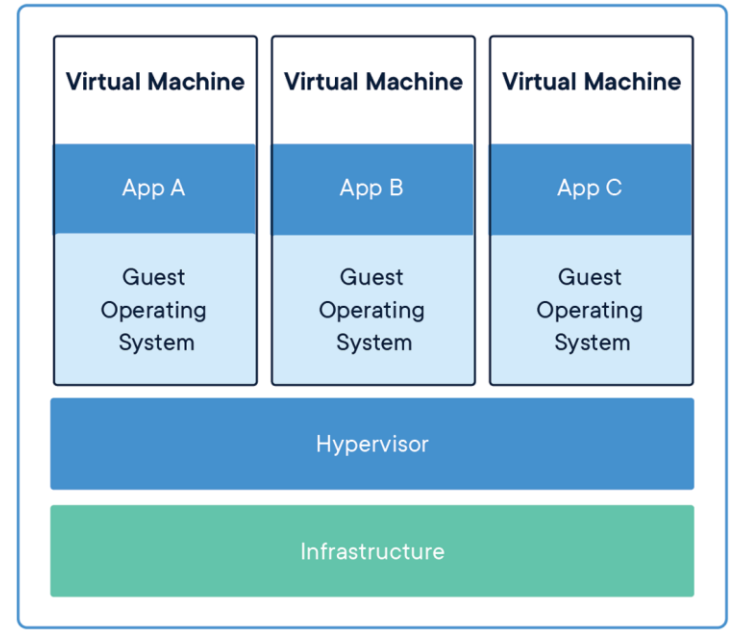
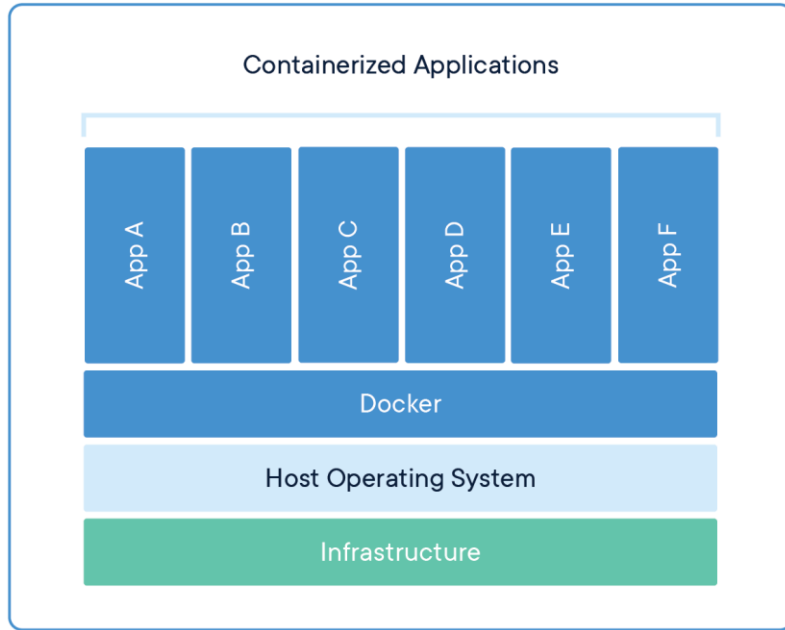
Introduction à
Docker

Qu'est-ce que **Docker** ?

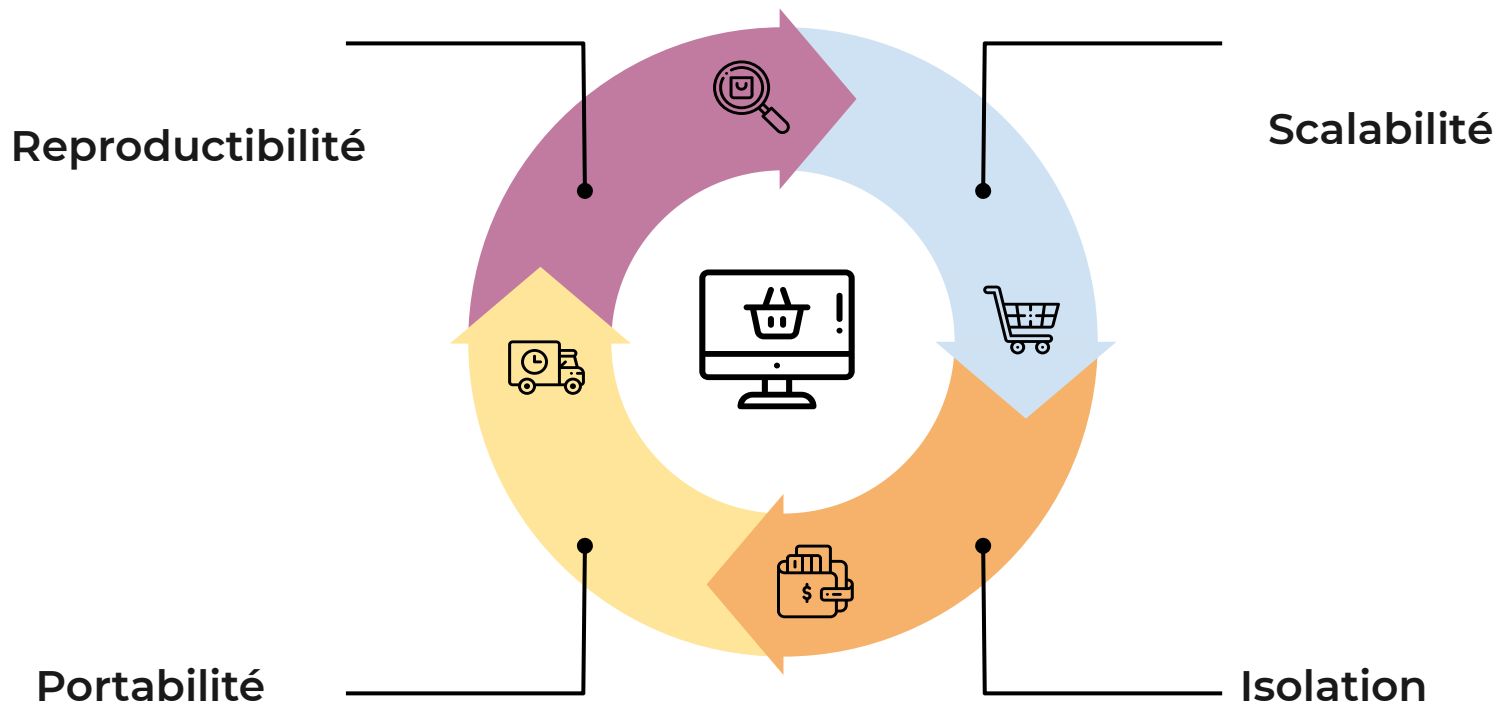
- Docker est une plateforme qui permet de développer, déployer, et exécuter des applications dans des conteneurs. Un conteneur est un package léger, autonome et exécutable qui inclut tout ce dont une application a besoin pour fonctionner : le code, les bibliothèques, et les dépendances.



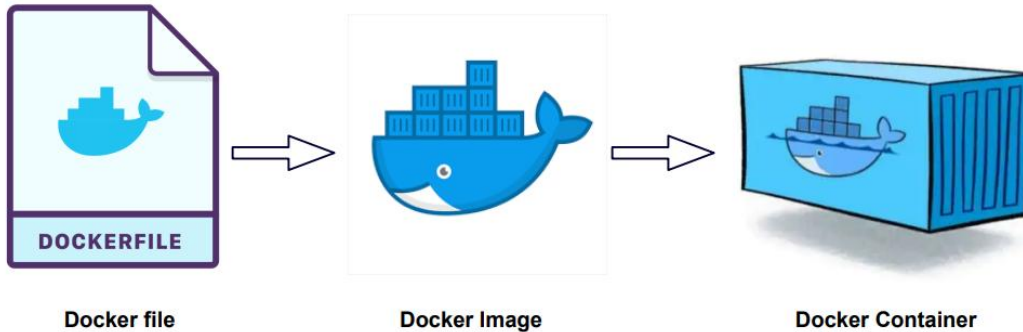
Containerisation vs Virtualisation



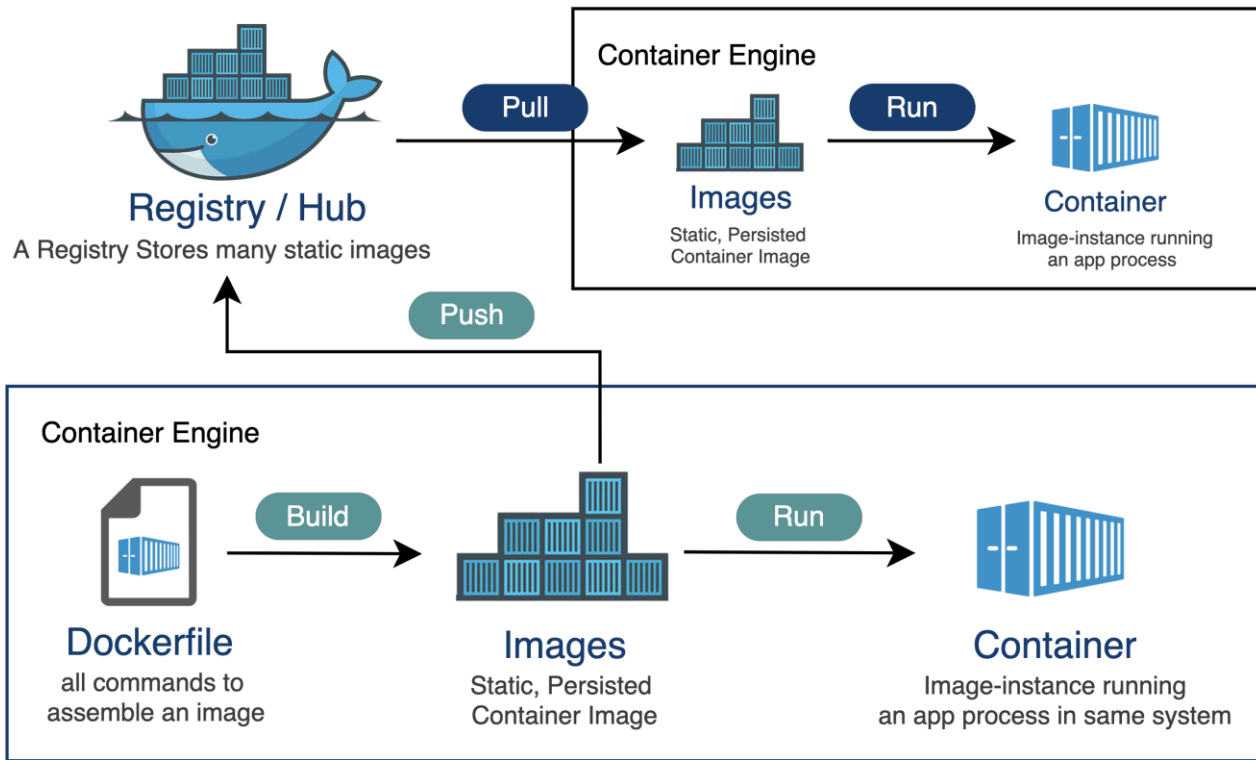
Avantages de Docker en Data



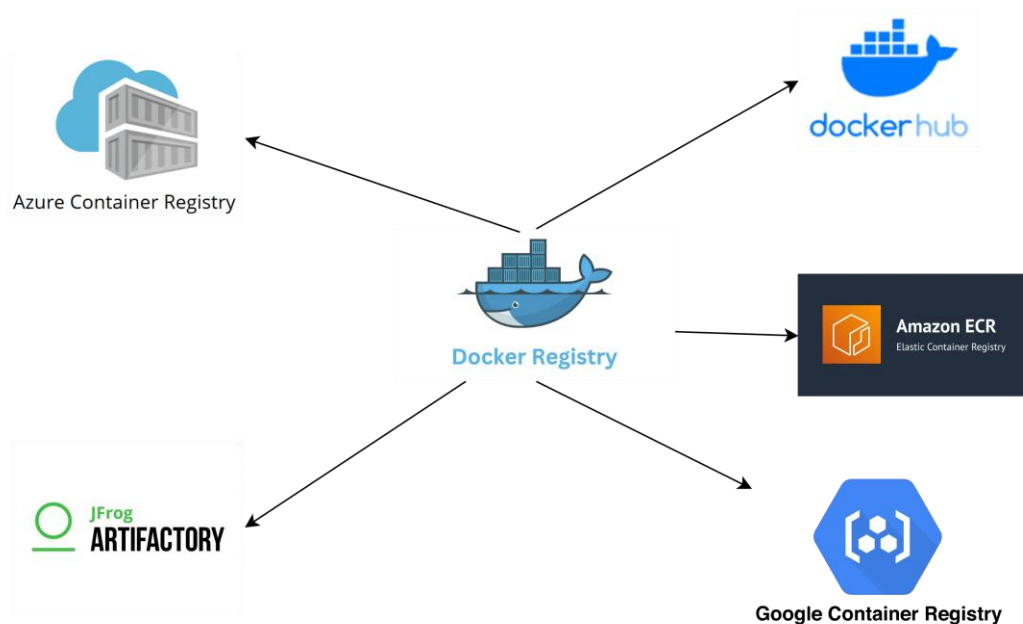
Concepts clés de Docker



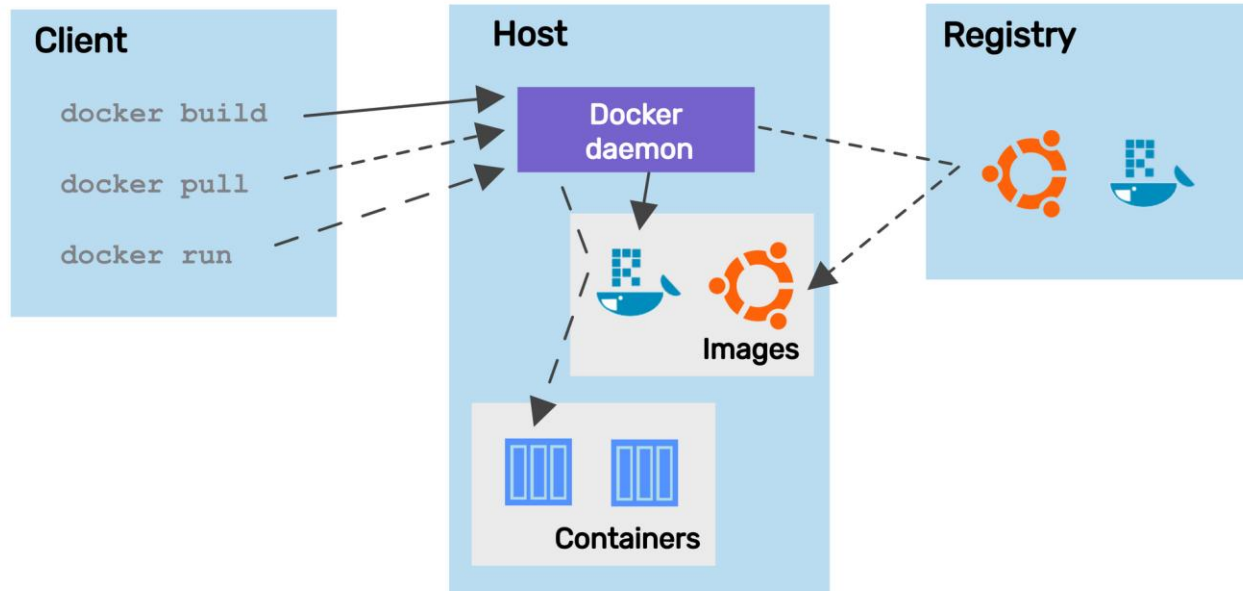
Docker Registry



Docker Registry

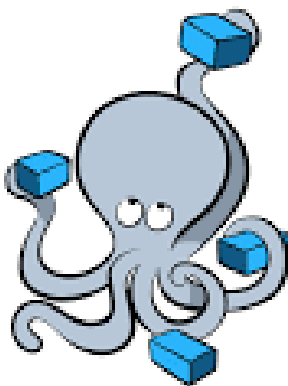


Architecture Docker



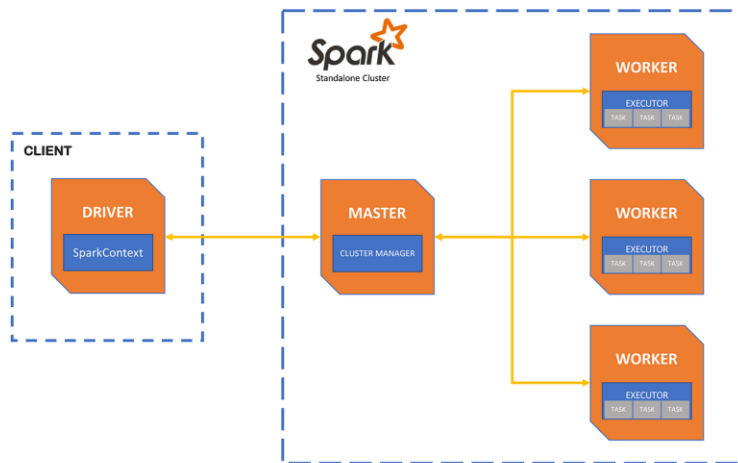
Docker Compose

- Un outil utilisé pour définir et exécuter des applications multi-conteneurs Docker. Il permet de gérer facilement des environnements composés de plusieurs conteneurs qui interagissent entre eux.



docker
Compose

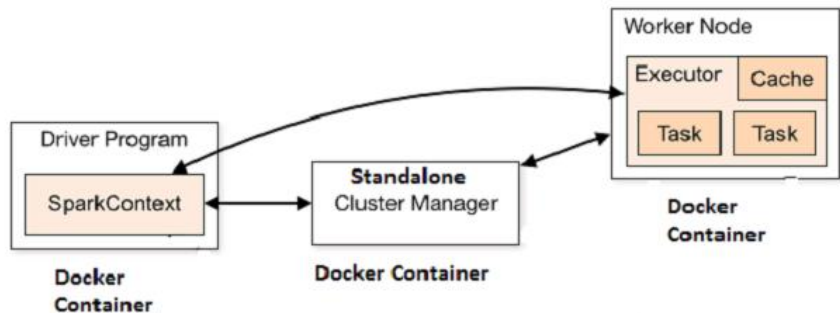
Docker Compose



L'idée est d'utiliser Docker Compose pour orchestrer plusieurs services :

- Un service pour Spark Master
- Un service pour Spark Worker
- Un service pour ton application PySpark
- Optionnel : Un service pour un système de fichiers distribué comme HDFS (ou simplement des volumes Docker)

Docker Compose



L'idée est d'utiliser Docker Compose pour orchestrer plusieurs services :

- Un service pour Spark Master
- Un service pour Spark Worker
- Un service pour ton application PySpark
- Optionnel : Un service pour un système de fichiers distribué comme HDFS (ou simplement des volumes Docker)

Pourquoi utiliser Docker Compose ?



Orchestrer facilement



Simplifier la mise en place



Favoriser la portabilité

Key commands Docker Compose ?

 Docker Compose

```
docker compose up
```

Démarrer et **d'orchestrer** plusieurs conteneurs Docker définis dans un fichier `docker-compose.yml`

 Docker Compose

```
docker compose down
```

Permet de **stopper** et **supprimer** tous les conteneurs,

 Docker Compose

```
docker compose logs
```

Permet de **visualiser les logs**

 Docker Compose

```
docker compose ps
```

Permet de **lister l'état** des conteneurs gérés par Docker Compose

Use case Docker Compose

Configurer le fichier Docker Compose :

Créer un fichier docker-compose.yml pour définir les services :

Spark Master : Service qui exécute le nœud maître du cluster Spark. Il sera responsable de la coordination des tâches.

Spark Worker : Service qui exécute les nœuds travailleurs du cluster Spark. Ils effectueront les calculs distribués.

Application PySpark : Service qui exécute l'application PySpark, en se connectant au nœud maître Spark et en envoyant les tâches de traitement des données.

Quiz : Prise en Main Docker

<https://www.wytasoft.com/wytasoft-group/index.php/wytasoftacademy/>



05

The Lakehouse Revolution



The Lakehouse Revolution

Overview Spark Delta Lake



Overview Spark **Delta Lake**



1. **ACID**



2. **Schema Enforcement & Evolution**



3. **Time Travel**



4. **Scaling Metastore**



5. **Upserts & Deletes**



6. **Indexing & Z-Ordering**



7. **Batch & Streaming**



8. **Lecture et d'Écriture**



9. **Databricks**



10. **Conversions de Tables**

Use case 1 – ACID Transactions et Spark

ACID : Ensemble de propriétés garantissant l'intégrité des transactions dans un système de gestion des bases de données.

- **Atomicité (A)** : Une transaction est réalisée entièrement ou pas du tout.
- **Cohérence (C)** : Les données doivent rester valides et conformes aux règles métier avant et après la transaction.
- **Isolation (I)** : Les transactions concurrentes ne se perturbent pas.
- **Durabilité (D)** : Une fois validée, une transaction reste permanente, même en cas de panne.

Use case 1 – Spark est-il ACID ?

Spark, en tant que framework de traitement de données, n'est pas intrinsèquement ACID.

Pourquoi ? :

- Spark est conçu pour des traitements analytiques massifs, pas pour des transactions en ligne.
- Les DataFrames et RDDs ne garantissent pas les propriétés ACID.

Solution ? :

- Utiliser des frameworks ou des systèmes qui s'intègrent avec Spark pour ajouter la gestion ACID, comme :
 - **Delta Lake.**
 - **Apache Hudi.**
 - **Apache Iceberg.**

Use case 1 – Principe de fonctionnement des ACID Transactions

1. Atomicité :

1. Les opérations partiellement exécutées sont annulées.
2. Exemple : Si une insertion échoue, rien n'est ajouté.

2. Cohérence :

1. Maintien des contraintes d'intégrité (exemple : clé étrangère).
2. Les données sont toujours dans un état valide.

3. Isolation :

1. Les transactions concurrentes n'interfèrent pas entre elles.
2. Exemple : Une lecture ne voit pas une transaction en cours.

4. Durabilité :

1. Les modifications restent valides même après un crash.
2. Implémentée via des journaux ou des snapshots.

Use case 1 – Atomicité (A)

Propriété ACID : Une transaction est entièrement réalisée ou annulée. Si une étape échoue, tout l'ensemble doit être annulé.

Ce que Spark n'assure pas :



```
df.write.parquet("/output/path")
```

- Lorsqu'une tâche Spark échoue en cours d'exécution (par exemple, une tâche d'écriture), les fichiers partiellement écrits ou corrompus peuvent rester dans le système de stockage.
- Spark ne fournit pas un mécanisme intégré pour "rollback" automatique.

Use case 1 – Cohérence (C)

Propriété ACID : Les données restent toujours valides et conformes aux règles métier ou aux contraintes définies.

Ce que Spark n'assure pas :

- Spark ne valide pas automatiquement les contraintes, comme les clés primaires, les clés étrangères, ou d'autres règles métier.
- Les utilisateurs doivent manuellement s'assurer de la cohérence des données via du code ou des vérifications externes.



```
val newDf = df.withColumn("id", lit(1))  
newDf.write.mode("append").parquet("/output/path")
```

Use case 1 – Isolation (I)

Propriété ACID : Les données restent toujours valides et conformes aux règles métier ou aux contraintes définies.

Ce que Spark n'assure pas :

- Spark ne valide pas automatiquement les contraintes, comme les clés primaires, les clés étrangères, ou d'autres règles métier.
- Les utilisateurs doivent manuellement s'assurer de la cohérence des données via du code ou des vérifications externes.



```
val newDf = df.withColumn("id", lit(1))  
newDf.write.mode("append").parquet("/output/path")
```

Use case 1 – Durabilité (D)

Propriété ACID : Une fois une transaction validée, ses effets doivent être permanents, même en cas de panne.

Ce que Spark n'assure pas :

- Spark ne gère pas nativement la durabilité. Si une écriture est en cours et qu'une panne survient, le système peut perdre des données ou laisser des fichiers partiellement écrits.
- Spark ne fournit pas de journal de transaction pour garantir que toutes les écritures soient finalisées.

Use case 1 – Spark et Delta Lake

Delta Lake : Extension open-source de Spark pour gérer les transactions ACID.

Avantages :

- Transactions ACID pour Spark.
- Gestion des versions des données (Time Travel).
- Écriture optimisée avec le format Parquet.



```
val df = spark.read.format("delta").load("/path/to/delta-table")
df.write.format("delta").mode("append").save("/path/to/delta-table")
```

Use case 2 – Schema Enforcement & Evolution

Définition :

- Schema Enforcement : Contrôle strict pour s'assurer que les données respectent un schéma prédéfini.
- Schema Evolution : Capacité à modifier le schéma tout en conservant la compatibilité avec les anciennes données.

Pourquoi c'est important ?

- Prévenir les erreurs de format ou d'incohérence dans les données.
- Garantir la continuité des traitements lorsque les besoins changent.

Use case 2 – Schema Enforcement - Contrôle Strict

Objectif : Prévenir l'injection de données non conformes.

Exemples d'outils et frameworks :

- **Spark** : Contrôle lors de la lecture/écriture de données.
- **Delta Lake** : `enforceSchema=true` lors des écritures.

Modes de validation :

- Validation stricte : Rejet des données non conformes.
- Tolérance partielle : Ajout de valeurs par défaut pour les colonnes manquantes.

Use case 2 – Schema Evolution - Adaptabilité des Données

Objectif : Faciliter l'ajout ou la modification de colonnes dans un schéma existant.

Types d'évolutions :

- Ajout de colonnes (backward compatible).
- Suppression de colonnes (non compatible sans migration).
- Modification des types de colonnes (ex. : int vers long).

Exemples d'implémentation :

- Spark : mergeSchema=true pour l'écriture.
- Delta Lake, Apache Hudi, Apache Iceberg.

Use case 2 – Schema Enforcement & Evolution

Critère	Schema Enforcement	Schema Evolution
Objectif	Cohérence et validation	Adaptabilité
Impact des erreurs	Rejet des données non valides	Mise à jour automatique
Exemples	Spark, Hive	Delta Lake, Hudi, Iceberg

Use case 3 – Time Travel (Versioning) avec Delta Lake

Qu'est-ce que le Time Travel ?

- Une fonctionnalité de Delta Lake permettant d'interroger et de restaurer des versions antérieures des données.

Pourquoi utiliser le Time Travel ?

- Déboguer des problèmes en examinant des états historiques des données.
- Auditer les modifications apportées aux données.
- Reproduire des résultats pour des exigences réglementaires ou de conformité.

Use case 3 – Fonctionnement du Time Travel

- **Journal des Transactions** : Delta Lake utilise un fichier `_delta_log` pour enregistrer toutes les modifications apportées aux données.
- **Interrogation des versions historiques via :**
 - **Numéro de version** : `versionAsOf`.
 - **Horodatage** : `timestampAsOf`.
- **Exemple d'évolution des données :**
 - Version 0 : Données initiales.
 - Version 1 : Mise à jour du schéma.
 - Version 2 : Correction des données.

Use case 3 – Exemple avec Numéro de Version



```
# Interroger les données à la version 0
df_version_0 = spark.read.format("delta").option("versionAsOf", 0).load("/tmp/delta/flights_data")
df_version_0.show()
```

Use case 3 – Exemple avec Horodatage



```
# Interroger les données à un moment précis
df_timestamp = spark.read.format("delta").option("timestampAsOf", "2023-11-01T12:00:00").load("/tmp/delta/flights_data")
df_timestamp.show()
```

Use case 3 – Advantages du Time Travel



Debugging



Auditability



Reproducibility



Flexibility

Use case 4 – Scaling Metastore avec Delta Lake

Problèmes des Metastores traditionnels :

- Latence élevée pour les tables avec des millions de partitions.
- Défaillances fréquentes dues à une surcharge des métadonnées.
- Goulots d'étranglement dans l'exécution des requêtes.

Use case 4 – Scaling Metastore avec Delta Lake

Comment Delta Lake améliore la scalabilité :

- Gestion distribuée des métadonnées :
 - Les métadonnées sont stockées dans des fichiers distribués au lieu d'un Metastore central.
- Gestion des partitions :
 - Plus besoin de gérer explicitement les partitions.
- Fichiers de métadonnées compacts :
 - Réduction de la taille et de la complexité des métadonnées.

Use case 4 – Avantages de Scaling Metastore avec Delta Lake



Support pour le Dynamic Partition Pruning.



Support pour le Dynamic Partition Pruning.



Support pour le Dynamic Partition Pruning.

Use case 4 – Partition Pruning

Activer le Dynamic Partition Pruning dans Spark :



```
spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.enabled", "true")
```

Use case 4 – Partition Pruning

Configurations supplémentaires pour optimiser le DPP

Pour maximiser les performances du Dynamic Partition Pruning, voici des configurations utiles :



```
spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.fallbackFilter.enabled", "true")
```

Use case 4 – Partition Pruning

Configurations supplémentaires pour optimiser le DPP

Pour maximiser les performances du Dynamic Partition Pruning, voici des configurations utiles :



```
spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.fallbackFilter.enabled", "true")
```

Use case 4 – Partition Pruning

Configurer un timeout pour le partition pruning :



```
spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.timeout", "300")
```

Use case 4 – Journaux transactionnels

Comment fonctionnent les journaux transactionnels dans Delta Lake ?

- Stockage structuré des métadonnées : Les journaux transactionnels sont stockés sous forme de fichiers JSON dans le répertoire `_delta_log`.
- Chaque opération (insertion, mise à jour, suppression) génère un nouveau fichier de journal contenant des métadonnées détaillées (versions, schéma, partitions modifiées, etc.).

Use case 4 – Journaux transactionnels

Structure des journaux transactionnels dans Delta Lake ?

```
{
  "add": {
    "path": "part-00000-xyz.snappy.parquet",
    "size": 123456,
    "partitionValues": {
      "year": "2023",
      "month": "05"
    },
    "modificationTime": 1685467982000,
    "dataChange": true
  }
}
```

Répertoire `_delta_log` :

- Contient tous les fichiers de journaux transactionnels d'une table.
- Chaque fichier représente une opération et est nommé avec un numéro incrémental, par exemple : 0000000000000000000001.json.

Exemple de contenu d'un journal transactionnel

Use case 4 – Journaux transactionnels

Pourquoi les journaux transactionnels sont-ils évolutifs ?

Optimisation des métadonnées :



```
spark.sql("OPTIMIZE '/wytasoft/project/delta/table'")
```

- Les fichiers JSON de journaux sont compactés périodiquement en fichiers Parquet via un processus de **compaction**.
- Commande pour compacter les journaux

Use case 4 – Journaux transactionnels

Lecture et écriture parallèles :

- Les journaux sont accessibles simultanément par plusieurs nœuds dans un cluster distribué, garantissant des performances élevées.

Use case 4 – Journaux transactionnels

Garbage collection :

- Les journaux obsolètes (versions anciennes) sont automatiquement nettoyés pour éviter une surcharge de stockage



```
spark.conf.set("spark.databricks.delta.retentionDurationCheck.enabled", "false")
```

Use case 5 – Upserts & Deletes

Définition :

- Upserts : Combinaison d'insertions et de mises à jour.
- Deletes : Suppressions directes de lignes basées sur des conditions.

Problématique :

- Avec des formats classiques (ex. Parquet), il est difficile de gérer les mises à jour et suppressions sur des données volumineuses.

Solution :

- Delta Lake et Databricks simplifient ces opérations grâce aux journaux transactionnels et à l'API MERGE INTO.

Use case 5 – Upserts & Deletes

ACID Transactions :

- Maintien de la cohérence des données lors des mises à jour ou suppressions.

Optimisation :

- Utilisation de journaux transactionnels.
- Écriture incrémentielle évitant la réécriture complète.

Simplicité d'utilisation :

- API SQL (MERGE INTO, DELETE FROM).
- Intégration avec PySpark, Scala, SQL.

Use case 5 – Exemple de Code : Upserts avec MERGE INTO

```
MERGE INTO customers AS target
USING updates AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
UPDATE SET target.name = source.name, target.age = source.age
WHEN NOT MATCHED THEN
INSERT (customer_id, name, age) VALUES (source.customer_id, source.name, source.age)
```

Use case 5 – Exemple de Code : Deletes



```
DELETE FROM customers  
WHERE last_purchase_date < '2023-01-01'
```

Use case 6 – Indexing & Z-Ordering

Delta Lake propose des optimisations avancées pour améliorer la performance des requêtes :

- **Indexing** : Facilite l'accès rapide aux données.
- **Z-Ordering** : Optimise la répartition des données sur disque pour des filtres courants.

Use case 6 – Indexing : Accélérer l'Accès aux Données

Qu'est-ce que l'Indexing ?

- Création de structures de données pour faciliter les recherches.

Types d'Index dans Delta Lake :

- **Data Skipping Indexes** : Filtrage rapide des fichiers non pertinents.
- **Cluster Indexing** : Regroupe des données similaires pour améliorer les requêtes.

Use case 6 – Z-Ordering : Optimisation des Données

- Réorganisation des données selon plusieurs colonnes.
- Priorise les colonnes souvent utilisées dans les filtres.



```
OPTIMIZE '/delta/wyatasoft/path'  
ZORDER BY (date, location)
```

Exemple :

- Z-Ordering sur les colonnes date et location.
- Réduit le nombre de fichiers scannés pour les requêtes filtrant par date et lieu.

Use case 6 – Avantages de l'Indexing et du Z-Ordering



Debugging



Z-Ordering

Use case 7 – Batch & Streaming

- Delta Lake permet un traitement unifié des données en **batch** et en **streaming**.
- La même table peut être utilisée pour **insérer**, **lire**, et **analyser** des données provenant de flux en temps réel ou de traitements par lots.

Use case 7 – Batch & Streaming

- **Lecture en streaming** : Lire les mises à jour en temps réel d'une table Delta.
- **Écriture en streaming** : Ajouter de nouvelles données en temps réel dans une table Delta.
- **Support du batch** : Effectuer des analyses complexes ou des transformations sur des ensembles de données historiques.

Use case 7 – Batch & Streaming

- **Simplification** : Plus besoin de maintenir des pipelines distincts pour le batch et le streaming.
- **Fiabilité** : Les transactions ACID assurent la cohérence entre les deux modes.
- **Flexibilité** : Possibilité de passer d'un mode batch à streaming selon les besoins.

Use case 7 – Batch & Streaming

```
from pyspark.sql import SparkSession

# Créer une SparkSession
spark = SparkSession.builder \
    .appName("Delta Unification") \
    .getOrCreate()

# Lecture en streaming
streaming_df = spark.readStream.format("delta").table("delta_table")

# Écriture en batch
batch_df = spark.read.format("delta").table("delta_table")
batch_df.write.format("delta").mode("append").save("/wytasoft/delta_table")

# Écriture en streaming
streaming_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", "/wytasoft/checkpoint") \
    .start("/wytasoft/delta_table")
```

Use case 7 – Use case Batch & Streaming



Analyse en temps réel



Mises à jour en
temps réel



Alertes prédictives

Use case 8 – Lecture et d'Écriture

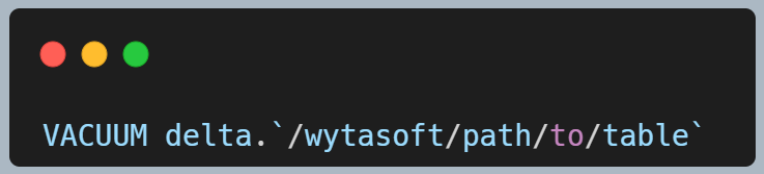
- Delta Lake et Databricks fournissent des mécanismes avancés pour améliorer les performances de lecture et d'écriture.
- Ces optimisations sont particulièrement importantes pour les grands ensembles de données et les workloads distribués.

Use case 8 – Lecture et d'Écriture

Configuration Spark:

- spark.sql.files.maxPartitionBytes
- spark.databricks.io.cache.enabled

Compaction:



```
VACUUM delta.`/wytasoft/path/to/table`
```


Use case 9 – Databricks

Gestion des métadonnées :

- Metastore intégré pour un accès rapide et cohérent.

Optimisation des requêtes :

- Support des Adaptive Query Execution (AQE).
- Amélioration automatique des performances.

Outils avancés :

- Job Scheduler : Automatisation des workflows.
- Delta Live Tables : Gestion des pipelines en temps réel.

Use case 9 – Databricks

Assurez la sécurité de vos données

Support de Unity Catalog :

- Gestion centralisée des permissions.
- Intégration avec Delta Lake pour une sécurité renforcée.

Audit et conformité :

- **Time Travel** pour l'audit des données.
- Suivi des changements grâce aux journaux transactionnels.

Use case 9 – Databricks

Layer	Staging		Core	Presentation	
Medallion	Bronze		Silver	Gold	Platinum
Sub-layer	Raw	Standardised	Conformed	Data Mart	Report
Purpose	Stockage As-Is des Données Stockage des données dans leur format original. Préservation de la version exacte des données pour l'auditabilité. Centralisation des Données Point de collecte unique pour toutes les sources de données. Rassemblement de données structurées, semi-structurées et non structurées. Historisation des Données Conservation de toutes les versions des données. Possibilité de revenir à une version antérieure en cas d'erreurs dans les étapes ultérieures. Flexibilité Les données restent non modifiées, permettant des transformations futures sans contraintes. Répond aux évolutions des besoins et exigences. Sécurité et Conformité Application de mesures de sécurité, comme le chiffrement des données sensibles. Assure la conformité réglementaire en matière de gestion des données. Optimisation des Coûts Utilisation de formats de compression pour réduire les coûts de stockage. Stockage à faible coût pour une gestion économique des données.	Nettoyage des Données Correction des erreurs, des valeurs manquantes et des incohérences dans les données. Élimination des doublons. Standardisation des Données Application de formats et de standards communs pour assurer la cohérence. Transformation des données dans des structures communes pour faciliter les analyses ultérieures. Enrichissement des Données Ajout d'informations supplémentaires ou de métadonnées pour augmenter la valeur des données. Combinaison de sources de données pour obtenir une vue plus complète. Historisation des Transformations Suivi des modifications et des transformations appliquées aux données pour garantir la traçabilité. Gouvernance et Documentation Application de règles métier pour assurer la qualité des données. Documentation des procédures et des standards appliqués pour garantir la transparence et la reproductibilité. Optimisation des Accès Stockage des données dans des formats optimisés pour les requêtes et les analyses ultérieures. Préparation des données pour les étapes suivantes de l'architecture, telles que la modélisation ou la présentation.	Référentiel Central d'Entreprise Serve comme dépôt centralisé pour les entités métier majeures (par exemple, clients, produits, transactions). Assure la cohérence des données à travers l'entreprise. Source Unique de Vérité Fournit une vue fiable et unique des données, éliminant les ambiguïtés ou les redondances. Intégration des Sources Multiples Combine et harmonise les données provenant de multiples sources pour créer une vue unifiée. Assure la cohérence entre différentes sources de données. Application des Règles Métier Imposition de règles métier pour garantir l'intégrité et la validité des données. Transformation des données selon les besoins métier spécifiques. Maintien de l'Historique Utilise des techniques comme le Partitioning pour suivre et gérer les changements au fil du temps. Enrichissement des Données Ajout d'informations de référence pour améliorer la compréhension et la valeur des données. Optimisation pour la Consultation Les données sont stockées de manière à faciliter les requêtes fréquentes et à améliorer les performances des analyses.	Données Optimisées pour l'Analyse Création de structures de données telles que des étoiles ou des flocons de neige (modèles étoiles ou flocons de neige) pour optimiser les performances des requêtes. Utilisation de schémas de type Kimball pour faciliter les analyses OLAP. Vue Segmentée des Données Fournit des sous-ensembles de données adaptés aux besoins spécifiques de différents départements ou fonctions métier. Permet une personnalisation en fonction des besoins de l'utilisateur final. Facilitation de l'Auto-service Analytique Les utilisateurs peuvent créer leurs propres rapports, tableaux de bord et analyses sans dépendre d'une équipe IT. Promouvoir la découverte de données et les insights par les utilisateurs métier. Intégration de Données Factuelles et Dimensionnelles Combinaison de tables de faits et de dimensions pour fournir une vue utilisateur complète. Facilite les analyses multidimensionnelles. Performance Optimale Les données sont souvent agrégées ou pré-agrégées pour accélérer les requêtes. Utilisation de technologies et d'index appropriés pour améliorer les temps de réponse. Flexibilité pour les Utilisateurs Finaux Permet aux utilisateurs de combiner, filtrer et segmenter les données selon leurs besoins. Fournit les outils nécessaires pour répondre à des questions métier spécifiques.	Préparation des Données pour la Consommation Finale Fournit des ensembles de données qui sont immédiatement utilisables par les outils de reporting et de visualisation. Les données sont souvent pré-agrégées pour faciliter les requêtes rapides et la génération de rapports. Vue Holistique des Données Combinaison de plusieurs ensembles de données pour offrir une vue complète pour l'analyse. Intégration de données factuelles et dimensionnelles pour un contexte complet. Optimisation des Performances Les structures de données et les index sont conçus pour des requêtes de reporting rapides. Réduction du temps nécessaire pour générer des rapports ou des dashboards. Accès Facilité pour les Utilisateurs Non Techniques Interface conviviale pour les utilisateurs métier et les cadres dirigeants pour accéder aux rapports. Supporte l'auto-service, permettant aux utilisateurs de créer et de personnaliser leurs propres rapports. Garantie de la Qualité et de la Fiabilité des Données Mise en place de contrôles de qualité pour s'assurer que les données sont exactes et à jour. Les données sont souvent validées et corrigées à cette étape pour garantir la précision des rapports. Support pour les Outils de Visualisation Compatible avec les outils populaires de visualisation de données et de BI (Business Intelligence) pour une intégration transparente. Les données sont formatées de manière à être facilement visualisables.
Consumers	Data Engineering	Data Analysts, Engineering & Scientists	Operational analyst, Data analysts, Data Science	BI analysts, Head of departments, Data Science	Executives

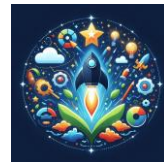
Avantages d'une Architecture Data Layers



Organisation des données



Optimisation des coûts



Amélioration des performances



Flexibilité



Gestion du cycle de vie
des données



Réduction des
redondances



Évolutivité



Sécurité renforcée



Agilité

Use case 10 – Conversions de Tables

Pourquoi convertir une table en table Delta ?

- **Delta Lake** permet une conversion facile de tables existantes en tables Delta.
- **Objectif :** Améliorer les performances et bénéficier des fonctionnalités avancées comme ACID Transactions, Time Travel, et Z-Ordering.
- Compatible avec des formats existants comme Parquet et Hive.

Use case 10 – Conversions de Tables

Quels sont les bénéfices ?

- **Transactions ACID** : Garantit la cohérence et l'intégrité des données.
- **Time Travel** : Permet d'accéder aux versions historiques des données.
- **Gestion des Métadonnées** : Optimisée pour les grandes tables.
- **Performance** : Amélioration des requêtes grâce à des optimisations comme Z-Ordering.

Use case 10 – Conversions de Tables

Étapes pour convertir une table existante

Table Parquet → Table Delta :

```
spark.read.format("parquet").load("/wytasoft/path/to/parquet_table") \
    .write.format("delta").save("/wytasoft/path/to/delta_table")
```

Table Hive → Table Delta :

```
CONVERT TO DELTA 'hive_table_location';
```

Ajouter les fichiers existants au journal transactionnel Delta :

```
CONVERT TO DELTA parquet.`/wytasoft/path/to/parquet_table`;
```

Use case 11 – Delta & Iceberg

Critère	Delta Lake	Apache Iceberg
Modèle transactionnel	Journal de transactions (<i>transaction log</i>) propriétaire (JSON + Parquet). ACID garanti via Spark nativement.	Format de métadonnées en JSON/Avro/Parquet. ACID via snapshots atomiques, indépendant du moteur d'exécution.
Formats supportés	Parquet exclusivement pour le stockage des données ; JSON + Parquet pour le log.	Parquet, ORC, Avro (et bientôt Iceberg-native Arrow) pour les données ; metadata en Avro/Parquet/JSON.
Time travel	Historisation via le transaction log : VERSION AS OF et TIMESTAMP AS OF.	Snapshots immuables : AT SNAPSHOT, AS OF timestamp et AS OF numéro de snapshot.
Évolution de schéma	Support de l'évolution des colonnes (ajout, enlèvement) ; options strictes (FAIL ou IGNORE) ou permissives.	Évolution riche : ajout, suppression, renommage, changement de type. Validation forte (no data-loss implicit).
Partitionnement	Partitionnement Spark traditionnel + clustering via Z-Order (OPTIMIZE ... ZORDER BY).	Partitionnements dynamiques sans réécriture ; "hidden partitioning" et "partition spec" versionné.
Optimisations I/O	OPTIMIZE (compaction) + Z-Order pour data skipping ; statistics min/max dans le log.	Pruning par metadata (min/max, metrics) et index de position des fichiers ; rewrite data pour compaction.
Catalogue et intégration	Intégration native Databricks/Hive Metastore/Unity Catalog ; API Spark, Python, Scala, Java.	Fonctionne avec Hive Metastore, AWS Glue, Nessie, Dremio, Flink, Trino ; API Java, Python, Spark, Flink.
Streaming	Source et sink Structured Streaming intégrés avec exactly-once.	Supporté via Spark (Structured Streaming) et Flink (Table API) ; exactly-once selon moteur.
Communauté & maturité	Forte adoption sur Databricks et AWS (via Lakehouse) ; gouverné par DLS.	Projet Apache incubé en 2020, adoption croissante chez Netflix, Apple, AWS ; large écosystème d'intégrations.
Fonctionnalités avancées	Fonctions Delta : VACUUM, RESTORE, HISTORY, DESCRIBE DETAIL, CHANGE DATA FEED (CDF).	Time travel, rollback, row-level deletes/upserts, partition evolution, hidden partition projections.

06

Spark Streaming



Spark Streaming



1. Spark Streaming - Part 1



2. Spark Streaming - Part 2

Spark Streaming

Spark Streaming - Part 1



Spark Streaming - Part 1



1. Introduction Spark Streaming



2. Structured Streaming



3. Streaming DataFrame



4. readStream



5. Transformation



6. window & Kafka



7. writeStream



9. foreachBatch

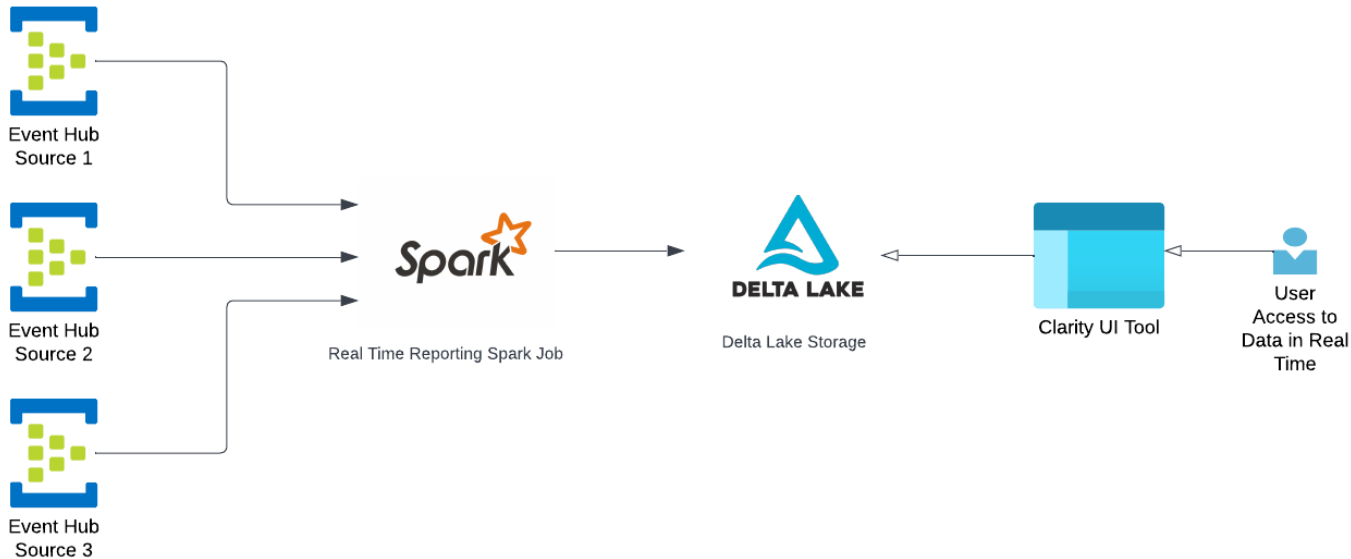


10. Intégration

Introduction au Traitement en Streaming

- Définition du streaming de données.
- Différence entre le traitement batch et le streaming.
- Présentation de **Spark Streaming** et **Structured Streaming**.

Architecture du Traitement en Streaming avec Spark



Exemple de Code pour Spark Structured Streaming

Spark read

```
# Exemple de lecture d'un flux Kafka
df = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

# Exemple de transformation en streaming
df_transformed = df.selectExpr("CAST(key AS STRING)",
    "CAST(value AS STRING)") \
    .filter(df.value.contains("important"))

# Exemple d'écriture dans un stockage
df_transformed.writeStream \
    .format("parquet") \
    .option("path", "/path/to/output") \
    .start()
```

- Exemple de chargement de données depuis un flux Kafka.
- Exemple de transformation continue des données (filtrage, agrégation).
- Exemple d'écriture des résultats dans une base de données ou un stockage cloud

Use case 1 – Spark Structured Streaming

Qu'est-ce que Spark Structured Streaming ?

- API de streaming évolutive et facile à utiliser.
- Basée sur le moteur Spark SQL, unifie batch et streaming.
- Garantit la **faible latence** et le traitement **en micro-batch** ou continu.

Use case 1 – Architecture de Spark Structured Streaming

- Source de données: Kafka, fichiers, S3, etc.
- Transformations: Map, Filter, Aggregations.
- Écriture des résultats vers: HDFS, Kafka, bases de données.
- Modèle de traitement: **Micro-batch** ou **Continuous Processing**.

Use case 1 – Principales Caractéristiques

- **Traitement Faiblement Couplé:** Pas besoin de maintenir des serveurs de streaming dédiés.
- **Stateful Processing:** Gestion de l'état, comme les fenêtres temporelles.
- **Garantie d'exactitude:** Exactly-once semantics.
- **Scalabilité:** Gestion dynamique des ressources.

Use case 1 – Principales Caractéristiques

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Structured Streaming Example") \
    .getOrCreate()

# Lecture depuis une source Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic") \
    .load()

# Transformation des données
transformed_df = df.selectExpr("CAST(value AS STRING) as json")

# Écriture dans une table Delta
transformed_df.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "/wytasoft/checkpoint") \
    .start("/wytasoft/delta_table")
```

- Lecture depuis une source Kafka.
- Transformation des données.
- Écriture des résultats vers une table Delta.

Use case 1 – Avantages de Spark Structured Streaming

- Unification du batch et du streaming.
- Intégration native avec Delta Lake pour le stockage et la gestion des données.
- Hautement scalable, conçu pour fonctionner avec de grands volumes de données.
- APIs familières pour les DataFrames et SQL.

Use case 2 – Streaming DataFrame

Qu'est-ce qu'un Streaming DataFrame ?

- Extension des DataFrames pour traiter les flux de données en continu.
- Intègre des API simples et familières pour les développeurs Spark.

Use case 2 – Streaming DataFrame

- **Source de données:** Kafka, fichiers, socket, etc.
- **Transformations:** Opérations sur colonnes, agrégations, filtres.
- **Écriture:** Bases de données, Delta Lake, systèmes de fichiers.
- **Traitement:** Micro-batch ou mode continu.

Use case 2 – Streaming DataFrame



```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Streaming DataFrame Example") \
    .getOrCreate()

# Lecture en continu depuis Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic") \
    .load()

# Transformation et affichage
query = df.selectExpr("CAST(value AS STRING) as message") \
    .writeStream \
    .outputMode("append") \
    .format("console") \
    .start()
query.awaitTermination()
```

- Exemple de lecture en continu depuis Kafka.
- Transformation et affichage des données.

Use case 2 – Streaming DataFrame

Micro-batch:

- Traitement par lots sur des intervalles définis.
- Idéal pour les systèmes nécessitant un certain délai de latence.

Continuous Processing:

- Latence ultra-faible, traitement ligne par ligne.
- Conçu pour des cas critiques en temps réel.

Use case 2 – Meilleures Pratiques

- Toujours spécifier un checkpoint pour garantir la tolérance aux pannes.
- Optimiser les ressources pour le traitement en continu.
- Tester le pipeline avec des flux simulés avant déploiement.

Use case 3 – readStream

Qu'est-ce que readStream ?

- Une méthode Spark pour consommer des données en continu.

Sources supportées par readStream:

- **Kafka:** Consommation de messages en temps réel.
- **Sockets:** Flux de données directs depuis une adresse IP/port.
- **Fichiers:** Lecture incrémentale des données ajoutées à un dossier.
- **Delta Lake:** Lecture des mises à jour continues sur une table Delta.

Use case 3 – readStream

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Kafka ReadStream Example") \
    .getOrCreate()

kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

kafka_df.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING)") \
    .writeStream \
    .format("console") \
    .start()
```

Paramètres essentiels:

- kafka.bootstrap.servers: Serveur Kafka.
- subscribe: Topic Kafka à consommer.

Format: key, value, topic, partition, etc.

Use case 3 – readStream



```
socket_df = spark.readStream \  
    .format("socket") \  
    .option("host", "localhost") \  
    .option("port", 9999) \  
    .load() \  
  
socket_df.writeStream \  
    .format("console") \  
    .start()
```

Paramètres :

- **host**: Adresse IP ou nom de domaine du serveur socket.
- **port**: Port sur lequel le flux est disponible.

Use case 3 – readStream

Configurer le checkpointing:

- Exemple: `option("checkpointLocation", "/path/to/checkpoint")`
- Importance: Garantit la tolérance aux pannes.

Gérer les volumes de données:

- Kafka: Ajuster `maxOffsetsPerTrigger` pour limiter les messages consommés.
- Sockets: Surveiller la latence pour les flux rapides.

Tester avec des données simulées:

- Exemple: Utilisation d'un serveur Kafka local ou de scripts pour envoyer des messages.

Use case 4 – Transformation de Flux

Offset Management:

- Kafka conserve les messages par partition, et chaque partition a un offset unique.
- Spark suit les offsets pour garantir que chaque message est traité une seule fois, même en cas de panne.

Spécificité:

- Contrairement aux transformations traditionnelles, les transformations sur Kafka doivent tenir compte des offsets pour assurer une continuité des flux et éviter des doublons.

```
kafka_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .option("startingOffsets", "earliest") \
    .load()
```

Use case 4 – Transformation de Flux

Les messages Kafka sont souvent non structurés (par exemple, JSON, Avro, ou texte brut). Cela nécessite des transformations supplémentaires pour extraire et structurer les données avant analyse.



```
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StringType

schema = StructType().add("key", StringType()).add("value", StringType())
structured_df = kafka_df.selectExpr("CAST(value AS STRING)") \
    .withColumn("data", from_json(col("value"), schema))
```

Use case 4 – Transformation de Flux

- Les opérations impliquant un shuffle, comme groupBy ou join, peuvent provoquer une latence élevée en streaming.
- Spark peut traiter chaque micro-batch en parallèle, mais les partitions doivent être équilibrées.



```
grouped_df = kafka_df.selectExpr("CAST(key AS STRING)", "CAST(value AS  
STRING)").groupBy("key").count()
```


Use case 4 – Transformation de Flux

Les opérations comme les **joins** entre deux flux en temps réel (stream-stream joins) nécessitent des fenêtres temporelles, ce qui peut augmenter la complexité.



```
from pyspark.sql.functions import window

windowed_df = kafka_df \
    .groupBy(window(col("timestamp"), "10 minutes"), col("key")) \
    .count()
```

Use case 4 – Transformation de Flux

- Kafka peut transmettre des messages en désordre (latence réseau, répliquions).
- Spark permet de gérer cela via des fenêtres avec **watermarks** pour éviter de traiter des messages trop anciens.



```
watermarked_df = kafka_df \  
    .withWatermark("timestamp", "10 minutes") \  
    .groupBy("key").count()
```

Use case 5 – window & Kafka

Objectif des fenêtres temporelles :

- Agréger des événements dans des plages de temps définies.
- Exemple : Nombre de ventes par minute.

Fonctionnalité Spark Structured Streaming :

- La fonction window permet de créer des fenêtres glissantes ou fixes.

Use case 5 – window & Kafka

Fenêtres fixes :

- Exemple : Une fenêtre de 5 minutes où les données sont regroupées uniquement par blocs de 5 minutes.

Fenêtres glissantes :

- Exemple : Une fenêtre de 5 minutes avec un décalage de 1 minute.

Fenêtres basées sur des sessions :

- Regroupement d'événements par périodes d'inactivité.

Use case 5 – window & Kafka

```
from pyspark.sql.functions import window, count

# Lecture d'un flux Kafka
kafka_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

# Transformation : Application de fenêtres temporelles
windowed_count = kafka_stream \
    .selectExpr("CAST(value AS STRING)", "timestamp") \
    .groupBy(window("timestamp", "5 minutes", "1 minute")) \
    .agg(count("value").alias("count"))

# Écriture des résultats
windowed_count.writeStream \
    .format("console") \
    .outputMode("complete") \
    .start()
    .awaitTermination()
```

- Lecture d'un flux Kafka avec Spark.
- Utilisation de window pour regrouper les données dans des fenêtres temporelles.

Use case 5 – window & Kafka

- Problème des données retardataires dans les flux.
- Utilisation de with **Watermark** pour gérer les événements arrivant en retard.Exemple d'application.



```
windowed_with_watermark = kafka_stream \  
    .selectExpr("CAST(value AS STRING)", "timestamp") \  
    .withWatermark("timestamp", "10 minutes") \  
    .groupBy(window("timestamp", "5 minutes", "1 minute")) \  
    .agg(count("value").alias("count"))
```

Use case 5 – window & Kafka

- Optimisation du nombre de partitions avec **repartition**.
- Réduction des shuffles grâce au broadcast.
- Ajustement des configurations Spark :
 - `spark.sql.shuffle.partitions`.
 - Compression des shuffles (**`spark.shuffle.compress`**).

Use case 6 – writeStream

- **Objectif** : Permet l'écriture continue des résultats d'un flux dans une destination.
- **Destinations supportées** :
 - Console (pour le débogage).
 - Systèmes de fichiers (CSV, JSON, Parquet).
 - Bases de données (JDBC).
 - Kafka, Elasticsearch, etc.
- **Fonctionnalité clé** : Gère les états intermédiaires automatiquement.

Use case 6 – writeStream



```
streaming_df.writeStream \  
  .format("format") \  
  .option("key", "value") \  
  .start("wytasoft/to/destination") \  
  .awaitTermination()
```

- **format** : Destination (console, parquet, kafka).
- **outputMode** :
 - append : Écrit uniquement les nouvelles lignes.
 - update : Met à jour les lignes existantes.
 - complete : Réécrit tout le tableau.
- **checkpointLocation** : Chemin pour enregistrer l'état de traitement.

Use case 6 – writeStream

Intégration native avec Kafka.

Cas d'utilisation :

```
streaming_df.writeStream \  
  .format("kafka") \  
  .option("kafka.bootstrap.servers", "localhost:9092") \  
  .option("topic", "output_topic") \  
  .option("checkpointLocation", "/wytasoft/to/checkpoint") \  
  .start()
```

- Propagation de données transformées.
- Chaîne de traitement distribué.

Use case 6 – writeStream

- **Objectif** : Permet l'écriture continue des résultats d'un flux dans une destination.
- **Destinations supportées** :
 - Console (pour le débogage).
 - Systèmes de fichiers (CSV, JSON, Parquet).
 - Bases de données (JDBC).
 - Kafka, Elasticsearch, etc.
- **Fonctionnalité clé** : Gère les états intermédiaires automatiquement.

Use case 7 – foreachBatch

- **Qu'est-ce que foreachBatch ?**
 - Une méthode de Spark Structured Streaming.
 - Permet un traitement par lots sur les données en streaming.
- **Objectif :**
 - Appliquer une logique personnalisée sur chaque micro-batch.
 - Intégration avec des systèmes externes ou transformations avancées.

Use case 7 – foreachBatch

- **Flexibilité :**
 - Appliquer des transformations spécifiques à chaque batch.
 - Écrire simultanément vers plusieurs sinks (destinations).
- **Efficacité :**
 - Évite l'écriture directe des résultats en streaming vers des sinks lents.
- **Intégration :**
 - Compatible avec JDBC, Parquet, Delta Lake, et autres formats.

Use case 7 – foreachBatch

- **Flux de Données :**
 - Les données en streaming sont divisées en micro-batches.
 - Chaque micro-batch est traité comme un DataFrame.
 - foreachBatch applique une logique personnalisée sur chaque lot.
- **Traitement Parallèle :**
 - Exploite l'architecture distribuée de Spark pour une grande échelle.

Use case 7 – foreachBatch

```
from pyspark.sql import SparkSession

# Création de la SparkSession
spark = SparkSession.builder \
    .appName("Streaming avec foreachBatch") \
    .getOrCreate()

# Lecture depuis Kafka
streaming_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

# Définir la fonction foreachBatch
def write_to_sink(batch_df, batch_id):
    batch_df.write \
        .format("jdbc") \
        .option("url", "jdbc:mysql://localhost:3306/database") \
        .option("dbtable", "table_name") \
        .option("user", "username") \
        .option("password", "password") \
        .mode("append") \
        .save()

# Utiliser foreachBatch
streaming_query = streaming_df.writeStream \
    .foreachBatch(write_to_sink) \
    .start()

streaming_query.awaitTermination()
```

- **Flux de Données :**
 - Les données en streaming sont divisées en micro-batches.
 - Chaque micro-batch est traité comme un DataFrame.
 - `foreachBatch` applique une logique personnalisée sur chaque lot.
- **Traitement Parallèle :**
 - Exploite l'architecture distribuée de Spark pour une grande échelle.

Use case 7 – foreachBatch

- **Optimisation des opérations sur les sinks** : Minimise la latence.
- **Gestion des erreurs** : Traitez les erreurs de manière batch-spécifique.
- **Idempotence** : Garantissez que chaque batch est écrit une seule fois sans doublons.
- **Surveillance** : Utilisez Spark UI pour analyser les performances.

Use case 8 – Intégration avec d'autres outils

- Kafka comme plateforme centrale de gestion de flux de données.
- Pourquoi l'intégration avec HBase ou Cassandra ?
 - **HBase** : Stockage distribué de données structurées.
 - **Cassandra** : Bases de données NoSQL optimisées pour les écritures rapides.
- Avantages :
 - Traitement en temps réel.
 - Stockage et récupération des données pour des analyses ultérieures.

Use case 8 – Intégration avec d'autres outils

- **Cas d'utilisation :**
 - Archivage de flux Kafka dans HBase.
 - Accès rapide aux données historiques pour l'analyse.
- **Étapes techniques :**
 - Consommer des messages Kafka avec Spark Streaming.
 - Stocker les messages dans HBase à l'aide de connecteurs.

```
val kafkaDf = spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("subscribe", "topic_name")
  .load()

kafkaDf.write
  .format("org.apache.hadoop.hbase.spark")
  .option("hbase.table", "my_table")
  .save()
```

Use case 8 – Intégration avec d'autres outils

- **Cas d'utilisation :**
 - Stockage des données Kafka dans Cassandra pour des analyses rapides.
 - Surveillance des applications et tableaux de bord en temps réel.
- **Étapes techniques :**
 - Lire les messages Kafka.
 - Écrire les données directement dans Cassandra à l'aide de Spark Cassandra Connector.

```
val kafkaDf = spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "localhost:9092")
    .option("subscribe", "topic_name")
    .load()

kafkaDf.write
    .format("org.apache.spark.sql.cassandra")
    .options(Map("keyspace" -> "my_keyspace", "table" -> "my_table"))
    .save()
```

Use case 8 – Intégration avec d'autres outils

- Flux Kafka comme source de données centralisée.
- **HBase** pour l'archivage des données et l'accès aux historiques.
- **Cassandra** pour des analyses rapides et des réponses interactives.
- Workflow typique :
 - Kafka produit des données en continu.
 - Spark consomme les données pour les transformer.
 - Les données sont stockées dans HBase pour un accès historique ou dans Cassandra pour des analyses rapides.

Use case 8 – Intégration avec d'autres outils

- **Évolutivité** : Kafka gère les flux massifs, tandis que HBase et Cassandra gèrent efficacement les données en volume.
- **Flexibilité** : Architecture modulaire pouvant s'adapter à différents besoins.
- **Performance** : Traitement des données en temps réel, stockage rapide et récupération optimisée.
- **Fiabilité** : Combinaison de Kafka pour la résilience et des bases distribuées pour la tolérance aux pannes.

Quiz : Ingestion des données

<https://www.wytasksoft.com/wytasksoft-group/index.php/wytasksoftacademy/>



Spark Streaming

Spark Streaming - Part 2



Spark Streaming - Part 2



1. **Accumulateurs & Broadcast Variables**



2. **Options de Streaming**



3. **checkpointLocation**

4. **Fault Tolerance**



6. **Fréquence Trigger**



7. **awaitTermination**



9. **createOrReplace TempView**

Use case 1 – Accumulateurs & Broadcast Variables

- **Accumulateurs** : Utilisés pour effectuer des agrégations globales à travers plusieurs tâches Spark.
- **Broadcast Variables** : Permettent de partager des données immuables avec les nœuds d'exécuteurs, évitant les duplications.

Use case 1 – Accumulateurs

- **Définition :** Variables utilisées pour agréger des données à travers plusieurs tâches.
- **Cas d'utilisation :**
 - Compter les événements dans des flux en temps réel.
 - Identifier les erreurs ou anomalies dans les données.



```
from pyspark import AccumulatorParam

acc_errors = spark.sparkContext.accumulator(0)

rdd.foreach(lambda record: acc_errors.add(1) if record.error else None)
```

Use case 1 – Broadcast Variables

- **Définition :** Variables partagées immuables distribuées efficacement entre les nœuds.
- **Cas d'utilisation :**
 - Partager des tables de référence ou des modèles ML avec tous les nœuds.
 - Éviter les duplications inutiles des données.



```
broadcast_rules = spark.sparkContext.broadcast({"rule1": "value1", "rule2": "value2"})  
rdd.map(lambda record: broadcast_rules.value.get(record.key))
```

Use case 1 – Accumulateurs & Broadcast Variables

Accumulateurs :

1. Compter les événements par type dans un flux en temps réel.
2. Identifier les anomalies dans les messages Kafka.

Broadcast Variables :

1. Partager des modèles ML pour effectuer des prédictions en temps réel.
2. Diffuser des configurations ou des tables de référence.

Use case 1 – Accumulators & Broadcast Variables



```
# Broadcast variable
broadcast_rules = spark.sparkContext.broadcast({"error": "INVALID"})

# Accumulator
error_count = spark.sparkContext.accumulator(0)

# Process streaming data
streaming_df = spark.readStream.format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "topic_name") \
    .load()

processed_df = streaming_df.map(lambda record: error_count.add(1) if record.value ==
broadcast_rules.value["error"] else None)
```

Use case 1 – Accumulateurs & Broadcast Variables

- Limiter la taille des broadcast variables pour éviter les dépassements de mémoire.
- Utiliser les accumulateurs uniquement pour des opérations commutatives et associatives.
- Toujours vérifier que les accumulateurs et les broadcast variables sont correctement initialisés avant utilisation.

Use case 2 – Options de Streaming

- Spark Structured Streaming offre des options pour configurer et optimiser le traitement des flux.
- Deux options principales :
 - **queryName:** Donne un nom unique à une requête de streaming pour faciliter le suivi.
 - **trigger:** Définit la fréquence à laquelle les données sont traitées.

Use case 2 – queryName

- Pourquoi utiliser queryName ?
 - Identifier et suivre les requêtes actives dans Spark UI.
 - Faciliter le diagnostic en cas de problème.



```
streaming_query = streaming_df.writeStream \  
    .queryName("KafkaToDeltaProcessing") \  
    .format("delta") \  
    .outputMode("append") \  
    .start("/wytasoft/to/delta_table")
```


Use case 2 – trigger



```
streaming_query = streaming_df.writeStream \  
  .trigger(processingTime="10 seconds") \  
  .format("console") \  
  .outputMode("append") \  
  .start()
```

Pourquoi utiliser trigger ?

- Contrôle le moment où les nouvelles données sont traitées.
- Deux principaux modes :
 - **Fixed-interval:** Spécifie un intervalle fixe (exemple : toutes les 10 secondes).
 - **Once:** Traite les données disponibles une seule fois, idéal pour les scénarios de batch.

Use case 2 – trigger

- **No Trigger (par défaut) :**
 - Les données sont traitées dès qu'elles arrivent.
- **Fixed Interval Trigger :**
 - Permet de spécifier un intervalle régulier pour le traitement.
 - Exemple : `trigger(processingTime="15 seconds")`.
- **Once Trigger :**
 - Traite toutes les données disponibles une fois, puis s'arrête.
- **Continuous Trigger (Spark >= 3.0):**
 - Tentative de traitement des données aussi rapidement que possible.
 - Exemple : `trigger(continuous="1 second")`.

Use case 2 – Bonnes Pratiques pour queryName et trigger

- **Nommer toutes les requêtes avec queryName :**
 - Facilite le suivi dans Spark UI.
 - Aide à distinguer les requêtes multiples dans les logs.
- **Utiliser trigger pour optimiser les ressources :**
 - Réduire la fréquence pour économiser des ressources.
 - Augmenter la fréquence pour réduire la latence.
- **Éviter les triggers trop courts :**
 - Peut surcharger le cluster avec des micro-batches trop fréquents.

Use case 3 – checkpointLocation

- Pourquoi un checkpoint ?
 - Reprise des flux après un échec.
 - Stockage des métadonnées, comme le dernier offset consommé.



```
streaming_df.writeStream \  
  .format("parquet") \  
  .outputMode("append") \  
  .option("checkpointLocation", "/wytasoft/checkpoints/stream_checkpoint") \  
  .start("/wytasoft/data/output")
```

Use case 3 – checkpointLocation

Exemple complet intégrant `partitionBy`, `path`, et `checkpointLocation` :



```
streaming_df.writeStream \  
  .format("parquet") \  
  .outputMode("append") \  
  .partitionBy("year", "month") \  
  .option("checkpointLocation", "/wytasoft/checkpoints/stream_checkpoint") \  
  .start("/wytasoft/data/output")
```

Use case 4 – Fault Tolerance

- **Défi principal** : Les pipelines de streaming doivent gérer les pertes de données potentielles (ex. : pannes réseau, partitions manquantes, erreurs d'écriture).
- **Solution Spark** : L'option ***failOnDataLoss*** pour contrôler le comportement face à une perte de données.

Use case 4 – Fault Tolerance

Contrôle si Spark doit échouer en cas de perte de données dans une source de streaming.

Paramètre par défaut : true.

- Spark stoppe la requête lorsqu'il détecte une perte de données.

Impact lorsque false :

- Spark ignore la perte de données et continue l'exécution.



```
spark.readStream \  
  .format("kafka") \  
  .option("failOnDataLoss", "false") \  
  .load()
```

Use case 4 – Fault Tolerance

Quand utiliser `failOnDataLoss=true` :

- Scénarios critiques où aucune perte de données n'est acceptable (ex. : audits, analyses réglementaires).

Quand utiliser `failOnDataLoss=false` :

- Scénarios où la continuité du traitement est prioritaire (ex. : suivi en temps réel avec tolérance aux lacunes).

Impact de chaque choix :

- `true` : Robustesse accrue, mais interrompt les pipelines.
`false` : Flux ininterrompu, mais avec risque d'analyses biaisées.

Use case 4 – Fault Tolerance

Scénario : Si des partitions Kafka sont supprimées ou inaccessibles :

- **Avec true** : Spark arrête la requête.
- **Avec false** : Spark continue sans ces données.

```
streaming_df = spark.readStream \  
    .format("kafka") \  
    .option("kafka.bootstrap.servers", "localhost:9092") \  
    .option("subscribe", "topic_name") \  
    .option("failOnDataLoss", "true") \  
    .load()
```

Use case 5 – Fréquence Trigger

Trigger : Définit la fréquence à laquelle Spark traite les données en streaming.

Modes de trigger disponibles :

- `Trigger.ProcessingTime("interval")` : Définit un intervalle fixe pour traiter les données (ex. : toutes les 10 secondes).
- `Trigger.Once` : Traite toutes les données disponibles et arrête le traitement.
- `Trigger.Continuous("interval")` : Active un streaming continu (bêta).

```
streaming_df.writeStream \  
  .format("console") \  
  .trigger(processingTime="10 seconds") \  
  .start()
```

Use case 7 – awaitTermination

awaitTermination : Bloque le processus principal tant que la requête de streaming est active.

Pourquoi c'est utile :

- Garantit que le streaming reste opérationnel.
- Empêche la fin prématurée de l'application Spark.



```
streaming_query = streaming_df.writeStream \  
    .format("console") \  
    .trigger(processingTime="10 seconds") \  
    .start()  
  
# Maintient le streaming actif  
streaming_query.awaitTermination()
```

Use case 8 – createOrReplaceTempView



```
streaming_df.createOrReplaceTempView("streaming_view")
result_df = spark.sql("""
    SELECT category, COUNT(*) AS count
    FROM streaming_view
    GROUP BY category
""")
```

createOrReplaceTempView(view_name):

- Enregistre un DataFrame comme vue temporaire.
- Accessible uniquement dans la session Spark active.

05

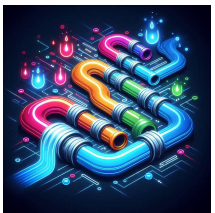
Overview Spark Advanced / Internals



Overview Spark **Advanced / Internals**



1. Monitoring : Yarn UI



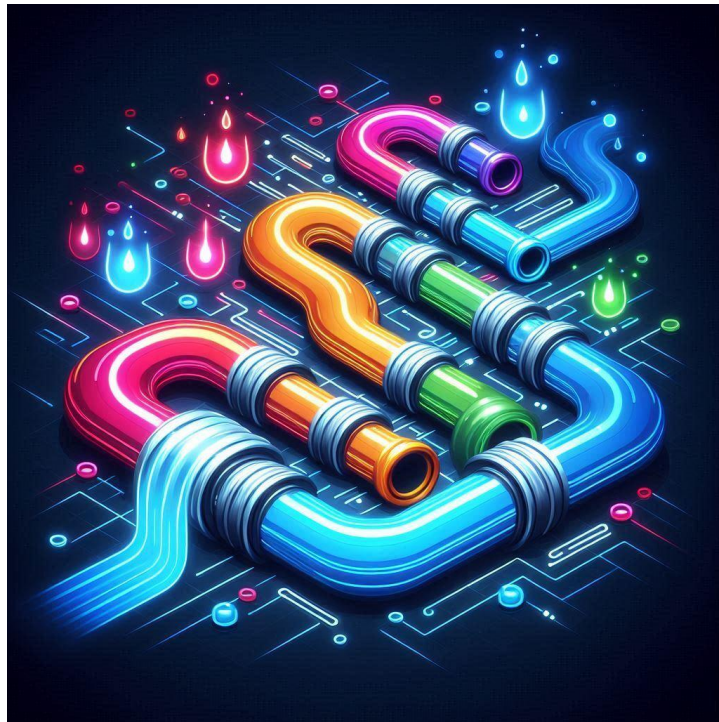
2. Paramètres et modalités d'exécution de programmes Spark

Overview Spark

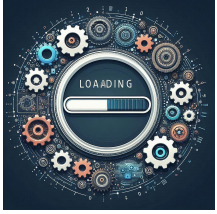
Advanced / Internals

Monitoring :

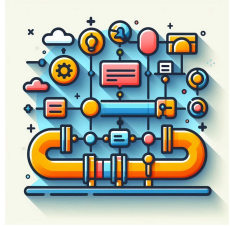
Yarn UI



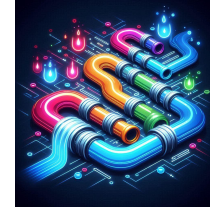
Monitoring: Yarn UI



1. Yarn Overview

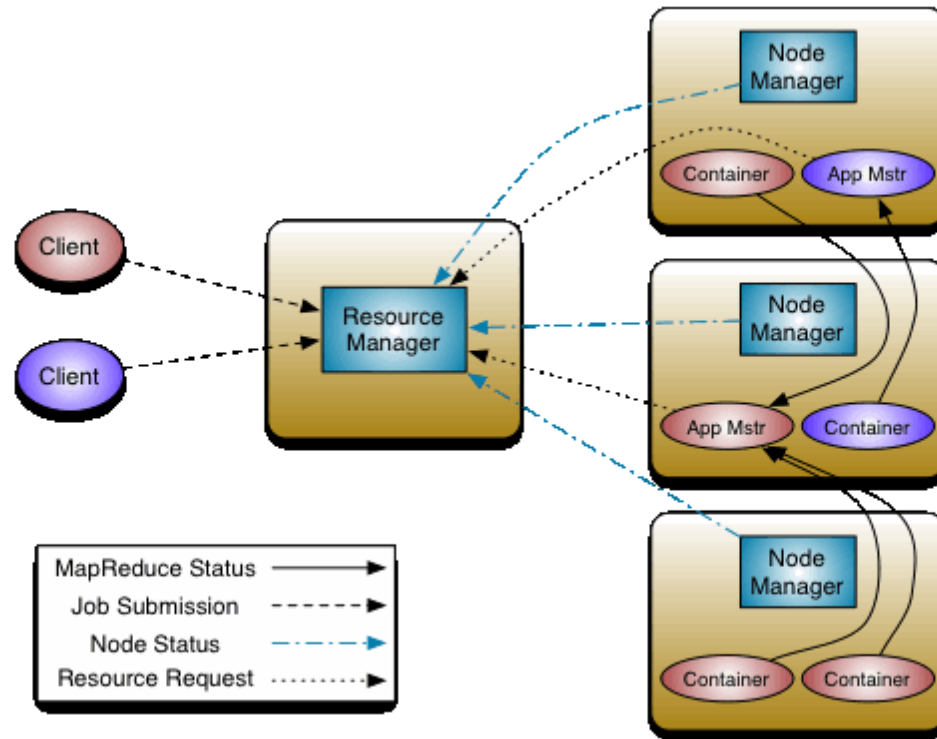


2. Logs d'Application



3. Diagnostic

Le Rôle YARN



YARN est le gestionnaire de ressources dans l'écosystème Hadoop

1. Gestion des ressources
2. Planification des tâches
3. Isolation et sécurité
4. Suivi et surveillance

Les Queues & les Applications YARN

Concept de Queues dans YARN : Les queues dans YARN sont des **structures logiques** utilisées pour organiser, isoler et gérer les ressources pour différentes applications ou groupes d'utilisateurs

Types de Planificateurs dans YARN :

- 1.FIFO Scheduler
- 2.Capacity Scheduler
- 3.Fair Scheduler

Focus sur la Queue d'exécution de YARN

- **Qu'est-ce qu'une Queue YARN ?**
 - Système de gestion des ressources dans Hadoop pour organiser et prioriser les tâches.
 - Permet de distribuer les ressources entre différentes applications selon les priorités définies.
- **Types de Queues :**
 - **Queues hiérarchiques :** Organisation en arborescence pour séparer les ressources par département, équipe ou type d'application.
 - **Priorités des Queues :** Contrôle de la priorité d'exécution pour des tâches critiques vs. non critiques.
- **Importance de la Queue dans l'optimisation des ressources :**
 - Amélioration de la répartition de la charge de travail, évitant les conflits de ressources.
 - Assure une allocation équitable et optimale des ressources CPU et mémoire selon les besoins des applications.
- **Monitoring et Ajustement :**
 - Suivi en temps réel des files d'attente pour adapter les ressources si nécessaire.
 - Utilisation des logs et métriques pour ajuster dynamiquement les priorités et les quotas des queues.

Configuration des **Queues** pour la Priorisation

Définition des Capacités : Chaque queue se voit attribuer une capacité minimale (et maximale) en pourcentage des ressources totales du cluster.

- **Priorités au sein des Queues**
- **Politiques de Prémption**
- **Accès Contrôlé**

Vue d'ensemble des Applications

YARN

- ID de l'application, nom et utilisateur.
- Queue d'exécution.
- État de l'application : (RUNNING, FINISHED, etc.)
- Progression de l'exécution.
- Type d'application : SPARK, MAPREDUCE, etc.
- Temps de démarrage et de fin.
- Utilisation de la mémoire et du CPU.

État de l'Application dans YARN

- Les différents états d'une application :
 - **ACCEPTED** : Application en attente d'assignation de ressources.
 - **RUNNING** : Application en cours d'exécution.
 - **FINISHED** : Application terminée avec succès.
 - **FAILED** : Application ayant échoué durant l'exécution.
 - **KILLED** : Application arrêtée manuellement ou en raison d'un manque de ressources.

Logs d'Application YARN

- **Accès aux logs** : YARN offre un accès centralisé aux logs d'application, **Structure des logs** :
 - *Logs de l'ApplicationMaster*
 - *Logs des Exécuteurs (Executors)*
 - *Logs de l'erreur*
- **Récupération des logs via l'interface YARN**
- **Conservation et archivage des logs**

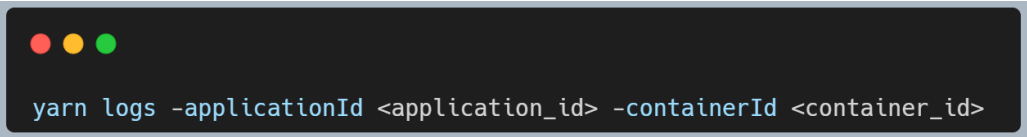
Récupération des Logs d'Application **YARN**

- Commandes de base pour accéder aux logs :

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The command `yarn logs -applicationId <application_id>` is displayed in a light blue monospace font.

```
yarn logs -applicationId <application_id>
```

- Obtenir les logs pour un conteneur spécifique :

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The command `yarn logs -applicationId <application_id> -containerId <container_id>` is displayed in a light blue monospace font.

```
yarn logs -applicationId <application_id> -containerId <container_id>
```


Yarn Cheat Sheet

yarn application -list
List running applications

yarn application -status <appid>
Status of an application

yarn queue -list
List all queues

yarn top
Track Resource Usage in Real Time

yarn application -kill <appid>
Kill an application by ID

yarn logs -applicationId <appid>
Retrieve application logs

yarn queue -status <queueName>
Status of a queue

yarn applicationattempt -list <Application_ID>
Application Resource Summary

Fetch logs for the most recently submitted application
with the specified name

```
yarn logs -applicationId  
$(yarn application -list | grep  
<application_name> | sort -  
rk5 | head -n 1 | awk '{print  
$1}')
```

yarn logs -user <username>
Get Logs by User

yarn container -list <appid>
List containers of an application

yarn container -status <containerId>
Status of a container

yarn scheduler
Cluster resource allocation

yarn application -kill <Application_ID>

yarn container -list <Application_ID>
yarn container -kill <Container_ID>

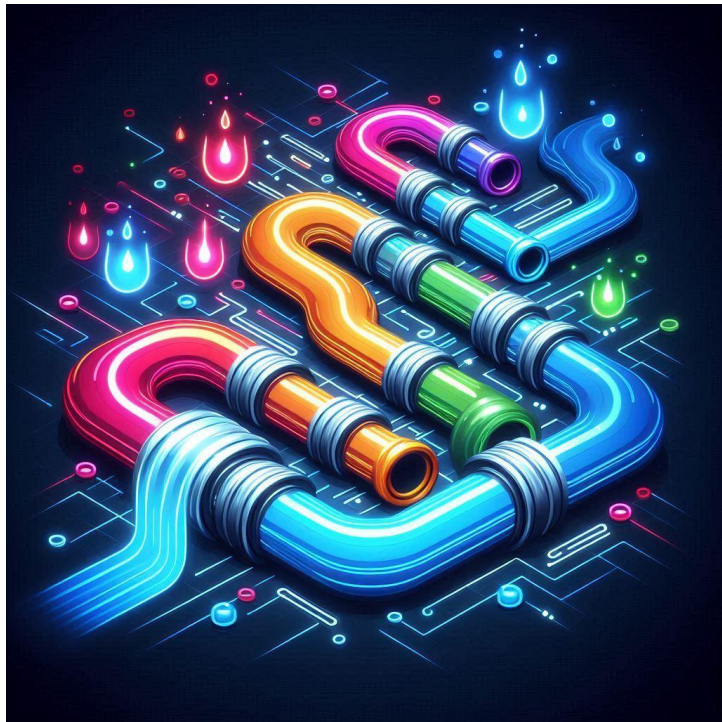
```
yarn application -list | awk '(!($5 >  
86400000) print $1)' | xargs -l {}  
yarn application -kill {}  
Kill Applications Running Over One  
Day
```

Overview Spark

Advanced / Internals

Overview

Paramètres et modalités d'exécution Spark



Paramètres et modalités d'exécution Spark



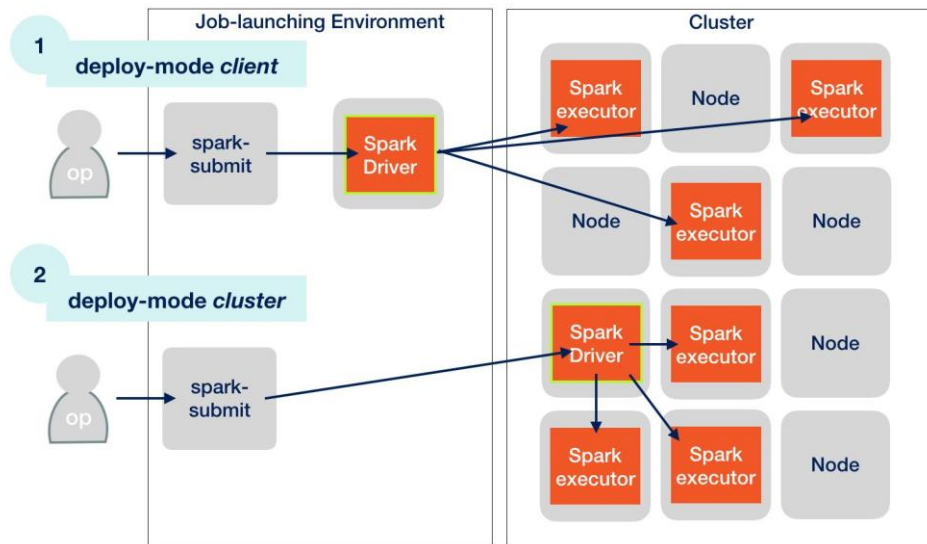
1. Mode de Déploiement



2. Commandes et Options Générales

Paramètres et modalités d'exécution Spark

Mode de Déploiement



Introduction aux Modes de Déploiement Spark

- Spark peut fonctionner dans différents modes de déploiement, selon les besoins de scalabilité et les ressources.
- Les trois principaux modes sont : **Local**, **Client**, et **Cluster**.

Mode Local

- **Description :** Exécution de Spark sur un seul nœud, utile pour les tests et le développement.
- **Avantages :**
 - Simple à configurer et à utiliser pour les développeurs.
 - Exécution rapide pour des tests ou des analyses légères.
- **Inconvénients :**
 - Limité par la mémoire et le CPU du nœud local, ne convient pas aux grandes charges de travail.

Mode Client

- **Description :** L'application Spark s'exécute depuis le client, tandis que les tâches sont distribuées aux nœuds du cluster.
- **Avantages :**
 - Permet un retour rapide au client sur les résultats intermédiaires.
 - Idéal pour les applications interactives et les analyses.
- **Inconvénients :**
 - Nécessite une bonne connexion entre le client et le cluster.
 - Peut entraîner une surcharge de la machine client si les ressources sont limitées.

Comparaison des Modes de Déploiement

Mode	Scénario Utilisation	Avantages	Inconvénients
Local	Tests & Développement	Simple, rapide	Limité par ressources locales
Client	Analyses interactives	Retour rapide	Dépend de la connexion
Cluster	Gros volumes de données	Très scalable	Complexité de mise en place

Paramètres et modalités d'exécution de programmes Spark

Commandes et Options Générales

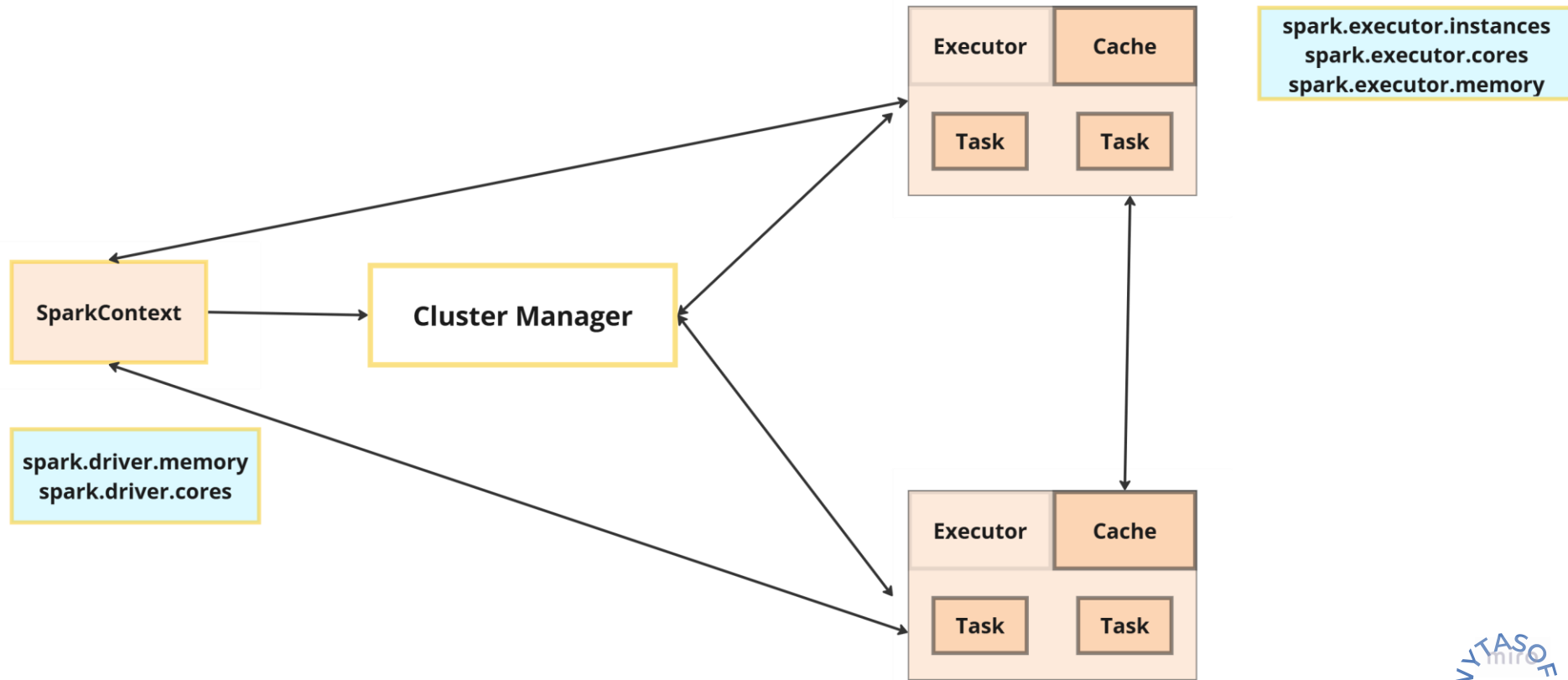


Overview spark-submit



```
spark-submit \  
  --class com.example.MainClass \  
  --master yarn \  
  --deploy-mode cluster \  
  --conf spark.executor.memory=2g \  
  --packages com.databricks:spark-avro_2.12:3.0.1 \  
  --jars /path/to/extra-library.jar \  
  --files /path/to/config.conf  
  /path/to/your-application.jar
```

Commandes et Options Générales



Paramètres du **Driver** Spark

- **spark.driver.cores**: Définit le nombre de cœurs attribués au driver Spark.
- **spark.driver.memory**: Alloue la mémoire principale pour le driver.
- **spark.driver.memoryOverhead**: Mémoire supplémentaire réservée pour les processus Java.
- **spark.driver.maxResultSize**: Taille maximale des résultats que le driver peut gérer.

Options pour la **journalisation** et les **exécuteurs**

- **spark.driver.extraJavaOptions & spark.executor.extraJavaOptions:** Ajout de configurations Java
- **Log4j.spark.executor.instances:** Nombre d'instances d'exécuteurs
- **spark.executor.cores:** Cœurs CPU alloués par exécuteur
- **spark.executor.memory:** Mémoire principale par exécuteur
- **spark.executor.memoryOverhead:** Mémoire supplémentaire par exécuteur, ici 4 Go.

Paramètres de Communication et Heartbeat

- **spark.executor.heartbeatInterval**: Intervalle de heartbeat pour vérifier l'activité
- **spark.network.timeout**: Timeout réseau
- **spark.scheduler.minRegisteredResourcesRatio**: Taux minimal de ressources pour démarrer.
- **spark.shuffle.io.maxRetries** & **spark.shuffle.io.numConnectionPerPeer**: Gèrent la connectivité pour le shuffle.
- **spark.logConf**: Active la journalisation des configurations Spark.

Paramètres Python et Divers

- `spark.python.worker.memory`: Mémoire dédiée aux processus Python.
- `spark.python.worker.reuse`: Permet de réutiliser les workers Python.
- `spark.jars`: Permet d'ajouter des JARs pour les dépendances.
- `spark.rpc.numRetries`: Nombre de tentatives pour les connexions RPC.
- `spark.task.maxFailures`: Nombre de tentatives maximales par tâche Spark.

Paramètres **YARN** spécifiques

- **spark.yarn.heterogeneousExecutors.enabled:**
Active/désactive les exécuteurs hétérogènes.
- **spark.yarn.max.executor.failures:** Limite de défaillances avant arrêt de l'application.
- **spark.yarn.am.memory & spark.yarn.am.memoryOverhead:**
Mémoire principale et mémoire supplémentaire pour l'ApplicationMaster.
- **spark.yarn.am.cores:** Cœurs CPU pour l'ApplicationMaster.
- **spark.yarn.am.extraJavaOptions:** Options Java supplémentaires pour l'ApplicationMaster.

Options supplémentaires de gestion de ressources et d'allocations

- **spark.dynamicAllocation.enabled:** Active ou désactive l'allocation dynamique des ressources.`spark.locality`.
- **wait:** Définit le temps d'attente pour obtenir des ressources locales.

À quoi servent les `--executor-cores`

- `--executor-cores` : indique combien de threads un executor Spark peut utiliser en parallèle pour exécuter des tâches. .
 - Chaque task Spark s'exécute dans un thread
 - Plus tu donnes de cores à un executor, plus il peut exécuter plusieurs tasks en parallèle.



Exemple visuel

Tu as 3 executors, chacun avec 4 cores :

→ Tu peux exécuter **12 tasks en parallèle**.

À quoi servent les `--executor-cores`

Paramètre	Impact
Trop peu de cores	L'executor travaille lentement, le parallélisme est sous-utilisé.
Trop de cores	Peut causer des contentions (mémoire/CPU) ou du gaspillage.
Bon nombre de cores	Exploite au mieux les ressources disponibles et accélère les traitements.

Optimisation logistique et énergétique via la prévision météo



Contexte métier

Une entreprise internationale de logistique opère dans plusieurs grandes villes du monde (Tokyo, Belgrade, Singapore, etc.). Afin d'améliorer la planification de ses activités, elle souhaite exploiter les données météorologiques historiques pour :

- **Optimiser l'acheminement des livraisons**, en évitant les conditions météorologiques défavorables telles que :
 - pics de chaleur,
 - pluies abondantes,
 - humidité élevée ou mauvaise visibilité.
 - **Réduire la consommation énergétique des bâtiments** (entrepôts, centres de tri), en anticipant les jours à forte sollicitation thermique (chauffage ou climatisation).
- Les données sont issues de **fichiers historiques AccuWeather** fournis par Databricks :

- `historical_daily_calendar_metric` (valeurs réelles observées)
- `forecast_daily_calendar_metric` (prévisions)

Calcul du z-score & détection des anomalies

`z_score_temp` : mesure combien d'écart-types une température s'éloigne de la moyenne.

Formule :

$$z = \frac{(x - \mu)}{\sigma}$$

Où x est la température, μ la moyenne, et σ l'écart-type.

Sentiment analysis

- **Contexte :**

- L'analyse de sentiment est cruciale pour les entreprises qui souhaitent comprendre ce que les clients pensent de leurs produits ou services.
- Avec l'explosion des réseaux sociaux et des plateformes de commentaires en ligne, les entreprises doivent être capables d'analyser ces données en temps réel pour réagir rapidement.
- Dans cet exercice, vous allez implémenter une solution qui utilise Spark Streaming pour ingérer des flux de commentaires en temps réel et appliquer un modèle de machine learning pré-entraîné (**DistilBERT**) pour détecter le sentiment (positif, négatif) de ces commentaires.

- **Objectifs :**

- Ingestion des flux de commentaires en temps réel à partir d'une source comme Kafka.
- Utilisation du modèle pré-entraîné DistilBERT de Hugging Face pour effectuer une analyse de sentiment.
- Affichage en temps réel des résultats (commentaire + sentiment) dans la console.

Sentiment analysis

Étapes :

1.Mise en place de l'environnement : Vous utiliserez Spark Streaming pour lire des flux de données provenant de Kafka.

2.Flux de données en temps réel : Un producteur Kafka va simuler des commentaires clients en temps réel.

3.Analyse sémantique avec DistilBERT : Vous intégrerez le modèle pré-entraîné DistilBERT pour prédire le sentiment des commentaires.

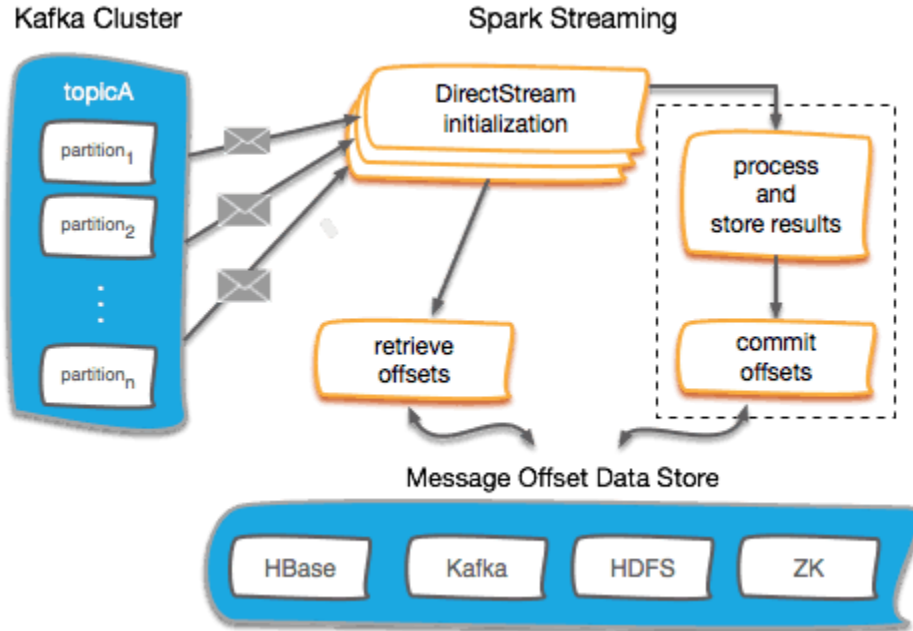
4.Sortie des résultats : Les résultats seront affichés en temps réel dans la console avec le sentiment associé (positif ou négatif).

Sentiment analysis

Livrables :

- Script Kafka pour simuler les commentaires clients.
- Script PySpark pour ingérer les commentaires en temps réel et appliquer le modèle de sentiment.
- Console affichant les résultats des commentaires avec leurs sentiments correspondants.

Architecture Générale



- Producteur Kafka : Envoi de commentaires en temps réel.
- Cluster Kafka : Réception et stockage temporaire des messages.
- Spark Streaming : Consommation des messages, analyse des sentiments et diffusion des résultats.
- Modèle DistilBERT : Prédiction des sentiments.

Producteur Kafka



```
def generate_comment(self):  
    return {  
        "comment": random.choice(self.comments),  
        "timestamp": datetime.now().isoformat()  
    }
```

- Utilisation de `KafkaProducer` pour envoyer des messages JSON.
- Génération de commentaires aléatoires.
- Envoi de chaque commentaire avec un timestamp.

Ingestion des Données dans Spark Streaming

Spark Streaming

```
comments_schema = StructType([
    StructField("comment", StringType(), True),
    StructField("timestamp", StringType(), True)
])

comments_df = spark.readStream \
    .format("kafka") \
    .option("subscribe", "comments") \
    .load()

comments_df_parsed =
comments_df.select(from_json(col("json_value"),
comments_schema).alias("data")).select("data.*")
```

- Connexion à Kafka avec readStream().
- Conversion des messages JSON en DataFrame.
- Schéma défini pour la structure des données.

Analyse de Sentiment avec DistilBERT

```
writeStream

comments_with_sentiment.writeStream \
  .outputMode("append") \
  .format("console") \
  .option("truncate", False) \
  .start()
```

- Chargement du modèle DistilBERT pré-entraîné.
- Création d'une UDF (User Defined Function) pour appliquer le modèle à chaque commentaire.
- Résultat : Label de sentiment ('POSITIVE' ou 'NEGATIVE').

```
DistilBERT

sentiment_model = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")
sentiment_udf = udf(lambda comment: sentiment_model(comment)[0]['label'], StringType())
comments_with_sentiment = comments_df_parsed.withColumn("sentiment", sentiment_udf(col("comment")))
```

- Mehdi TAJMOUATI
 - mehdi.tajmouati@wytasoft.com
 - <https://www.wytasoft.com/wytasoft-group/>
 - <https://www.linkedin.com/in/mtajmouati/>
 - <https://medium.com/@mehdi.tajmouati.wytasoft>
-

